

碳化硅与硅外延层的双角测厚与多光束检验

摘要

本文从反射光的明暗条纹中估算晶圆表面薄层的厚度，并判断多次反射是否需要改变测厚结果。碳化硅与硅分别拟合，同一晶圆在不同入射角下共享厚度，避免把角度造成的光谱差异当成厚度差异。

单次返回的条纹同时受厚度和材料折射特性影响。采用一次往返场展开建立两束干涉模型，保留折射率随波数变化的色散效应，以周期投影提取条纹间距。实测条纹间距折算的周期等效量为 $19.54\ \mu\text{m}$ ，其几何厚度换算的不确定性来自未测光学参数；合成锚例在光学、角度等条件扰动下的厚度重估范围为 $6.66\text{--}10.01\ \mu\text{m}$ 。

同片碳化硅的两角谱条纹位置接近，背景与振幅却有差别，因此采用双角色散消干扰模型，让两角共享厚度和色散。算法以变投影直接求出逐角背景、振幅，再从多个起点搜索物理参数。按不贴参数边界的代表点，碳化硅厚度为 $10.80\ \mu\text{m}$ ，连续留段、厚度与折射率组合及光学条件变化形成 $0.64\text{--}18.11\ \mu\text{m}$ 的非概率情景范围。

多次返回是否改变测厚，以同一波段内的两束与完整往返场对照判断。采用逐次往返厚度对照，将返回光场按几何级数累加，再以多起点约束优化求解。硅的全量拟合厚度为 $2.17\ \mu\text{m}$ ，区段、光学条件及拟合效果相近的候选厚度形成 $1.72\text{--}18.16\ \mu\text{m}$ 的非概率范围。两材料的高阶返回收益随区段反向，碳化硅沿用前问的代表厚度与范围。

稳健性检验区分合成测厚误差与实测光谱预测误差：匹配合成情景中，场反演的厚度平均绝对相对误差为 0.0008% ，常折射率峰距法为 2.59% ；硅连续留段中，完整往返相对两束的标准化反射率预测误差一折降低 25.4% ，另一折增加 9.52% 。这表明谱形改善随区段变化，测厚仍依赖光学假设；缩窄厚度范围需要折射率标定，确认多光束还需核对相位关系的稳定性、光束重叠及仪器分辨能力。

关键词： 外延层厚度；双角光谱；色散；变投影；多光束干涉

一、问题重述与总体分析：两种材料的双角测厚

红外光在外延层上表面与衬底界面返回后形成的条纹，携带着外延层厚度信息。本章先交代题面条件与回答对象，再把三问串成一条由一次返回到逐次往返的测厚链。

1.1 问题重述：一次往返与有条件修正

题目给定的观测构成一条逐步加深的测厚链：四个附件按材料分成两组，分别对应同一块碳化硅晶圆和同一块硅晶圆。每块晶圆均有 10° 和 15° 入射条件下的观测，记录反射率随波数的变化。光学标定缺项统一列于第2.2节。题目要求按一次返回条件建立外延层厚度关系，分别处理碳化硅与硅，并判断何时需要纳入多次往返影响。

问题一从图4的表面反射和衬底界面首个返回出发，建立入射角、波数、材料光学性质与厚度的关系。本文将这两束光叠加后的光场称为两束截断场，即光进入外延层后只计入首次返回，后续往返留待问题三处理。题面给出的四组光谱是这一建模背景，问题一的模型由两束截断场构成，第三问再引入多次往返分母，区分各阶返回的影响。单次返回关系建立后，问题二再用同一片碳化硅晶圆的双角观测估计厚度。

同片碳化硅的坐标对应关系见第2.1节；图6所对应的两角反射率均方根差为 2.14 个百分点，说明角度差异不能被一个共同振幅抹平。比较共享厚度与逐角厚度简单平均后，同片关系与折射率资料缺口促使模型共享厚度和随波数变化的折射率。这种安排还保留每个角度各自的光谱背景与条纹强度，使共同的厚度信息与角度造成的响应差异分别进入模型。

硅与碳化硅在问题三分成两条分支：硅晶圆无论是否满足多光束干涉条件，都必须给出厚度模型、算法和结果。碳化硅只有在确认多次往返出现且确实影响厚度精度时才修正。图10区分往返形成与可观测性，图13并列两种材料的答案。针对第2.2节的观测缺项，谱形拟合与多光束可观测条件分项核对。三项任务的边界由此固定，下一节将用总体分析图串起数据入口、模型承接和结果出口。

1.2 总体分析：两角约束连接三问

材料分组先于方法选择：图 1 按三问并列输入、方法与输出，附件一、二属于碳化硅，附件三、四属于硅，两种材料均保留 10° 与 15° 观测。一次返回关系沿虚线传入问题二，碳化硅厚度沿实线进入问题三的对照。

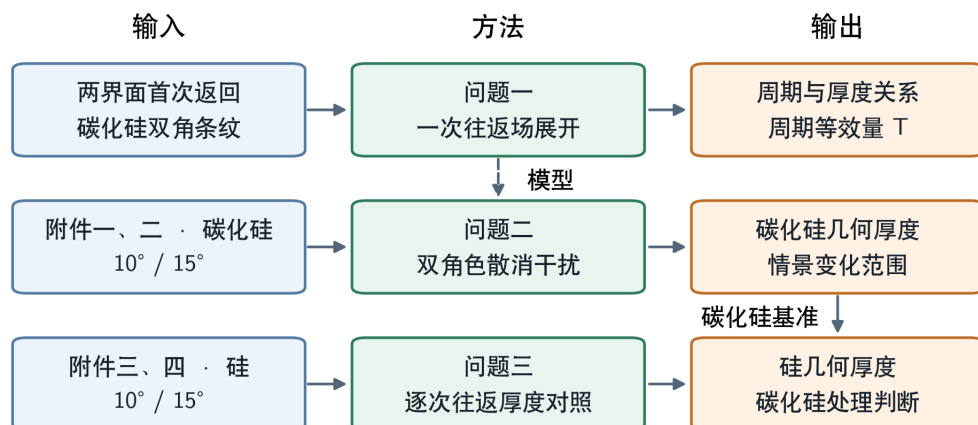


图 1 四附件沿三问展开共同测厚路径

总体组织吸收了高相关性的核对结果：两角原谱相关系数约为 0.9987，但来自同一块晶圆，并非独立厚度样本。原先以高相关性评价厚度可靠性的尝试，因而改为连续留段检验，即留出连续波段，用其观测检验双角预测。

三问的结果按第3.2节的物理量定义区分；周期等效量为 $19.54\ \mu\text{m}$ ，表 1 将其与两种材料的测厚答案并列。

表 1 三问回答的物理量与适用条件

问题	物理量	数值及单位	适用条件
问题一	周期等效量 T	$19.54\ \mu\text{m}$	条纹周期折算
问题二	碳化硅几何厚度	$10.80\ \mu\text{m}$	非触边代表值
问题三	硅完整往返条件估计	$2.17\ \mu\text{m}$	给定经验色散
问题三	硅非概率条件包络	$1.72\text{--}18.16\ \mu\text{m}$	多情景联合范围

注：硅点估计对应全量光谱；包络是多情景范围，不是置信区间。

碳化硅行采用透明两束与经验色散假设，代表值保留参数未碰到搜索边界的原估计。另有 $13.53\ \mu\text{m}$ 的候选损失更低，但折射率已到搜索下界；分段重估、近优解及参数扰动构成的完整情景范围仍为 $0.64\text{--}18.11\ \mu\text{m}$ 。

周期等效量与几何厚度的换算条件统一见第3.2节。
入口、角度和结果对象被这样锁定后，后文即可先检查原始数据边界，再分别展开一次返回、双角估计与高阶往返的物理关系。

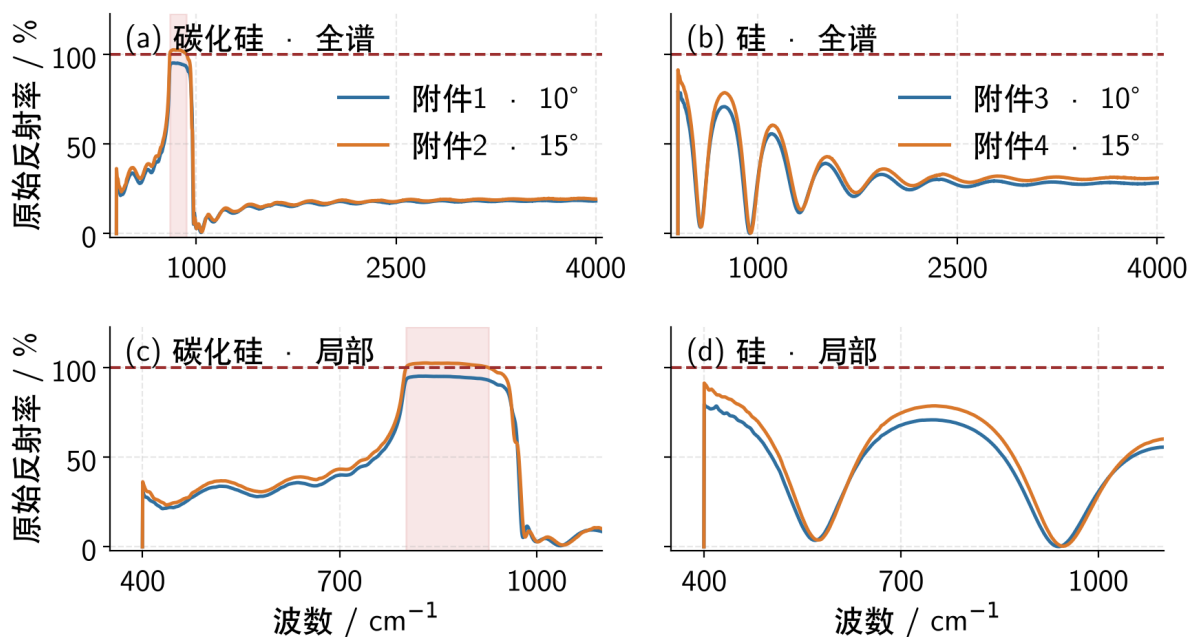
二、数据审查与建模边界：保留原谱和未知条件

上一章区分了周期等效量与拟合厚度，本章先核对支撑这些结果的原始光谱，再确定模型使用的观测范围。附件口径见表 2，共同坐标与拟合区段见附录图 14。

异常原值遵循“标记、保留、单列影响”。附件二有 262 个超出反射率物理范围的原始值，集中在低波数端；图2和表2保留其位置与幅值。第一次尝试按物理范围裁剪，核对后改为保留原值，因为异常段与主拟合窗口不相交；保留或裁剪这些窗外点，主拟合窗口内的输入均相同。后文据此展开附件体检与参数缺口核对。

2.1 附件体检：共同坐标与异常原值

共同首点和超界原值先定义了本节的边界：四谱的共同首点都是 0% 反射率，附件 2 另有 262 个超过 100% 的原始点，最高约为 102.74%。图 2 与表 2 保留异常的位置与幅值；图中全谱与低波数端放大并列，异常被作为可见的数据事实保留，而不是静默改成缺失。



附件2：262点超过100%，最高102.74%；四谱首点均为0%

图 2 四谱原值在低波数端的异常分布

这些质量标记与坐标事实并列后，表 2 给出四谱进入后续比较前的统一口径。

表 2 四谱附件体检摘要

检查对象	共同事实	质量标记	处理口径
四谱共同坐标	7469 点	399.67–4000.12 cm ⁻¹	保留原坐标
共同首点	4 点；0%	四谱同首波数	标记、保留
附件 2 超界	262 点；最高 102.74%	801.28–927.11 cm ⁻¹	标记、不裁
主拟合窗口	1200–3800 cm ⁻¹	与超界段无交集	单独说明

四谱各有 7469 个逐点一致的波数坐标，使双角比较无需插值；相邻步长略有差别，并非严格等间距。曾以端点等距坐标替代原始坐标，发现仍有坐标偏差，因此相位和峰距计算保留原始波数。共同坐标、拟合窗口与质量标记的对照见附录图 14。

主拟合窗口是各模型共同使用的连续波段。低波数端的强烈起伏与中高波数条纹分段处理，高波数末端留作预测核对，主比较沿用原型的 1200–3800 cm⁻¹ 窗口。这两个端点是建模选择，并非裁剪超界点后形成的边界；其对测厚的影响由模型检验章的窗口移动比较考察。

比较按物理范围裁剪与保留原值两种口径后，发现异常段与主拟合窗口不相交，因而保留原始数值并单列质量标记。这里的标记注明异常的位置和幅值，窗外原值仍用于全谱展示与分波段统计。异常区段位于主拟合窗口之外，主拟合使用的原始数值未因此改变。

问题三的硅全量在同一窗口每角保留 5392 个原始点。

量纲也在此处锁定：反射率按百分数记录，计算按比例使用；波数保持 cm^{-1} ，厚度进入传播相位时用 cm ，输出再换成 μm 。因此，原值遵循“只标不裁”，坐标遵循“逐点对齐”；下一节据此检查不同波段的散布是否能够共用一个尺度，并核对光学参数缺口。

2.2 可用信息：波段差异与光学参数缺口

要判断哪些光谱变化能够进入后续测厚，先要确认全谱是否共享同一把尺度。
标准差衡量一段光谱内反射率围绕平均值的起伏大小。附件 1 的 $500\text{--}1000\text{ cm}^{-1}$ 波段反射率散布标准差约为 28.9 个百分点。
相较于低波数段，同一附件的 $3000\text{--}3500\text{ cm}^{-1}$ 波段标准差仅约为 0.189 个百分点。这表明原始反射率的散布随波段明显变化，因此在相同窗口比较两角光谱；这里的标准差同时包含趋势与条纹，不用作测量噪声估计。图 3 将四条光谱的分波段分布并置后，这一判断落在各段的整体形态上，而不是某一个孤立极值上。

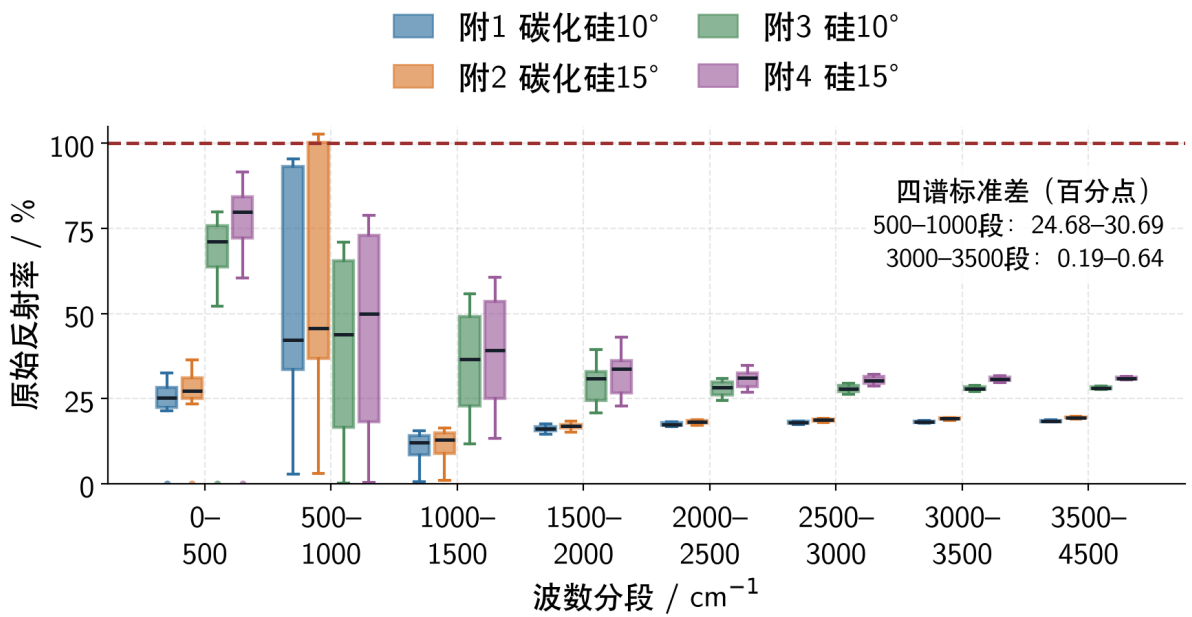


图 3 四谱低波数段反射率散布更大

这种分段差异决定了选窗还须核对条纹连续性，并通过后文的窗口重估检查其影响。表 3 对附件的材料与角度关系，原值保留规则沿用上一节。

表 3 附件与可用光学信息

对象	材料与角度	原始规模与单位	标记或缺口
附件 1	碳化硅, 10°	7469 点; cm^{-1} 、%	超百值 0
附件 2	碳化硅, 15°	7469 点; cm^{-1} 、%	超百值 262
附件 3	硅, 10°	7469 点; cm^{-1} 、%	超百值 0
附件 4	硅, 15°	7469 点; cm^{-1} 、%	超百值 0
共同缺口	两种材料, 双角观测	仅有反射率与波数	五类输入缺失

坐标使用规则见第 2.1 节，测厚还需要附件之外的光学信息。折射率曲线、消光系数、偏振状态、仪器分辨率和真实厚度均未给定；相干长度与光束空间重叠也缺少独立观测。前几项决定相位和反射强度如何换算为厚度，后两项关系到多次返回能否形成可观干涉。

初步核对固定折射率的同型峰距法时，上述标定缺口阻断了直接换算，因此将折射率变化显式保留在相位模型中。

未标定的光学量将在下一章转为显式假设，并逐项说明其生效位置与检验方法。

三、模型假设与符号说明：让光学条件可追溯

本章把每项简化固定在它实际发生的建模位置，并给出可复核的检验。模型将空气、外延层和衬底分成三层，外部入射角取 10° 与 15° ；三层结构用于界面反射、透射和纵向传播因子的表达，分别由场展开互验、被动传播支检查和双角重估核对。

三问依次使用首个返回场、双角共享相位与完整往返场；透明近似、偏振权重和有效损耗的适用位置由假设表逐项限定。

单位约定贯穿三问：反射率先按比例进入计算，波数以 cm^{-1} 记录，厚度在相位中以厘米计算，输出时再换算为微米。厚度沿 $8\mu\text{m} \rightarrow 0.0008\text{ cm} \rightarrow 8\mu\text{m}$ 回算一致，说明波数与厚度相乘的相位无量纲；

下面两小节将把这些生效位置展开为假设表，并用统一符号承接三问的场模型。

3.1 模型假设：分层介质与有限信息

本节先把光路放回实物：空气、外延层和衬底构成相邻的分层介质；假设的职责是规定波矢穿过界面、返回场保留阶数，以及缺失光学量由何种参数承接。本文采用非磁、标量、平面平行且层内均匀的介质近似。三层界面两侧沿界面方向的波矢分量连续，两种偏振采用同一套界面规则。导纳可理解为界面上电场与磁场关系的比例量；纵向传播因子把折射率与层内传播方向合在一起，透明时等于该层折射率乘以光线与垂直界面方向夹角的余弦。 s 偏振的电场垂直入射面，其导纳取纵向传播因子； p 偏振的电场平行入射面，其导纳取该层折射率平方与纵向传播因子的比值，二者共同进入界面系数。空气折射率取 1，外延层和衬底折射率仍按随波数变化的未知函数处理。表4把假设绑定到公式位置和核验方式。

表 4 模型假设的作用位置与核验方式

假设	作用位置	核验方式
平面平行层	传播、界面系数	场表达式互核
被动传播	传播因子	衰减与单位回换
偏振加权	导纳、反射率	权重重估
经验色散	往返相位	常数与扰动对照
有效损耗	传播因子模长	损耗与收敛核对
逐问返回阶数	两束场、往返场	级数与闭式核对
允许观测偏差	灵敏度检验	保留原值重估

被动传播要求幅度沿光路不增长；偏振等权是加权模型中的一个情景。有效损耗合并吸收与界面造成的振幅衰减，避免把它拆成资料无法区分的来源。

场表达式互核比较直接计算的光场强度与交叉项展开式，核对两种写法的代数一致性；结构适用性则由生成光路与估计光路不同的失配情景检查。

界面规则确定后，返回阶数按题面逐问收紧。问题一把到达衬底界面后返回的首束记为 B ，把直接反射记为 r_{01} ，两束截断场记为 E_2 ；问题三才允许后续每次往返再乘同一个往返复倍率 u 。这里的往返复倍率是光再绕一周时同时改变振幅和相位的因子，完整往返场记为 E_∞ ，其关系写成

$$E_2 = r_{01} + B, \quad E_\infty = r_{01} + B(1 + u + u^2 + \cdots) = r_{01} + \frac{B}{1 - u}, \quad |u| < 1. \quad (1)$$

其中 $|u| < 1$ 表示每增加一次往返，返回场的振幅就按小于一的倍率递减，需逐点检查，不由材料名称预设。该条件保证返回场级数收敛；高阶条纹能否观测，还取决于光束相干、空间重叠及测量分辨条件，问题三分别检查。具体而言，各束光须保持稳定的相位关系并落在同一探测区域，高阶条纹的变化还须能与噪声区分且能被仪器分辨。返回阶数固定后，色散承接相位变化，损耗承接传播振幅的衰减。

损耗来源的处理来自一次参数化取舍：曾比较分项描述消光、界面损耗和偏振权重，与合并有效往返损耗两种方案。附件缺少分辨各损耗来源的独立观测，因而采用合并结构，折射率随波数的变化仍用少量参数描述。原始反射率允许存在测量偏差，异常值、选段和残差用于检查假设对厚度估计及光谱预测的影响。

这些约束把“采用什么假设”落实为“假设在哪一步生效、如何被扰动核对”；下一节据此统一外角、复折射率、相位因子以及厘米和微米的符号与量纲。

3.2 符号说明：波数相位与厚度量纲

波数、角度和厚度必须先各自归位，才能让同一条相位关系在三问之间不换口径。真空波长记为 λ_{vac} ， $\sigma = 1/\lambda_{\text{vac}}$ 为波数，单位为 cm^{-1} ；题面给出的 10° 、 15° 是空气侧外部入射角，记为 θ ，层内传播角记为 θ_j 。外部角度只负责界面入射条件，层内相位由纵向因子 q_j 承接，不能把 10° 或 15° 直接代入层内相位。

用 $j = 0, 1, 2$ 标记空气、外延层和衬底，外延层几何厚度记为 d ，相位计算取其厘米值 d_{cm} 。透明层内可写作 $q_j = N_j \cos \theta_j$ ，复折射率存在虚部时仍以根式定义的 q_j 作为传播量。介质 j 的复折射率写为 $N_j(\sigma) = n_j(\sigma) + i\kappa_j(\sigma)$ ，其中 i 为虚数单位，于是

$$q_j(\sigma, \theta) = \sqrt{N_j(\sigma)^2 - \sin^2 \theta}, \quad z(\sigma, \theta; d) = \exp(4\pi i d_{\text{cm}} \sigma q_1(\sigma, \theta)). \quad (2)$$

式中 q_j 表示沿厚度方向的传播，根号取虚部非负的根，即 $\text{Im } q_j \geq 0$ ，透明介质取非负实根，使传播幅度不增长；虚部为正时光在传播中衰减。 z 表示光在外延层中往返一次后的复传播因子。这样，外部角度和层内传播方向被明确分开。

角度和波数统一后，还要把一次返回与完整往返场分成两个符号。令 r_{jk} 、 t_{jk} 分别表示从介质 j 到介质 k 的复反射、透射系数；以下对每种偏振分别定义，省略偏振下标，入射场振幅归一为 1。将表面直接反射加首个返回记为 E_2 ，将表面反射加全部往返记为 E_∞ ，则

$$E_2 = r_{01} + t_{01} t_{10} r_{12} z, \quad E_\infty = r_{01} + \frac{t_{01} t_{10} r_{12} z}{1 - r_{10} r_{12} z}. \quad (3)$$

其中 E_2 只保留表面直接反射和首个返回, E_∞ 才把后续返回写成几何级数; d 始终表示外延层几何厚度, 同片双角共享其值, 硅与碳化硅分别估计。同一比较内两种场共用 z ; 问题三以实折射率和有效损耗参数化其相位与幅度。表 5 汇总符号含义、单位及双角共享或各角独有的用法。

表 5 符号含义、单位与角度用法

符号	短含义	单位与使用属性
$j, \lambda_{\text{vac}}, \sigma$	层编号、真空波长、波数	无量纲、cm、 cm^{-1}
θ, θ_j	外角、层内角	度; 外角给定、内角求得
$N_j = n_j + i\kappa_j$	复折射率	无量纲; 随材料、波数变化
q_j, z	纵向因子、往返因子	无量纲; 随波数、外角变化
λ	有效往返损耗	无量纲; 同片双角共享
$d_{\text{cm}}, d_{\mu\text{m}}$	几何厚度	厘米、微米; 同片共享
r_{jk}, t_{jk}	复反射、透射系数	无量纲; 随角度、偏振变化
E_2, E_∞	两束场、完整往返场	无量纲; 归一化复振幅
$\rho_{\%}, R$	百分数反射率、反射率比例	$R = \rho_{\%}/100$

表中 N_j 的实部决定相位、虚部决定衰减; q_j 描述纵向传播, z 描述在外延层中的一次往返。 E_2 叠加表面反射与首个返回, E_∞ 叠加表面反射与全部返回; λ 用于问题三的有效往返损耗。

几何厚度 d 描述外延层上下界面的实际间距; 周期等效量 T 则是把有限波段内拟合的条纹间距折算成的长度。令有效周期 $\Delta\sigma$ 以 cm^{-1} 计, T 以 μm 计, 则

$$T = \frac{10^4}{2\Delta\sigma}. \quad (4)$$

其中 $\Delta\sigma$ 是整段光谱的周期统计量, 不是每一对相邻条纹的局部间距。色散与界面相位的贡献均包含在条纹变化中; 在透明近似下, 只有这些变化足够小且相位在窗口内近似均匀变化时, T 才近似为 $d_{\mu\text{m}}q_1$ 。这一区分使周期读数与外延层几何厚度各有明确对象, 局部关系见式(14)。

厚度由厘米换为微米时, $d_{\mu\text{m}} = 10^4 d_{\text{cm}}$ 。这些符号固定了外角、层内传播与量纲, 问题一据此推导表面反射和首个返回的合场强度。

四、模型的建立与求解：从两束反射到逐次往返

上一章统一了外角、纵向传播因子与厚度量纲, 本章据此从一次返回场出发, 将同片双角约束与后续往返逐步写入模型。图 1 把这条关系画成同一条数据路径; 问题一的两束边界和问题三的往返判据分别落在图4与图10中。返回阶数随题目需求逐层增加, 一次返回先解释条纹, 同片双角与完整往返场再分别处理两种材料。

三类验证量各有参照: 问题一主方法的合成匹配平均误差为 0.0008%, 问题二连续留段标准化均方根误差为 1.21, 问题三同口径误差为 1.02。前者衡量已知光学条件下的厚度回收, 后两者分别衡量碳化硅与完整往返场的反射率预测表现。

结果关系随模型边界保留: 问题二得到的 $10.80 \mu\text{m}$ 碳化硅拟合厚度, 作为问题三继续比较时的碳化硅基准; 硅则由完整往返场模型给出 $2.17 \mu\text{m}$ 的全量厚度估计。碳化硅的修正判断同时核对高阶返回的相干性、空间重叠和仪器分辨率, 并比较同片厚度随场模型的变化。

双角原谱高相关曾被用来评价厚度可靠性, 核对同片观测关系后, 改用双角联动的连续留段检验评估共享参数。正文据此按问题一、问题二、问题三依次展开: 先确定两束物理关系, 再处理同片双角, 最后比较完整往返场及其材料分支。

4.1 问题一的建模与求解：一次往返场展开

本问先回答条纹直接给出了什么量, 再建立它与外延层厚度的联系。真实碳化硅光谱的周期等效量为 $19.54 \mu\text{m}$, 换算几何厚度还需光学参数。“一次往返场展开”是指: 把外延层上表面的直接反射场, 与光进入外延层、到达衬底界面后返回的首束场相加; 光再次往返形成的更高阶返回不放入本问主模型。

两谱有效周期约为 254.85 和 256.99 cm^{-1} , 以上述周期的中位数按第3.2节的定义折算。该实测量保留有限窗口内的色散和界面相位贡献, 局部近似的条件由式(14)给出。

先限定为两束截断场, 才能把角度、色散和偏振对条纹的作用逐项放入同一表达式, 并把后续往返的收益单独检验; 若直接引入高阶项, 表面反射与首个返回的作用会失去清晰边界。两束的来源和真实数据的光学参数缺口将在分析与准备中交代, 随后由界面导纳写出色散相位及其峰距关系。

4.1.1 分析与准备：为何保留界面反射场

本节先处理一个边界问题: 问题一究竟应当把哪些返回光写进模型。条纹峰距只反映传播相位的变化, 若折射率随波数变化而又没有标定, 直接把峰距换成几何厚度就会把材料光学性质误当成厚度; 因此主路线需要同时保留入射角、色散和偏振, 而不是先填入一个固定折射率。

实测输入沿用第2.1节的共同坐标, 光学缺项见第2.2节。两种角度改变了光线在外延层中的传播方向, 却没有改变同片几何厚度; 这使双角可以共同约束一条厚度参数, 同时保留各自的界面振幅。直接

采用全谱同型峰距并配固定折射率的尝试，正是因为这些输入缺失而被改为使用“波数乘纵向传播因子”的色散坐标。

两束边界由题面限定为表面直接反射和首个返回，后续往返不在本问提前叠加。图4把空气、外延层与衬底、外部入射角、层内传播方向和厚度路径放在同一几何关系中；所谓“双束界面场反演”，就是由这两束光的复场叠加计算反射率，再寻找使计算值与观测值接近的厚度。这样既保留了 10° 与 15° 的角度差异，也为问题三比较高阶返回留下清晰的基准。

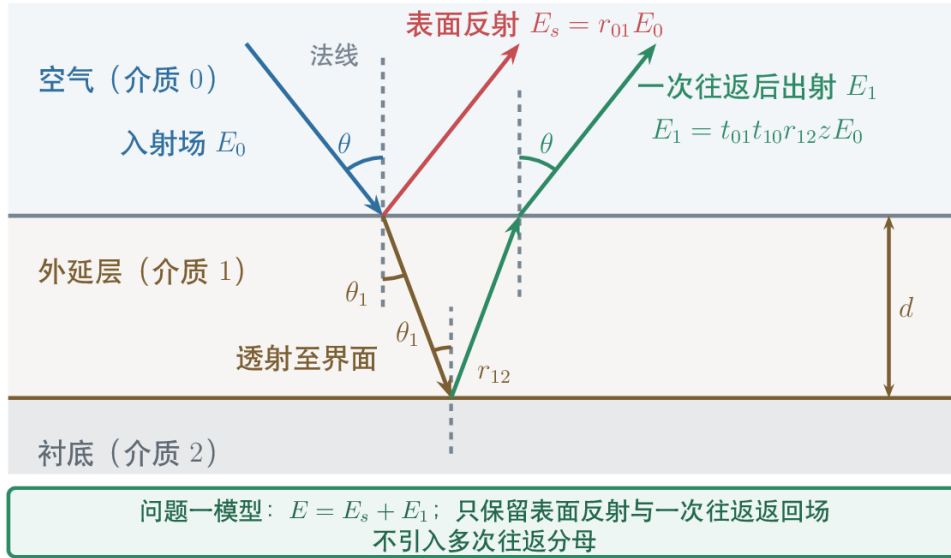


图4 一次往返的两束反射场路径

路线选择也由同口径对照决定。已知光学参数的24组原型双角合成案例中，双束界面场反演的平均相对误差为0.0010%，低于色散峰序反演的0.0201%和连续相位拟厚的0.0041%；逐观测输入一致的结论限定于这三个已核验原型，故没有把不同路线的结果平均，而是将后两者保留为级次与相位的独立核对。色散峰序反演按折射率变化修正峰谷位置，再按条纹级次求厚度；连续相位拟厚则沿波数展开条纹相位，由相位变化反推厚度。误差以合成光谱的已知厚度为参照，比较各路线在匹配生成机制下的回收表现。

正式复算另用一组生成观测，主方法平均误差为0.0008%，与它使用相同输入的峰距基线为2.59%；原型与正式组的含噪观测不同，二者不能拼成一组路线排名。

强度形状进一步排除了“全谱恒幅条纹”的简化：附件1在 $500\text{--}1000\text{ cm}^{-1}$ 和 $3000\text{--}3500\text{ cm}^{-1}$ 两个诊断波段的反射率标准差分别为28.9和0.189个百分点，散布尺度相差悬殊。故主求解直接拟合两束截断场展开后的强度，局部极值和峰距承担相位解释与级次核对；下一节将从两束截断场推导偏振强度、色散相位及条纹间距。

4.1.2 模型建立：色散相位决定条纹间距

一次返回光谱的条纹来自两条路径的复场叠加：一条是外延层表面的直接反射，另一条是到达衬底界面后的首个返回。要让这两条路径同时承接本题的外部角度与波数，先把界面振幅和层内传播相位放在同一套符号中。

介质 $j = 0, 1, 2$ 依次为空气、外延层、衬底，外角为 θ ，厚度 d 以 $d_{\text{cm}} = d$ 计算、以 $d_{\mu\text{m}} = 10^4 d_{\text{cm}}$ 报告，波数为 σ (cm^{-1})，复折射率为 $N_j = n_j + i\kappa_j$ （实部 n_j 、消光系数 κ_j ），空气取 $N_0 = 1$ 。平面界面上切向波矢守恒，归一化切向分量为 $N_0 \sin \theta = \sin \theta$ ，故纵向分量满足 $N_j^2 = q_j^2 + \sin^2 \theta$ 。层内纵向传播因子和一次往返传播因子定义为

$$q_j = \sqrt{N_j^2 - \sin^2 \theta}, \quad z = \exp(4\pi i d_{\text{cm}} \sigma q_1). \quad (5)$$

其中 q_j 描述光在相应介质中沿厚度方向的传播， z 则把外延层厚度、入射角和波数合并为一次往返的复相位；复根取 $\text{Im } q_j \geq 0$ ，透明时取非负实根，使有损介质中的传播幅度不增长。

偏振差异先进入界面导纳。记 s, p 为两种相互正交的偏振， $Y_j^{(a)}$ 为偏振 a 在介质 j 中的切向电场导纳。对任一偏振，在相邻介质 j, k 的界面处，把入射切向电场振幅归一为一，并将反射、透射振幅暂记为 r, t ；切向电场与磁场连续分别给出

$$1 + r = t, \quad Y_j(1 - r) = Y_k t. \quad (6)$$

这里 Y_j, Y_k 表示所选偏振在界面两侧的导纳。联立两式消去 t 求得 r ，再由 $t = 1 + r$ 求透射系数；恢复

偏振标记后有

$$Y_j^{(s)} = q_j, \quad Y_j^{(p)} = \frac{N_j^2}{q_j}, \quad r_{jk}^{(a)} = \frac{Y_j^{(a)} - Y_k^{(a)}}{Y_j^{(a)} + Y_k^{(a)}}, \quad t_{jk}^{(a)} = \frac{2Y_j^{(a)}}{Y_j^{(a)} + Y_k^{(a)}}. \quad (7)$$

这里 $r_{jk}^{(a)}$ 和 $t_{jk}^{(a)}$ 分别是介质 j 到介质 k 的反射、透射振幅系数。令表面直接反射项 $a_a = r_{01}^{(a)}$ ，首个返回的界面项 $b_a = t_{01}^{(a)} t_{10}^{(a)} r_{12}^{(a)}$ ，题面的一次返回边界便给出

$$E_a = a_a + b_a z. \quad (8)$$

因此， s 、 p 偏振的差别留在 a_a, b_a 中，厚度则只通过 z 进入； 10° 与 15° 两条观测支路可以共享 d ，同时保留各自的界面响应。场表达式确定后，下一步是把正交偏振的强度混合，并展开其中的交叉干涉项。

设 s 偏振强度权重为 w ， $0 \leq w \leq 1$ ， p 偏振权重为 $1 - w$ 。由于两种偏振正交，观测反射率比例是两种偏振强度的加权和，而不是把两种电场直接相加。令 $w_s = w$ 、 $w_p = 1 - w$ ，并定义

$$U = \sum_a w_a |a_a|^2, \quad V = \sum_a w_a |b_a|^2, \quad C = \sum_a w_a a_a^* b_a, \quad (9)$$

则一次返回模型的强度展开为

$$R_2 = \sum_a w_a |E_a|^2 = U + V|z|^2 + 2\operatorname{Re}(Cz). \quad (10)$$

其中 U 是直接反射背景， $V|z|^2$ 是首个返回的幅度项， $2\operatorname{Re}(Cz)$ 是两条路径相遇产生的条纹项。

两种强度写法在已知条件的合成光谱上相符；这里的合成光谱，是先给定厚度、折射率曲线与偏振比例，再由两束光相加生成的光谱。表6列出数值核对的条件，直接复场强度与交叉项展开的最大绝对差为 1.7×10^{-16} （反射率比例）；另令衬底与外延层折射率相同，在零返回核对点保留与去掉返回项的反射率差为 0。

表 6 两种强度写法的数值核对条件

核对项目	取值
几何厚度	3, 8, 20 μm
外部入射角	$10^\circ, 15^\circ$
波数采样	附件 1 原始坐标每隔 32 点取一点
外延层折射率	$n_1(\sigma) = 2.8 + 0.08(\sigma - 2200)/1800$
衬底折射率	$n_2(\sigma) = n_1(\sigma) + 0.8$
吸收与偏振	$\kappa_1 = \kappa_2 = 0, \quad w = 0.5$
零返回核对点	$d_{\mu\text{m}} = 8, \quad \theta = 10^\circ, \quad \sigma = 2199.9 \text{ cm}^{-1}$

附录图 17 进一步交叉比较折射率基值为 2.4 或 3.4、色散斜率为 0 或 0.08 的组合，厚度、角度、衬底折射率增量与偏振设置沿用表中的一般核对条件。散点贴近等值线，表示两种写法在同一波数点给出相同的反射率，而不是比较不同波数点的光谱形状。

两种强度写法的代数一致性确定后，进一步考察色散对条纹位置的作用。透明且包络变化较慢时，令 $\psi = \arg C$ ，主干相位可写为

$$\Phi(\sigma, \theta) = 4\pi d_{\text{cm}} \sigma \operatorname{Re} q_1 + \psi, \quad \Phi' = 4\pi d_{\text{cm}} \left(q_1 + \sigma \frac{dq_1}{d\sigma} \right) + \frac{d\psi}{d\sigma}. \quad (11)$$

式中撇号表示对波数 σ 求导。若折射率实部记为 $n(\sigma)$ ，则 $q_1 = \sqrt{n^2 - \sin^2 \theta}$ 且 $dq_1/d\sigma = nn'/q_1$ ，所以相位导数中的色散贡献为 $\sigma nn'/q_1$ 。这一步把“条纹间距”落到相位对波数的变化率，而不是把一个全谱固定系数直接当成厚度。

相位级次与反射率峰谷之间还隔着背景和振幅的变化。把强度展开整理为

$$R_2 = B + A \cos \Phi, \quad R'_2 = B' + A' \cos \Phi - A\Phi' \sin \Phi. \quad (12)$$

其中 $B = U + V|z|^2$ 为反射率背景， $A = 2|Cz|$ 为条纹振幅。只有 A 不接近零，且 $|B'|$ 、 $|A'|$ 远小于 $|A\Phi'|$ 时，反射率极值才近似对应相位级次。上一节的波段包络差异表明，全谱背景和振幅并不恒定，因此峰距承担局部相位核对，主求解仍拟合两束截断场强度。

在上述缓变条件下，相邻同型极值的相位差近似为一个完整周期，故其波数差 $\Delta\sigma$ 满足

$$2d \Delta(\sigma q_1) + \frac{\Delta\psi}{2\pi} \simeq 1, \quad d_{\mu\text{m}} \simeq \frac{5000(1 - \Delta\psi/(2\pi))}{\Delta(\sigma q_1)}. \quad (13)$$

对实际相邻同型条纹，将波数差记为 $\Delta\sigma_{\text{loc}}$ ，相应局部周期等效量记为 T_{loc} 。将同型条纹关系除以该波

数差，得到

$$T_{\text{loc}} = \frac{5000}{\Delta\sigma_{\text{loc}}} \simeq d_{\mu\text{m}} \frac{\Delta(\sigma q_1)}{\Delta\sigma_{\text{loc}}} + \frac{2500}{\pi} \frac{\Delta\psi}{\Delta\sigma_{\text{loc}}}, \quad (14)$$

$$T_{\text{loc}} - d_{\mu\text{m}} q_1 \simeq d_{\mu\text{m}} \frac{\sigma n n'}{q_1} + \frac{2500}{\pi} \psi'.$$

这里差商表示相邻同型条纹间的平均相位变化，第二行为局部近似，右端给出偏离厚度乘纵向传播因子的两项来源。两项均可忽略时才有 $T_{\text{loc}} \simeq d_{\mu\text{m}} q_1$ 。实测 T 来自有限窗口的最佳正弦周期，不等于逐点测得的 T_{loc} ；将前者也简写为 $d q_1$ ，还须核验窗口内相位斜率近似恒定。本题没有估计或扣除这些导数项，因而保留 T 作为周期统计量，不作厚度乘积的实测标定。

峰到谷只跨过半个周期，必须改写为

$$2d \Delta(\sigma q_1) + \frac{\Delta\psi}{2\pi} \simeq \frac{1}{2}, \quad d_{\mu\text{m}} \simeq \frac{2500(1 - \Delta\psi/\pi)}{\Delta(\sigma q_1)}. \quad (15)$$

当 $n' = 0$ 、界面相位变化缓慢且包络导数相对主相位项可忽略时，才退化为 $5000/(q_1 \Delta\sigma)$ 和 $2500/(q_1 \Delta\sigma)$ 。两个系数相差一倍，正是峰谷不能直接套用同型峰距式的原因；图18进一步把色散斜率与波数对常数峰距偏差的共同作用分开显示。

峰谷系数的分开处理并非形式上的修补。奇偶级次反演在色散坐标中区分同型极值与半周期事件，同型极值不足时未用峰谷混合补足；全部合成案例均保留。因此本节保留两类级次的不同方程。

固定光学条件后搜索得到的厚度解构成厚度候选，优劣以两角反射率的联合残差平方均值衡量，而不是预先给定干涉级次。相位坐标由此解释双角条纹疏密，并为下一节的合成回收、实测周期提取和低残差厚度候选提供共同依据。

4.1.3 求解与结果分析：合成回收与相位歧义

色散相位已经解释条纹位置，本节进一步回答这条关系能否把厚度回收到正确分支，并把合成条件、真实光谱和方法边界分开。求解时以双角联合的两束界面场为主线，峰序与连续相位只承担结构核对。

在生成机制与反演机制一致的共同合成情景中，双束界面场反演的平均绝对相对误差为 0.0008%，同一输入下的常折射率峰距基线为 2.59%。在相同合成输入上，双束界面场反演的厚度回收误差低于常折射率峰距基线。

因为这里要检验厚度反演本身，正式双束复算把膜层与衬底的折射率、消光系数及偏振权重固定为给定情景，只估计两角共享的厚度。联合损失就是模型反射率与观测反射率之差的平方在两角全部采样点上取平均，比较对象是反射率，而非电场或相位。每角按均匀索引向下取整选取 256 个原始波数坐标，厚度在 0.5–40 μm 内搜索；这个范围是算法搜索域，不是由观测推得的概率区间。

粗扫先比较 129 个等距厚度点，再对损失最低的 4 个候选作黄金分割精化，逐步缩小候选附近区间，最后取损失最小者。合成真厚度仅参与光谱生成和误差评分，搜索过程依据观测反射率选择候选。连续相位拟厚沿波数展开条纹相位，由相位变化反推厚度，并与场模型相互核对相位级次。

共同案例全部保留，表7分列“原型三路线共同 24 例”与“正式双束复算共同 24 例”：案例集相同，配置不同，误差分别计算；破折号表示相应配置下未运行该路线。图中的原型逐例点对应前一行，正式复算的双束误差与常折射率峰距基线则配在后一行。

逐例复核发现，连续相位路线在一个案例中同时保留了 3.0000 μm 与 36.8799 μm 两个候选；这里的相位歧义，是不同厚度都通过了相位拟合的候选筛选。计分采用相位回归残差较小的前一个厚度，另一个候选及歧义标记也保留，因此表中平均误差衡量的是所选主候选的回收程度。依据这项全案例核对，双束界面场被选作主方法，峰序与连续相位继续承担独立对照。

表 7 三路线与朴素基线的合成厚度误差

情景	例数	双束场/%	峰序/%	相位/%	峰距基线/%
原型三路线共同	24	0.0010	0.0201	0.0041	—
正式另组共同输入	24	0.0008	—	—	2.59
新增匹配	16	0.0011	—	—	—
模型失配	10	0.3028	—	—	—

共同案例的低误差并没有延伸到所有生成条件。新增匹配的 16 例扩展了厚度、折射率、色散斜率与曲率、偏振和噪声组合，生成与反演仍使用同一光学关系；其中折射率基值改为 2.8 和 3.1，噪声标准差最高为 0.0015（反射率比例）。这组案例检验的是改变参数后能否继续回收厚度。

曲率和包络系数采用统一的波数尺度：把原波数减去 2200 cm^{-1} 再除以 1800 cm^{-1} ，得到无量纲的归一化波数。

模型失配则指生成光谱包含反演时遗漏或误设的成分。失配组共 10 例，包括遗漏 ± 0.12 的色散曲率、将 0.2 的垂直偏振权重按 0.5 求解等情景，图5分区比较参数变化与模型缺项的误差。

其他失配还包括遗漏系数为 ± 0.05 的线性包络、系数为 0.01 的二次基线、消光系数为 0.002 的吸收，以及生成端含 8 束返回而反演仍按一次返回处理；包络与基线以归一化波数为自变量。部分案例同

时改变两项，失配组均值因此反映这一组设定的综合影响。

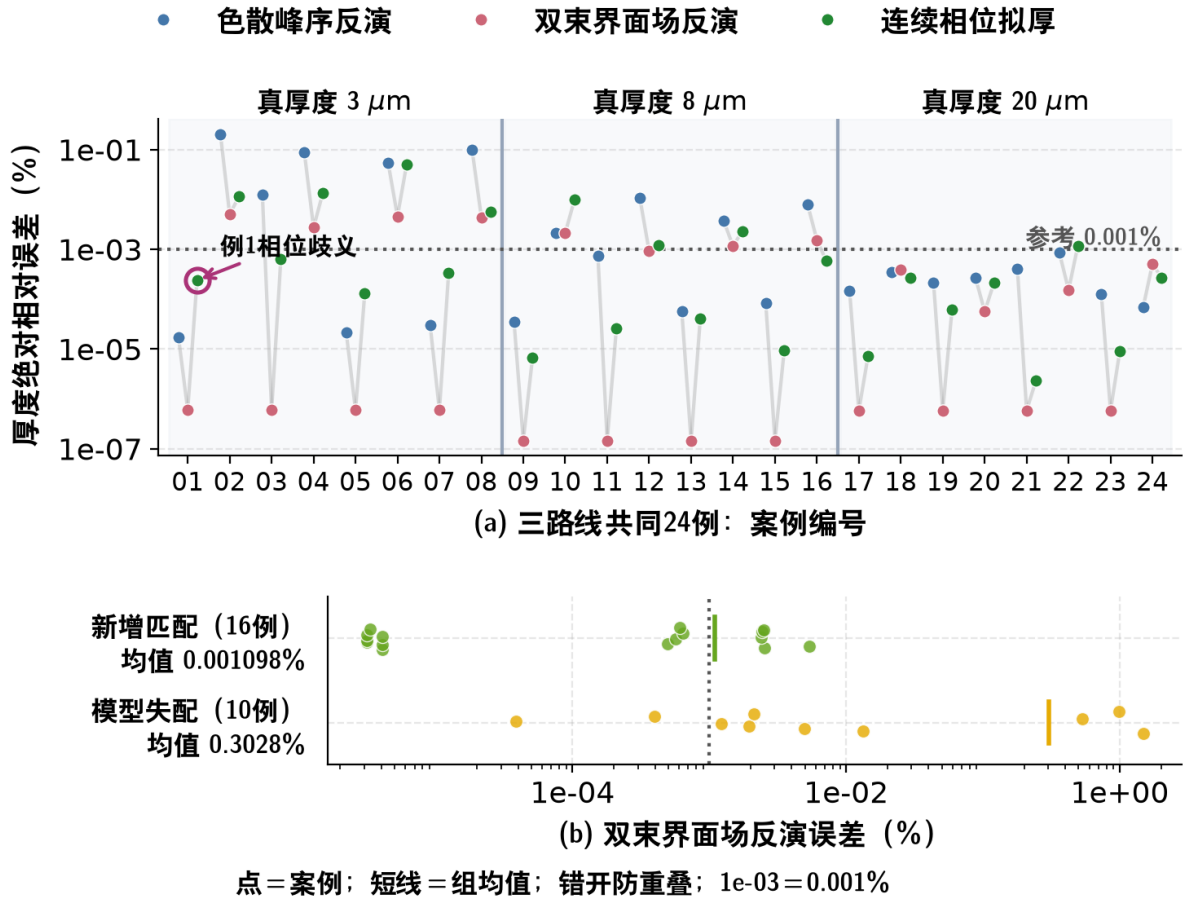


图5 三路线共同案例与失配情景误差

图5下部仅展示主方法在新增匹配与模型失配情景中的误差。上部是原型三路线在共同案例上的逐例配对，对应表7的原型行。另将一个角度整谱用于估计厚度，另一个角度整谱用于检验预测，再互换两角，称为两角互换验证；其厚度预测平均绝对相对误差为 0.0018%，说明角度互换后的方向性核对与共同案例结论一致。

保持按 $8\ \mu\text{m}$ 厚度生成的同一组合成观测不变，调整折射率、偏振、平滑宽度和入射角后重估，厚度为 $6.66\text{--}10.01\ \mu\text{m}$ 。这一厚度重估范围记录同一合成样本对设定变化的响应，比较时固定观测，以分开输入变化与估计条件变化的作用。

真实光谱的周期提取先处理缓慢变化的背景。在 $2800\text{--}3600\ \text{cm}^{-1}$ 窗口内，将反射率与候选周期的正余弦列投影到同一二次基线并取残差，扣除慢变背景后搜索最能解释振荡的周期。

两谱提取的周期分别为 254.85 和 $256.99\ \text{cm}^{-1}$ ，将两周期的中位数代入定义式得到 $T = 19.54\ \mu\text{m}$ 。这里报告的是有限窗口周期统计量；色散和界面相位贡献未被分离，不能直接解释为 d_{q1} 或几何厚度。式(14)约束的是实际相邻条纹的局部差商，不能把其中的近似条件当作本次窗口拟合已经验证的结论。

问题一的结论为：双束界面场反演在固定合成条件下的平均回收误差为 0.0008%，优于峰距基线的 2.59%，实测周期等效量为 $T = 19.54\ \mu\text{m}$ 。

本问置信等级为低：周期等效量有独立复算支持，合成回收、常折射率基线比较与两角互换验证支持方法选择。真实附件缺少折射率、消光系数、偏振状态和厚度真值，厚度重估范围较宽，周期等效量尚未换算为标定后的几何厚度。合成误差衡量给定光学条件下的厚度回收，不代表真实晶圆的测量精度。

由此，下一问需要利用同片双角的共同信息进一步压缩未知光学因素，把周期等效量推进到可比较的给定光学条件下的厚度。

4.2 问题二的建模与求解：双角色散消干扰

两条碳化硅光谱来自同一块晶圆，逐行共享 7469 个波数点，却有 2.14 个百分点的反射率均方根差；这意味着厚度应当共享，基线、振幅和相位却不能把两角硬压成一条曲线。双角色散消干扰据此把共同的条纹位置交给厚度与色散，把角度特有的慢变响应留在各自的有限参数中，而不是将两角原谱先平均。

全量同片拟合给出 $10.80\ \mu\text{m}$ 的拟合厚度；五折连续留段显示厚度分支明显漂移，因此谱形解释与厚度辨识并列呈现。正式留段标准化均方根误差为 1.21，相对二次趋势基线改善 16.2%，说明条纹项增加了模型对留出反射率的解释力。

这条主线把“共享厚度”与“角度响应”分成两个层次，下面依次给出分块准备、共同厚度投影模型及其结果边界。

4.2.1 分析与准备：分块观测与受限干扰项

问题二要在同一块晶圆的两条光谱中找出共同的厚度信息，同时把入射角造成的外观差异留在各自模型里。为避免相邻波数点被误当成独立测试，先把主拟合窗口的原始坐标组织成连续区段，再分别安排估计、校准与计分。在 $1200\text{--}3800\text{ cm}^{-1}$ 主拟合窗口内，先按波数升序排列观测，再将首末观测之间的序号范围均分为 479 份，分点序号向下取整，连同两端共取 480 个原始坐标点；保留附件给出的波数值，不插值为等距波数网格。

这些坐标按顺序每 40 点组成一块，形成 12 个连续块，块号随波数递增。连续块数和训练边界的 20 cm^{-1} 保护带均由两角共同使用并预先固定，保护带隔开邻近观测以减轻信息串入。

训练段估计参数并确定归一化尺度，校准段只校准预测区间半宽，测试段只计分。正式比较采用两折，即两套训练、校准和测试的区段分工，具体块号见表8；上一节的五折是在同一组连续块上另行轮换测试区段，用于检查厚度随留段位置的变化，并非另分五个块。图6展示两角采用相同波数分组的两折训练、校准和测试安排；这种安排使模型选择、区间校准与预测评价各有来源，不把随机拆行或统一平滑当作独立验证。

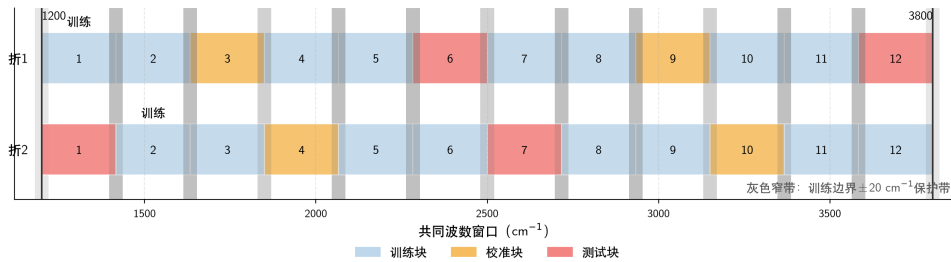


图6 两角同步留段的训练校准测试区段

末端 $3800\text{--}4000.122\text{ cm}^{-1}$ 另作外层检验，保护窗口为 $3750\text{--}3800\text{ cm}^{-1}$ 。

两谱的原始相关系数为 0.9987，但反射率均方根差仍有 2.14 个百分点。初次以双角高相关评价厚度可靠性，核对后改为同波数双角分组，以训练尺度归一化的留段预测误差比较方法。波数逐行对齐决定双角同步分组，同片物理条件决定共享厚度；原谱高相关包含慢变基线，留段预测用于检验模型对谱形的解释。

超界观测来自附件 2，共有 262 个反射率超过 100% 的原值，所在区间与主拟合窗口不相交，点数及波数范围列入表8。这些观测不参与主拟合窗口拟合，原文件保持不变。

附件一（十度入射）在 $500\text{--}1000$ 与 $3000\text{--}3500\text{ cm}^{-1}$ 两段的反射率散布明显不同，表8保留其标准差。这里的标准差衡量一段光谱内反射率围绕平均值的起伏大小，既包含慢变包络，也包含条纹变化。

选型时曾核对绝对振幅反演与峰谷序列反演对包络和校准的依赖，分段散布差异促使慢变部分与条纹分别处理。包络和条纹幅值随波段变化，因此用逐角低阶响应描述慢变部分，即各角度的基线最高取二次、幅值最高取一次；两角损失以训练残差尺度归一化，该尺度描述训练段扣除二次基线后剩余差异的散布，原谱标准差则描述总体起伏。峰序与相位回归承担独立核对，同片厚度由双角共享参数的主方法给出。

“双角色散变投影”就是先给定厚度和折射变化，再从观测中求出各角度的基线与振幅，让搜索集中在较少的物理参数上。两种角度共同决定厚度 d 、参考折射率参数 n_0 与色散参数 c ，每个角度自己解释慢变基线、振幅和常相位偏置 ψ_θ 。给定这些非线性参数后，再用线性投影求各角度的二次基线系数 $b_{\theta,0}, b_{\theta,1}, b_{\theta,2}$ 和一次幅值系数 $a_{\theta,0}, a_{\theta,1}$ ；因此干扰项有边界，不能扩张为逐波数自由相位。逐角独立求厚后再平均也被排除，因为它既丢掉同片约束，也无法说明两角差异来自何处。表8将窗口、区段职责和参数角色集中列出；其中触边解指参数达到预设取值上下限的候选解。

表8 问题二数据切分与参数搜索口径

环节	主设置	参数角色	作用
主拟合窗口	$1200\text{--}3800\text{ cm}^{-1}$	480 点、12 个连续块	双角同步取样
第一折	训练：1, 2, 4, 5, 7, 8, 10, 11	校准：3, 9	测试：6, 12
第二折	训练：2, 3, 5, 6, 8, 9, 11, 12	校准：4, 10	测试：1, 7
区段隔离	训练边界保护带 20 cm^{-1}	测试块只计分	减轻邻近信息串入
超百分点	$801.278\text{--}927.1104\text{ cm}^{-1}$	附件 2, 262 点	原值保留，主窗外
低波数段	$500\text{--}1000\text{ cm}^{-1}$	标准差 28.9 个百分点	附件一，十度入射
高波数段	$3000\text{--}3500\text{ cm}^{-1}$	标准差 0.189 个百分点	附件一，十度入射
变投影	共享 d, n_0, c ；分角常相位 ψ_θ	给定非线性量后解线性系数	保留共同条纹与角度差异
搜索边界	$d \in [0.5, 40]\text{ }\mu\text{m}$	$n_0 \in [1.2, 6]$ ； $c \in [-1, 1]$	每个情景重新估计
粗网格	厚度 96 点网格 $\times$ 9 个光学情景	局部起点最多 12 个	多起点局部细化
停止条件	单情景预算 42 s；固定参数须一致	只保留满足物理约束的候选	避免以触边解缩小范围

在共享物理参数与分角线性响应的分工下，下一节给出消去线性系数后的联合目标。

4.2.2 模型建立：共同厚度的色散变投影

同片双角共享厚度和折射变化，基线、幅值与固定相位则分别估计。先前考虑逐角求厚再平均，却会丢失同片约束，因而改用这项分工。

令 d 为厚度 (μm)， σ 为波数 (cm^{-1})， $\theta \in \{10^\circ, 15^\circ\}$ 为入射角。透明各向同性假设下，实折射率 $n(\sigma)$ 与一次往返相位 Φ_θ 满足

$$n(\sigma) = n_0 + c \left[\left(\frac{\sigma_0}{\sigma} \right)^2 - 1 \right], \quad \Phi_\theta = 4\pi 10^{-4} d \sigma \sqrt{n(\sigma)^2 - \sin^2 \theta}. \quad (16)$$

其中 n_0 为参照波数 $\sigma_0 = 2000 \text{ cm}^{-1}$ 处的折射率， c 为无量纲色散参数， 10^{-4} 将微米换为厘米；传播要求 $n(\sigma) > \sin 15^\circ$ 。缺少实测折射率，故用这两个参数描述折射变化。

取训练波数的中点 $\bar{\sigma}$ 与半跨度 h ，令 $x = (\sigma - \bar{\sigma})/h$ ，以比例反射率 y_θ 写出观测式

$$y_\theta = b_{\theta,0} + b_{\theta,1}x + b_{\theta,2}x^2 + (a_{\theta,0} + a_{\theta,1}x) \cos(\Phi_\theta + \psi_\theta) + \varepsilon_\theta. \quad (17)$$

系数 $b_{\theta,j}$ 和 $a_{\theta,j}$ 分别描述基线与幅值， ψ_θ 为常相位， ε_θ 为残差。前节两角反射率并不相同，因此用各角低阶响应吸收慢变背景和包络差异；二次基线、一次幅值是限制干扰自由度的工作假设，其影响由改变阶数后的重新估计检查。幅值可变号而相位固定，避免自由相位吞掉条纹。

给定共享参数 $\eta = (d, n_0, c)$ 和 ψ_θ ，令 $z_\theta = \cos(\Phi_\theta + \psi_\theta)$ 。变投影先解角度响应，再搜索物理参数，线性部分只含五列（表8）：

$$X_\theta = [1, x, x^2, z_\theta, xz_\theta], \quad \hat{\beta}_\theta = \arg \min_{\beta_\theta} \|y_\theta - X_\theta \beta_\theta\|_2^2. \quad (18)$$

其中 X_θ 在训练点逐行取值， y_θ 为观测向量，系数向量 β_θ 按 X_θ 列序排列。外层在表8约束内最小化两角等权损失

$$L(\eta) = \frac{1}{2} \sum_{\theta \in \{10^\circ, 15^\circ\}} \min_{0 \leq \psi_\theta < \pi} \frac{\|y_\theta - X_\theta \hat{\beta}_\theta\|_2^2}{N_\theta s_\theta^2}. \quad (19)$$

训练点数记为 N_θ ， s_θ 取训练二次趋势残差的四分位距，下限为 10^{-4} ，搜索中固定。用去趋势后的尺度归一化，使两角的幅值差异不直接决定损失权重；该尺度只在训练点计算并在搜索中固定。幅值变号可吸收 π 的相位平移，故常相位只搜索半周期。

粗网格搜索以 96 个厚度点覆盖 $0.5\text{--}40 \mu\text{m}$ ，再从多起点有界细化并全量重拟合。

参数搜索范围就是计算时允许各参数变化的上下限，并非材料测量给出的取值范围。参考折射率允许 $n_0 \in [1.2, 6]$ ，色散参数允许 $c \in [-1, 1]$ ，并须满足前述传播约束；这些边界会排除范围外的组合，多起点用于比较范围内不同厚度与折射率组合。

相位中厚度与折射因子相乘，折射率下降可由厚度增大补偿。

全量拟合厚度为 $10.80 \mu\text{m}$ ，参考折射率为 1.4930。相位中的厚度与折射因子相乘，不同组合可以产生相近条纹，因而继续检查损失接近最小值的候选解。

连续切分、光学参数补偿以及基线和色散形式改变，共同形成 $0.64\text{--}18.11 \mu\text{m}$ 的厚度重估范围（图7）。令 D 为纳入范围的候选厚度集合， I_{cond} 为其最外端点构成的区间，则

$$D = D_2 \cup D_5 \cup D_{\text{prof}} \cup D_{\text{sens}}, \quad I_{\text{cond}} = [\min D, \max D]. \quad (20)$$

其中 D_2 、 D_5 分别收集两折、五折连续留段重新拟合的候选厚度； D_{prof} 固定全量拟合色散，逐点重估厚度与参考折射率网格上的各角响应，收集损失不超过剖面最小值 1.10 倍的厚度，称为近等损失剖面候选。

D_{sens} 收集改变基线、幅值、色散或波段后重新估计的厚度，按“是否纳入厚度重估范围”筛选；满足传播约束和指定参数条件且被列为纳入的候选进入集合，人为改动指定参数的边界调整结果排除在外。这些候选对应各自的模型设置与参数约束， I_{cond} 汇集它们的最外端点，记录厚度随估计条件的变化。

下一节据共同相位与逐角响应，报告两角预测、留段误差及反射率覆盖的缺口。

4.2.3 求解与结果分析：碳化硅厚度及双角核对

共享厚度与逐角低阶响应分开后，求解的重点便是交出厚度，并检查这个数能否解释未参与拟合的光谱。这里先报告同片结果，再用连续留段和双向留角度核对条纹。

碳化硅外延层的同片非触边代表厚度为 $10.80 \mu\text{m}$ ，厚度重估范围为 $0.64\text{--}18.11 \mu\text{m}$ 。点值对应透明两束、经验色散及逐角低阶响应；范围则记录搜索、切分和光学条件变化带来的厚度移动。表9将计算设置、同片代表值与分项核对并列，两个附件共同决定厚度，而非分别求厚后取平均。

这里采用的是固定比较基准的规则，而非加密后重新选优：在原厚度网格 96 点搜索内取最低损失可行解，各参数距上下界的距离除以搜索跨度后均须大于 10^{-4} 。原点的最小相对边界距离为 0.0610，满足该规则；表25中的加密触边候选仍参与全部候选的损失比较，不因触边而被排除。

按训练损失排序，现有补查最低的厚度是 $13.53 \mu\text{m}$ ，不是沿用的 $10.80 \mu\text{m}$ 。前者折射率触及搜索下界，后者承担跨问题比较坐标；这两个身份分列，搜索产生的分支差异也与改变光学假设的情景差异分别讨论。厚度保留两位小数是报告格式，测量精度仍受光学假设和取段变化限制。

附件一（十度）与附件二（十五度）在共同原始坐标上同步抽样。给定厚度与折射变化后，先直接求出各角基线和幅值，再作多起点有界搜索；这种安排把条纹位置留给共享参数，把角度造成的包络差别交给低阶响应。连续块承担训练、校准和测试的不同任务，训练边界另留 20 cm⁻¹ 保护带，避免相邻波数点混入留段检验。

表 9 碳化硅计算条件与厚度核对

项目	设置或结果	项目	设置或结果
主拟合窗口	1200–3800 cm ⁻¹	原坐标采样	每角 480 点
逐角基线	二次	逐角幅值	一次
参考折射率	1.4930	色散系数	–0.2089
全量同片厚度	10.80 μm	厚度重估范围	0.64–18.11 μm
五折厚度范围	5.53–13.51 μm	五折相对极差	0.7604
正式留段误差	1.2074	二次趋势误差	1.4404
谱形相对改善	16.17%	测试经验覆盖	49.06%
原型主方法误差	1.1717	原型峰序误差	1.2354
原型相位误差	1.2202	误差及极差单位	无量纲
测试区段	10° → 15° 误差	测试区段	15° → 10° 误差
首折第六块	0.501	首折第六块	0.257
首折第十二块	1.186	首折第十二块	1.087
次折第一块	3.457	次折第一块	2.965
次折第七块	0.168	次折第七块	0.249

宽范围来自厚度与折射率的互相补偿：不同组合可以解释相近的条纹。五折厚度的相对极差为 0.7604，即最大与最小厚度之差除以中位数；图7把这种切分移动与光学情景分开。范围按式(20)取两折、五折、光学剖面及满足参数约束的灵敏度候选的最外端点，其中剖面保留训练损失较小值增加不超过 10% 的组合。

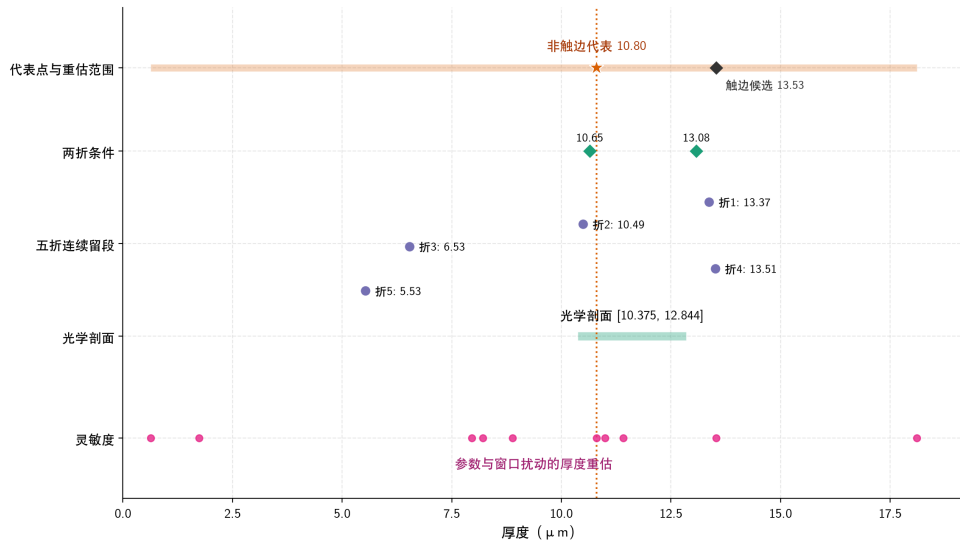


图 7 碳化硅厚度随切分和光学设置变化

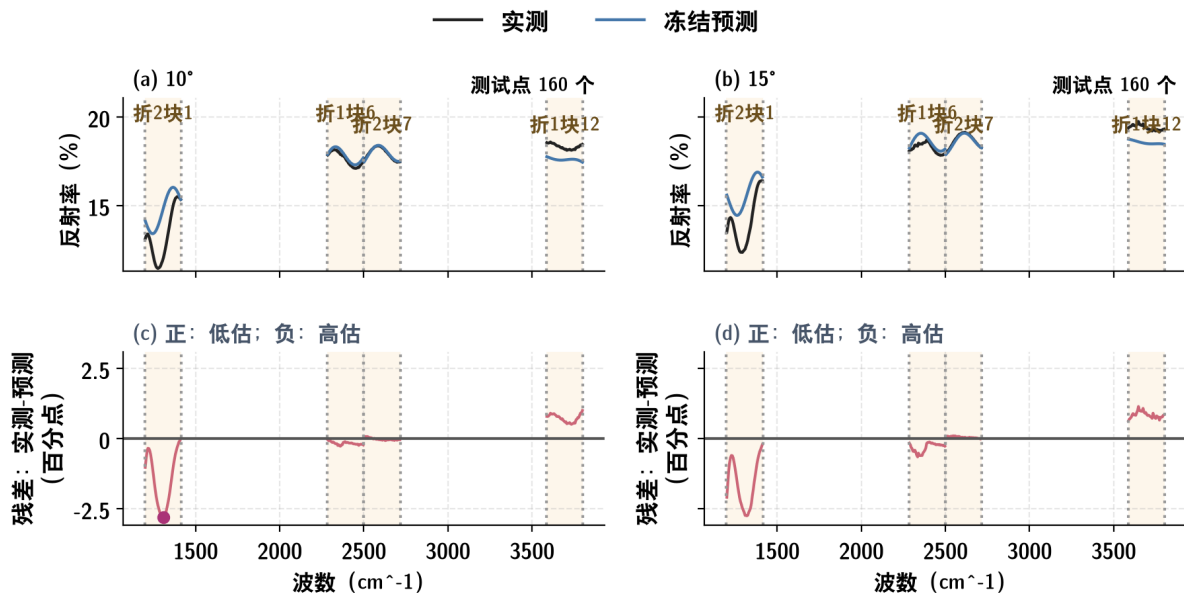
厚度分支虽随区段移动，条纹项仍增加了对留出光谱的解释力。正式连续留段标准化均方根误差为 1.2074，低于二次趋势基线的 1.4404，相对改善 16.17%；两项误差均无量纲，表示反射率预测误差相对于训练残差尺度的大小，按测试块等权平均。图8并列整体预测与局部残差，展示条纹拟合的总体优势和仍有偏差的波数区段。

局部偏差也曾改变区间的解释。最初把经验区间称为名义 90% 区间时，连续测试块覆盖率仅为 50.63%；这次核对促使区间改用固定经验分位描述，并单独报告留出观测实际落入区间的比例。

当前反射率区间以校准块绝对残差的 75% 经验分位作为对称半宽，即将校准偏差由小到大排列，取该位置的偏差加在预测值两侧；测试经验覆盖率为 49.06%。图9以校准分位为参照线，分块偏离说明校准段的误差宽度没有充分容纳测试段的偏差。

覆盖缺口把注意力引向跨角度的局部偏差。双向留角度核对固定每折共享厚度、色散及来源角相位，只在目标角训练点重估低阶基线与幅值；另一角的偏差便无法再靠自由调整厚度吸收。误差最大的测试块在正、反方向分别达到 3.457 和 2.965，均无量纲，图15与表9保留了其余块的差别。

双向核对检验同片参数跨角度解释谱形的能力，碳化硅的同片拟合厚度为 10.80 μm，厚度重估范围为 0.64–18.11 μm。本问置信等级为低：五折厚度随切分明显变化，反射率测试覆盖不足，灵敏度情景形成较宽的厚度重估范围。该范围汇集光学假设下的重估结果，不是概率区间，反射率覆盖率也不是厚

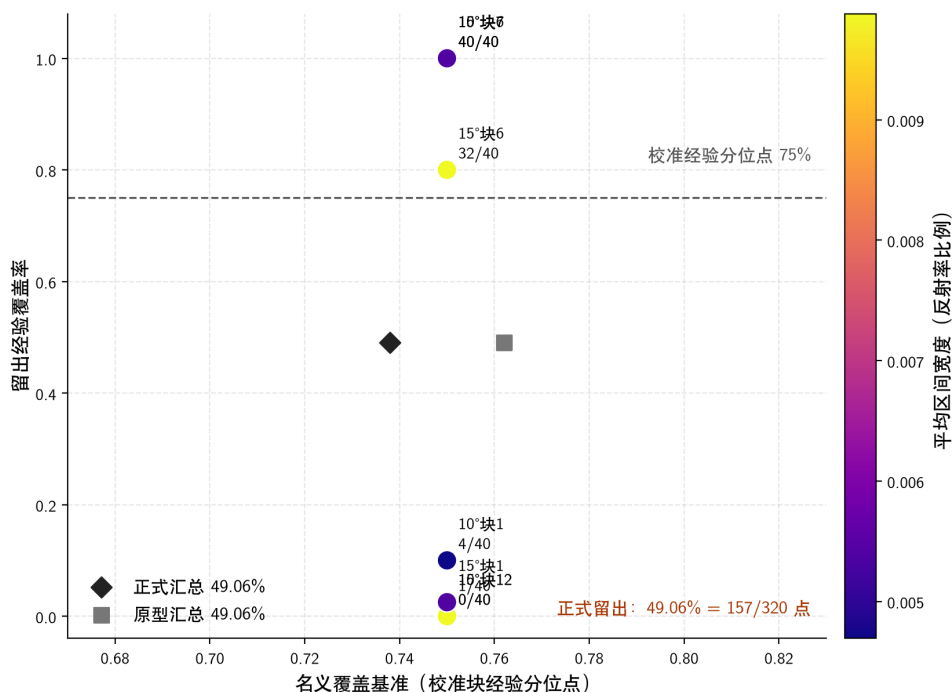


残差 = 实测-预测；负：预测高估；正：预测低估

10°、1308.947 cm⁻¹：预测高估

实测 12.0556% - 预测 14.8681% = -2.8125 个百分点

图 8 双角光谱预测显示局部残差



点标签给出每角每块分子/分母；此覆盖只针对反射率点预测，不等于厚度覆盖

图 9 留出反射率覆盖偏离校准分位水平

度覆盖率，绝对厚度仍需光学标定或标准厚度样片核对。问题三据此保留两束模型基准，检验更高阶往返是否足以改变这块晶圆的厚度判断。

4.3 问题三的建模与求解：逐次往返厚度对照

上一问给出了碳化硅的两束模型基准，本问采用逐次往返厚度对照，在相同设置下比较两束截断场与完整往返场。完整往返场将表面反射与各轮返回相加，往返衰减场反演据此计算反射率，再搜索与观测相符的厚度。硅在这一框架下求厚度，碳化硅则在高阶往返可观测且影响厚度时修正。

完整往返场给出硅全量厚度 2.17 μm ，两次连续留段分别为 1.73、16.04 μm ；碳化硅缺少相干、空间重叠和仪器分辨率资料，保留前问基准 10.80 μm 。

两材料在同波段、同留出点及同响应设置下比较，完整往返场的总体标准化留段均方根误差为 1.0231，高于两束的 0.9812（均无量纲）。逐块收益随材料和区段改变，完整场并未取得总体预测优势。这个分歧要求先核对观测条件与资料缺口，再推导往返场及其厚度影响。

4.3.1 分析与准备：必要条件对应观测证据

多光束判定须分清形成、计算、观测与测厚影响。非零界面返回、时间相干、空间重叠与仪器分辨四类条件缺一不可；两个界面存在只涉及首项。

往返复倍率即每轮返回场的共同衰减与相位因子。其最大模（幅度比例）在硅两次留段为 0.1333 和 0.0769，碳化硅为 0.0258 和 0.0076，均无量纲；图10中的光路说明相邻返回为何乘以同一复数，收敛与实验观测由不同条件控制。

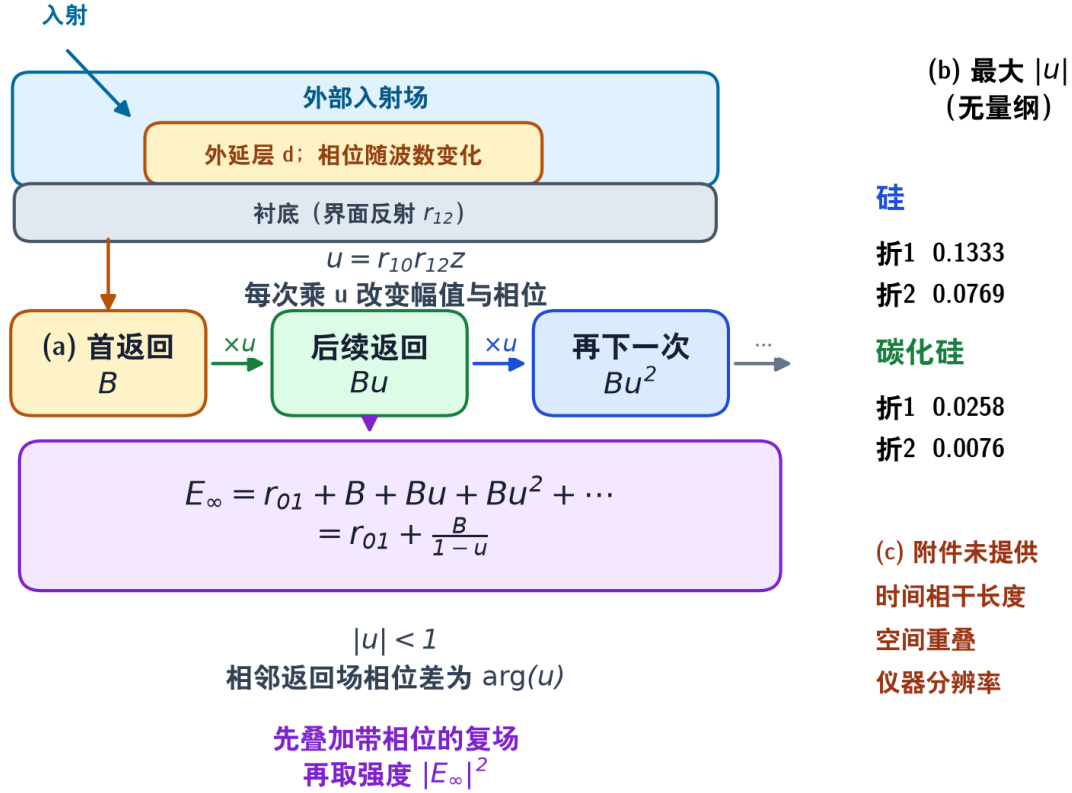


图 10 返回光的复振幅随往返次数递减

逐项核对发现，硅双角原始相关系数虽达 0.9943，相干、重叠及分辨资料却均缺失，因而排除高相关直接判定；表10把已知量与未知量分列。

表 10 多光束必要条件与四附件证据

必要条件	核对对象	四附件已有信息	缺失信息	替代解释
界面返回	界面反射系数	模型可核验	实测光学常数	界面弱反射
时间相干	往返相位关联	均未给资料	相干长度	相位随机化
空间重叠	返回光束重合	均未给资料	光路、偏振	光束错位
仪器分辨	高阶谱形差	反射率均已给	分辨率、噪声	基线与色散

慢变基线也可抬高相关；折射率随波数的变化称为色散，也会改变谱形。故两束截断场与完整往返场采用同波段、同训练及留出区段、同基线复杂度，逐角评价谱形相容性。下一节沿此对照推导有限往返及收敛闭式，再讨论厚度影响。

4.3.2 模型建立：衰减场求和与厚度偏差

首个返回场记为 B ，后续每轮返回都乘同一往返复倍率 u ，即共同的衰减与相位因子。按几何级数求和，分清高阶返回改变的是谱形还是厚度估计。

沿用界面振幅系数 r_{ij}, t_{ij} ， d, σ, θ 分别为厚度（ μm ）、波数（ cm^{-1} ）和入射角。令 n_1 为外延层实折射率，纵向传播因子 $q_1 = \sqrt{n_1^2 - \sin^2 \theta}$ ，则

$$z = \exp(-\lambda n_1 / q_1 + i\Phi_\theta), \quad \Phi_\theta = 4\pi 10^{-4} d \sigma q_1, \quad (21)$$

$$B = t_{01} t_{10} r_{12} z, \quad u = r_{10} r_{12} z.$$

其中 z, Φ_θ 为一次往返的传播因子与相位， $\lambda \geq 0$ 为有效损耗；取正传播支，传播幅度不增长。保留 K

次返回的场记为 $\mathcal{E}_K$, 当 $|u| < 1$ 时

$$\begin{aligned}\mathcal{E}_K &= r_{01} + B \sum_{k=0}^{K-1} u^k = r_{01} + B \frac{1-u^K}{1-u}, \\ E_\infty &= r_{01} + \frac{B}{1-u}, \quad E_2 = \mathcal{E}_1 = r_{01} + B, \\ |E_\infty - \mathcal{E}_K| &\leq \frac{|B||u|^K}{1-|u|}.\end{aligned}\quad (22)$$

这里 K 为正整数, k 为返回阶次。尾项界给出截断误差, $u \rightarrow 0$ 时完整往返场退回两束截断场; 各次返回须先在复场上相加, 再取模平方。

多光束干涉首先要求光路能产生第三束出射光。光透入外延层、在衬底反射并穿出上表面, 形成 $B = t_{01}r_{12}zt_{10}$; 留在层内的光又经上表面内反射和衬底反射, 形成 $Bu = t_{01}t_{10}r_{10}r_{12}^2z^2$ 。因此透入、透出及两个界面的内部反射通道均须非零。级数收敛约束这些返回场的总和, 探测器记录的干涉还取决于光束间的关联。记直接场振幅 $a_0 = r_{01}$ 、第 m 次返回振幅 $a_m = Bu^{m-1}$, $m \geq 1$ 。对探测区域和时间作平均, 实际强度中的交叉项为

$$\begin{aligned}I &= \sum_{m \geq 0} |a_m|^2 + 2 \operatorname{Re} \sum_{m < n} a_m a_n^* \gamma_{mn} \eta_{mn}, \\ J_{\text{ho}} &= 2 \operatorname{Re} \sum_{\substack{m < n \\ n \geq 2}} a_m a_n^* \gamma_{mn} \eta_{mn}.\end{aligned}\quad (23)$$

其中 γ_{mn} 为两返回场的归一化时间相干度, 表示相位关联经时间平均后的保留比例; η_{mn} 为探测区域内的归一化空间与偏振重叠积分。 J_{ho} 专指至少含一束后续返回光的干涉交叉项, 不含各束自身的强度。

逐一断开光路即可检验必要性: $Bu = 0$ 会消去全部后续返回, $\gamma_{mn} = 0$ 会使该对光束的相位平均消失, $\eta_{mn} = 0$ 会使探测面上的交叉积分消失。

形成高阶干涉须存在 $n \geq 2$ 的场对满足 $a_m a_n^* \gamma_{mn} \eta_{mn} \neq 0$, 且这些交叉项合计后仍随波数变化。非零返回、相位关联与空间偏振重叠由此分别落到可检查的因子上。

相位关联和光束重叠可分别换成长度比较。令 $\omega = 2\pi c_0 \sigma$, c_0 为真空光速, ϕ_m 为第 m 束在同一观测面的相位, 定义群时延差 $\Delta\tau_{mn} = |\partial(\phi_m - \phi_n)/\partial\omega|$, 等效往返光程差为 $L_{mn} = c_0 \Delta\tau_{mn}$ 。在单一相干包络、平行界面与有限宽光束的近似下, 明显交叉项要求

$$L_{mn} \lesssim \ell_c, \quad |m-n| 2d_{\text{cm}} \tan \theta_1 \lesssim w_b, \quad L_{mn} = \frac{|m-n|}{2\pi} \left| \frac{\partial \arg u}{\partial \sigma} \right| \quad (m, n \geq 1). \quad (24)$$

这里 ℓ_c 为有效相干长度, θ_1 为层内角, w_b 为探测面对应的光束横向尺度, 所有长度统一用厘米。末式由返回场每轮累积同一相位得到, 已包含传播色散和界面相位; 无色散、界面相位不变时, 单次光程差退化为 $2d_{\text{cm}}q_1$ 。长度比较描述常见包络下的量级要求, 严格条件仍是式(23)中的关联与重叠积分非零。

干涉形成后还要通过仪器响应。设 h 为归一化谱响应核, δR_{ho} 为 J_{ho} 除以入射强度后的反射率贡献, 比较时固定同一组光学参数; ε_R 为噪声与背景分离误差尺度, 则辨认高阶贡献要求

$$\|h * \delta R_{\text{ho}}\| > \varepsilon_R, \quad |C_j \hat{h}(f_j)| \gtrsim \varepsilon_R, \quad f_j = \frac{j}{2\pi} \left| \frac{\partial \arg u}{\partial \sigma} \right|. \quad (25)$$

其中 C_j, f_j 是相隔 j 轮的返回场交叉项在局部的振幅和波数频率, $\hat{h}$ 是仪器响应的傅里叶变换; $\arg u$ 同时保留传播及界面相位。若谱响应宽度为 $\delta\sigma_{\text{inst}}$, 避免明显平均抵消通常要求 $2\pi f_j \delta\sigma_{\text{inst}} \lesssim 1$, 严格判断仍以卷积后的幅度为准。采样间距描述相邻观测点的距离; 仪器谱响应与噪声尺度则需独立标定, 附件未提供这两项资料。因而 $|u| < 1$ 回答级数能否求和, 非零交叉项回答干涉能否形成, 响应后超出噪声才回答实验能否辨认。

曾尝试分别估计消光、界面损耗和偏振权重; 因附件缺少对应标定, 改用 λ 合并吸收与界面损失。该量描述总衰减, 而非实测消光系数。

同片双角共享 $(d, n_{\text{ref}}, c, \Delta n, \lambda)$; n_1 沿用式(16), 参考折射率记为 n_{ref} , 衬底折射率取 $n_2 = n_1 + \Delta n$ 。主情景取偏振等权, 各角背景最高二次、幅值最高一次:

$$\begin{aligned}P_\theta &= \frac{1}{2} (|E_{\theta,s}|^2 + |E_{\theta,p}|^2), \\ R_\theta &= \beta_{\theta 0} + \beta_{\theta 1}x + \beta_{\theta 2}x^2 + \left[\gamma_{\theta 0} \frac{1-x}{2} + \gamma_{\theta 1} \frac{1+x}{2} \right] P_\theta + \varepsilon_\theta.\end{aligned}\quad (26)$$

其中 x 按主拟合窗口中心与半宽归一化; P_θ, R_θ 为理想与观测比例反射率, s, p 为正交偏振, $\beta, \gamma, \varepsilon$ 依次为背景系数、有界非负幅值端点和残差。拟合时先从物理响应列扣去可被背景解释的分量, 再约束标标准化增益并施加收缩惩罚; 阶数与惩罚由训练内比较选定, 两模型共用所选设置, 避免背景吞掉条纹。

冻结折射率、损耗和界面系数, 高阶返回只引入基本相位的整数倍, 故

$$P_\theta(\Phi_\theta + 2\pi) = P_\theta(\Phi_\theta), \quad \Delta\sigma = \frac{10^4}{2dq_1}. \quad (27)$$

这里 $\Delta\sigma$ 为同型条纹的基本波数周期。谱峰可以变尖而周期不变; 色散、损耗或响应随波数变化会移动

极值，误把整数倍频的谐波当成基频也会改变测厚所用的峰距。

峰形变化对厚度的作用取决于它能否被厚度方向解释。令 $\delta R = R_\infty - R_2$ 为两模型的谱差， $J_d = \partial R_2 / \partial d$ 描述厚度微变引起的谱差。固定其余参数，作局部线性近似，两束解附近的厚度改变量 $\delta d = d_\infty - d_2$ 满足

$$\langle J_d, J_d \delta d + \delta R \rangle = 0 \implies \delta d \simeq - \frac{\langle J_d, \delta R \rangle}{\langle J_d, J_d \rangle}. \tag{28}$$

内积按同一双角尺度加权；谱差与厚度方向正交时，一阶偏移为零。重估色散等参数时，先扣除两向量中可被其解释的分量；厚度与折射率的互相补偿会减弱厚度约束，因此谱差与厚度改变量分别评价。下一节分列硅逐角结果，再判碳化硅修正。

4.3.3 求解与结果分析：硅厚度与碳化硅修正判断

双角光谱先确定硅厚度，再区分谱形改善和测厚影响。

逐次往返厚度对照给出硅全量厚度 $2.17\ \mu\text{m}$ ，对应每角 5392 个原始点。

两折厚度为 1.73 和 $16.04\ \mu\text{m}$ ，相差 $14.31\ \mu\text{m}$ ，区段切分改变了硅厚度分支。

同段比较沿用 $1200\text{--}3800\ \text{cm}^{-1}$ 主拟合窗口。为隔开背景变化，两场同阶响应；训练估参、校准定区间、测试评分。

完整往返场的四附件汇总误差 1.0231 高于两束的 0.9812，两种朴素对照的误差并列于表11。这里先按各角训练残差的四分位距标准化，再对全部角块等权平均，分数为无量纲。

同一搜索预算下，完整场仍未降低总体误差。两模型保留原粗候选对照，并在相同条件内整合近优分支、从原始起点精修，采用相同的响应设置。分别估参后的分数差反映两种模型的预测表现；固定参数的场差与重估后的厚度差另行区分，以免把参数补偿解释为纯粹的高阶作用。采用值与计算文件的对应关系见附录8.3，两场公式与候选选择分别列出。

表 11 求解设置与两材料厚度前后对照

核对项目	数值或设置	对象与含义
初始共同搜索	61 个粗候选、3 个起点	原候选保留作共同对照
配对精修	每起点 150/300 次两级评估	两模型均重估五个物理量
硅全量厚度	$2.32 \rightarrow 2.17\ \mu\text{m}$	同段两束 $\rightarrow$ 完整往返场
硅折间厚度	$1.73, 16.04\ \mu\text{m}$	每角 480 点，两次连续切分
硅模型厚度差	$-0.15\ \mu\text{m}$	全量完整往返减两束
硅条件包络	$1.72\text{--}18.16\ \mu\text{m}$	非概率的已算情景范围
碳化硅同段厚度	$2.79 \rightarrow 2.83\ \mu\text{m}$	同步重估光学参数
碳化硅采用厚度	$10.80 \rightarrow 10.80\ \mu\text{m}$	实际处理前 $\rightarrow$ 处理后
碳化硅采用范围	$0.64\text{--}18.11\ \mu\text{m}$ (保持)	前后沿用厚度重估范围
留段标准化误差	$0.9812 \rightarrow 1.0231$	四附件两束 $\rightarrow$ 完整往返场
朴素对照误差	$2.1929, 1.4434$	训练均值、二次趋势
高阶可观测性	复杂模型适配，仪器资料缺失	改善来源未分离
稳定测厚影响	光学参数与模型影响混合	碳化硅不替换基准

硅首折同一测试块中，附件三的预测误差由 0.4281 降至 0.3000 个百分点，附件四由 1.0923 降至 0.8759 个百分点。这一局部改善需与全部测试块合看；图11承担同段预测与残差的对照。

逐附件核对发现，硅附件三、四的高阶方案各在 2/4 个测试块上改善。早期对照曾因总体误差较低而保留完整往返，逐块检查却发现硅附件三仍有反向区段，因此保留各块的正负差值，表43与图12分别承担数值核对和方向比较。

碳化硅附件一、二各有 2/4 个测试块出现高阶方案改善，正向收益均未贯穿全部区段。表10列出的物理可观测性与这些预测差异是两类证据。前者逐项检验式(23)至式(25)的独立条件；后者比较留出光谱的预测误差。

正向块并存负向块，说明高阶方案的预测收益没有贯穿四个附件。全量拟合中，硅与碳化硅的往返倍率模上界分别为 0.1441 和 0.0271；表12将这些模型内的非零返回与仍需测量的观测条件分列，不能从收敛直接跳到实验检出。

表 12 四附件的返回场与可观测条件

附件	正向块/总块	模型内可确认	谱形相容证据	未知观测条件
一	2/4	非零返回、收敛	仅部分区段改善	相干、重叠、分辨及噪声
二	2/4	非零返回、收敛	仅部分区段改善	相干、重叠、分辨及噪声
三	2/4	非零返回、收敛	仅部分区段改善	相干、重叠、分辨及噪声
四	2/4	非零返回、收敛	仅部分区段改善	相干、重叠、分辨及噪声

表中的非零返回与收敛均在所设光学模型内成立，相位关联、光束重叠和仪器响应则没有独立测量。图10中的返回路径对应 Bu ，相干叠加对应 $\gamma_{mn}\eta_{mn}$ ；场幅逐次衰减本身不能确定卷积后的高阶交叉项是否高于噪声。

厚度影响也须单独识别。碳化硅同段厚度从 2.79 变为 $2.83\ \mu\text{m}$ ，折射率与损耗也同时改变，差值混合了参数补偿和往返影响。因此表11将模型对照与实际采用值分行。

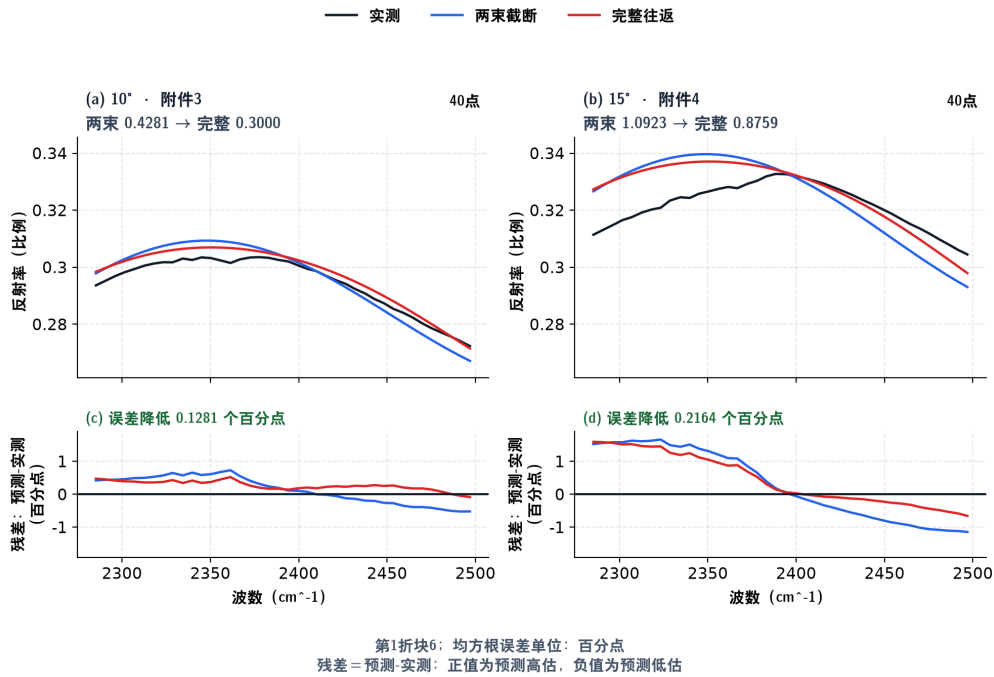


图 11 硅两角高阶场预测残差存在差异

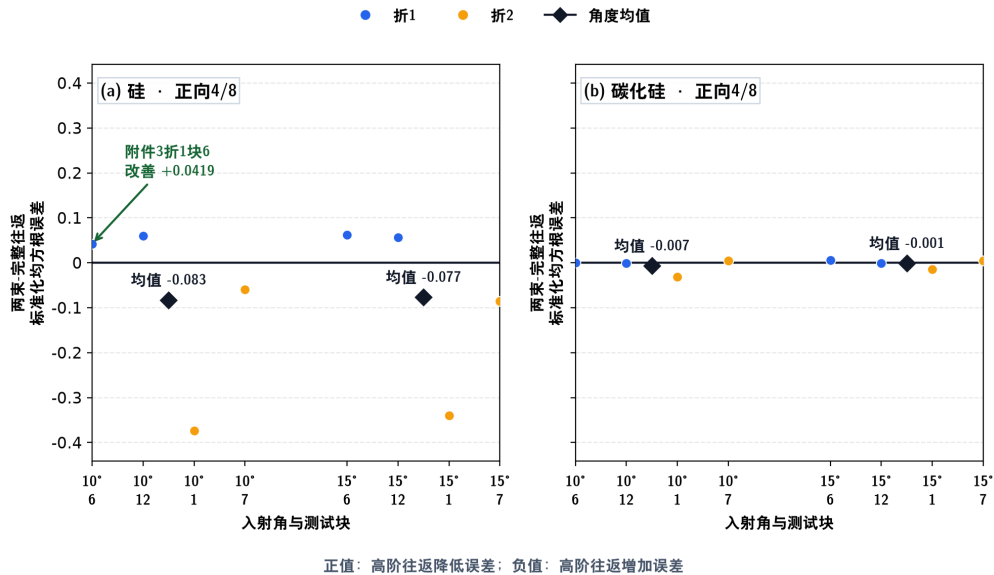


图 12 两材料完整往返场的逐块误差改善

历史参数扰动中, 碳化硅厚度变化介于 -1.6720 与 $2.1261 \mu\text{m}$; 图16保留这组重估对照, 区别于固定参数的高阶场差, 也未替换第二问的采用厚度。

两项修正要求未同时成立, 碳化硅维持 $10.80 \mu\text{m}$ 及 $0.64\text{--}18.11 \mu\text{m}$ 。图13并列两材料答案, 硅的两折跨度与完整情景包络分开, 碳化硅线段表示厚度重估范围。

硅采用完整往返场的全量拟合厚度, 已计算的条件包络为 $1.72\text{--}18.16 \mu\text{m}$, 碳化硅沿用前问基准。包络由两模型全量、两折、近优候选及已算扰动的最外端点构成, 是非概率的情景范围。

本问置信等级为低: 硅次折相对训练均值劣化 11.30%, 相对两束劣化 9.52%, 两材料高阶收益跨折反向; 同起点且均收敛的衬底扰动使厚度下降 13.83%, 其他扰动的竞争分支差值跨零。已有结果给出有限光学条件下的最佳估计及描述性包络; 已用于模型开发的测试区段承担探索性核对, 独立同条件复算仍待补全, 故尚不能确定唯一真厚度或概率区间。

完整场的反射率测试经验覆盖率为 73.91%, 低于名义 90%。下一章将反射率预测误差与厚度误差分开检验。

五、模型检验：把拟合误差与测厚误差分开

前章保留硅的完整往返场厚度与碳化硅的两束模型基准, 本章分别核对反射率预测表现及厚度随光学条件的变化。

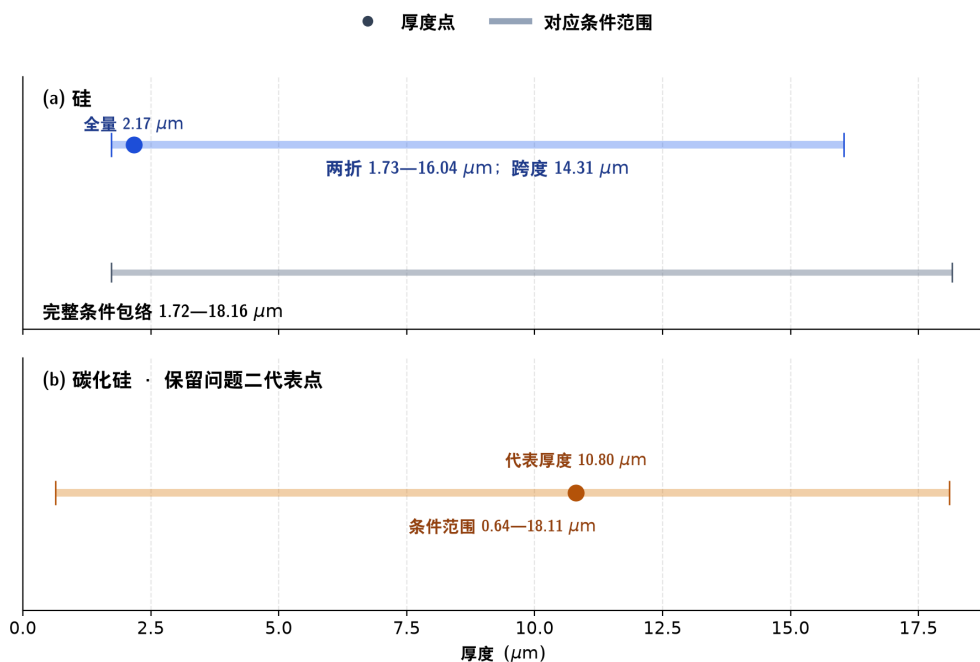


图 13 两材料厚度随切分与参数设置变化

匹配合成数据的厚度平均绝对相对误差为 0.0008% (表7); 正式谱形分数在问题二为 1.2074 (表9)、问题三为 1.0231 (表11), 均无量纲。前者以合成光谱的已知厚度为参照, 后两者以留出反射率为参照, 分别评价厚度恢复与曲线预测。

曲线预测仍有覆盖缺口: 问题二测试覆盖率为 49.06%, 低于 75% 校准分位水平; 问题三为 73.91%, 低于 90% 名义目标。图9按观测点统计反射率预测区间的实际覆盖, 再与各自的校准分位或名义目标比较。

光学条件扰动后, 问题一固定合成样本的厚度重估为 6.66–10.01 μm ; 问题二合并切分、剖面与参数情景, 范围为 0.64–18.11 μm (图7)。这些跨度记录厚度与光学参数的相互补偿, 硅另考察连续区段差异, 碳化硅沿用问题二基准。以下两小节先厘清误差参照, 再追问厚度为何随重估改变。

5.1 误差分析: 连续留段与经验覆盖

碳化硅的谱形预测有所改善, 区间覆盖却出现缺口。误差检验先拆开这两件事, 再用材料分项和合成回收辨明测厚误差的参照。

原型比较曾给出主方法误差 1.1717; 改用相同连续留段复核后, 正式误差为 1.2074, 低于二次趋势的 1.4404, 改善 16.17%, 故采用正式评分衡量预测增益。上述误差均为无量纲的标准化反射率均方根误差, 图8中的局部残差对应这一比较。

预测增益没有消除漏盖: 测试反射率仅有 157/320 点落入预测区间, 经验覆盖率为 49.06%。它衡量留出观测落入区间的比例; 图9与表13将这一比例和区间宽度并列。问题二采用校准残差的 75% 分位确定半宽, 再由留出的测试块统计实际覆盖。

表 13 正式实测反射率的点覆盖与整块覆盖

对象	分位/名义	点覆盖	全宽均值	整块覆盖	漏盖点
碳化硅	75%/—	157/320	0.7424	3/8	163
两材料	90%/90%	473/640	3.0875	9/16	167

表中全宽以反射率百分点计, 两材料行合计硅与碳化硅的完整往返场预测。整块覆盖要求该块全部观测落入同一点预测带, 漏盖点保留逐点预测失败的次数。此处统计的整块覆盖不是另构造的块同时预测带覆盖; 连续谱点彼此相关, 点覆盖与整块覆盖分别检验该点预测带在单个观测及整段观测上的表现。

材料分项揭示了总体均值掩盖的冲突。完整往返场误差 1.0231 高于两束的 0.9812; 硅附件三、四各有 2 个负改善块。表43逐项保留差值, 图11与图12分别展示局部残差和块间方向; 局部优势没有转化为总体预测改善。

第三问原第三版往返场估计器在低损耗、强损耗和无高阶负对照的 9 例匹配情景中, 最大厚度相对误差绝对值为 0.0521%, 但近优条件包络仅有 1/9 包含真厚度。表32保留全部已知厚度案例; 这组固定光学参数检验衡量厚度搜索的回收能力, 与现行候选整合后的实测结果分列。

色散失配时, 厚度相对误差绝对值分别为 3.71%、8.96% 和 7.35%。这组检验将估计模型中的色散设为零, 偏差随之放大, 说明匹配光学参数时的小回收误差不能代表光学关系失配时的测厚表现。

合成近优包络收集固定生成条件下训练目标接近最优的厚度解; 实测多情景包络还联合区段切分和

光学条件变化，构造对象不同。前者的真值包含比例是这组模拟的经验计数，后者没有实测厚度真值可供同样计数，二者均未建立概率覆盖保证。

模型失配也放大了问题一的厚度误差：平均绝对相对误差由 0.0008% 升至 0.3028% (图5)，共同匹配组的常折射率峰距基线失败 0 次。该项检验与第三问的直接厚度回收分别报告。合成回收以已知厚度为参照核对求解过程，光学参数扰动则考察固定观测下的厚度变化，下节沿这一方向展开重估比较。

5.2 灵敏度分析：重新估计后的厚度变化

问题一先固定同一组合成光谱，考察光学条件改变后厚度怎样移动，再将这一比较延伸至实测材料。每项扰动均重新估计厚度，其余未固定参数随之重估。

合成例在折射率、色散斜率、偏振、平滑和角度扰动下，厚度落在 6.66–10.01 μm ，最大绝对漂移为 2.01 μm 。双束界面场反演仍给出有限厚度，但未知光学条件足以使它发生微米量级移动。

实测碳化硅放大了厚度与折射率的互相补偿：切分、剖面 and 满足参数约束的情景合并后，厚度范围为 0.64–18.11 μm ；五折极差除以中位数为 0.7604，无量纲。图7分开呈现这些来源，问题三的碳化硅沿用同一范围。

为分开光学变化与优化继续推进的影响，硅次折增加同起点的未扰动控制：控制厚度为 16.0477 μm ，衬底对比增加 20% 后为 13.8276 μm ，扣除控制后的厚度变化为 -2.2201 μm 。两个终点均满足收敛标准，因而这对差值对应同一起点附近的光学条件响应。

原第三版的参数扰动保留为历史对照：硅厚度变化 -0.4571 至 3.9259 μm ，碳化硅变化 -1.6720 至 2.1261 μm 。图16和表14保留这些相对原折基准的差值；它们没有扣除未扰动优化推进，与同起点控制差分别解释。

表 14 扰动重估改变厚度与拟合损失

重估对象	扰动设置	厚度或差/ μm	训练损失	判断
合成厚度	折射率曲线 $\pm 20\%$	6.66–10.01	—	分支移动
碳化硅厚度	原设置	10.80	0.2747	参照
碳化硅厚度	折射率 $-3\%/+3\%$	8.21/7.96	0.3384/0.3294	分支移动
碳化硅厚度	窗口下/上界 $+100$	11.41/8.89	0.2005/0.3161	分支移动
碳化硅厚度	网格 96 $\rightarrow$ 192 点	13.53	0.2744	分支移动
硅历史差	光学、响应与窗口	-0.4571–3.9259	—	分支移动
碳化硅历史差	光学、响应与窗口	-1.6720–2.1261	—	分支移动

网格加密使碳化硅厚度移动 25.27%，而原代表点的损失仅比补查最低值高 0.1167%。这是固定点集、模型和物理边界后的数值搜索变化，不能与改变光学假设的情景变化混称为同一种不确定性。表25保留两类近优点及其边界状态。

表14中窗口移动以 cm^{-1} 计，斜线前后逐项对应；不同窗口的损失对应不同点集。异常原值均在主拟合窗口外，保留原值并不改变拟合点集，故裁剪异常不能解释厚度改善。

窗口比较曾沿用原波段，两种边界结果重合；因此改为各向内移动 100 cm^{-1} ，在既定训练成员与新窗口的交集上重估。两折按所移动的边界分别保留每角 273 或 292 个训练点，选段影响才得以分开。

光学与角度扰动取折射率 $\pm 3\%$ 、衬底对比及损耗 $\pm 20\%$ 、入射角 $\pm 0.5^\circ$ ，偏振分别取两种纯态。重估后，硅沿用完整往返场，碳化硅保留两束基准。厚度响应记录这些设置对估计的影响，窗口和网格比较则揭示分支移动；下一章据此评价方法与推广条件。

六、模型评价与推广：数值答案及适用范围

硅全量厚度为 2.17 μm ，非概率条件包络为 1.72–18.16 μm 。上一章的重估比较显示，厚度分支会随光学参数和窗口设置移动，本章据此评价双角约束的作用及后续标定方向。

碳化硅厚度为 10.80 μm ，厚度重估范围为 0.64–18.11 μm (图13)。碳化硅沿用两束基准，未施加多光束修正 (表11)。

区段重估使硅厚度落在相距较远的分支；碳化硅的双角留段误差较二次趋势降低。完整往返场的高阶收益跨折反向，说明增加返回光并未稳定改善谱形预测。表13保留两问反射率测试覆盖的分子与分母，曲线预测区间仍有覆盖缺口。

现有双角观测缺少折射率曲线、相干长度及仪器分辨率标定。下文分别讨论双角约束的优点、光学参数互补的缺点，以及新增角度、标准厚度样片与光学标定的推广方向。

6.1 优点：两角约束与可退化场模型

同片双角应共同解释条纹，同时保留各自的响应差异。

共享厚度、逐角响应使条纹与背景分开解释。图8与表9保留连续留段对照，避免逐角求厚度后平均。

场模型也能相互核对：直接取模平方与交叉项展开的最大差为 1.7×10^{-16} (反射率比例)，图17验证了两种强度表达的一致性。

完整往返场沿用同一首返回场，往返振幅比趋零即退回两束。图10展示两模型的代数联系；分别估计的厚度差仍包含光学参数补偿。

测厚与高阶判断分开，图13与表11保留材料处理的区别，测厚不以检出高阶为前提。厚度与折射率的补偿、外部标定缺失，则是下节讨论的局限。

6.2 缺点：光学参数互补与外部标定缺失

连续波段重估后的厚度仍然分散，厚度重估范围也随光学假设扩展。这使评价重点从曲线拟合好坏转向厚度是否被观测唯一确定。

碳化硅补充五折连续留段检验后，厚度分布于 5.53–13.51 μm ，极差与中位数之比为 0.7604。部分分支超出原范围，因此将全部有效切分纳入范围；图7保留了这种波段依赖。

波段依赖反映厚度与未知折射率变化的互相补偿，即不同厚度也能配出相近条纹。纳入损失接近最小值的近优候选及参数扰动后，范围扩展至 0.64–18.11 μm ；这一跨度由已检验情景的最外端点确定，反映现有观测对厚度的约束较弱。

物理解释还受观测条件制约：相干性、空间重叠、仪器分辨率和偏振状态四类资料均缺失（表10）。完整往返场模型在部分区段改善谱形，总体误差却高于两束；物理可观测条件仍有缺项，碳化硅沿用问题二基准（图13）。

区间覆盖也存在缺口：问题二、三的反射率测试经验覆盖率分别为 49.06%、73.91%，前者低于 75% 校准分位水平，后者低于 90% 名义目标（图9、表13）。校准残差给出的区间宽度未能充分容纳测试偏差，而真实附件的厚度真值也缺少独立参照。推广应从光学常数、标准厚度样片和重复测量补足这些缺口。

6.3 推广：新增角度和标定可改善辨识

双角约束仍难分开厚度与折射变化，推广应先补充观测，而非增加模型阶数。现有入射角只有 10° 和 15°（图1）；建议增加角度，借层内传播方向的差别（表5）检验各角度能否共用同一厚度和折射率曲线，冻结参数预测留出角度，用光谱误差衡量改进。

标准厚度样片能为厚度提供外部参照。针对图7、图13与表14中的分支分散，建议独立测定折射率与消光系数，用已知厚度约束往返衰减，再比较独立样片标定前后的厚度偏差，检验参数补偿是否减弱。

高阶项还需仪器观测支持。针对图10与表10，建议补测相干尺度、空间重叠、分辨率和偏振状态，再以同条件重复测量比较高阶谱形与噪声，用厚度离散程度检验重复性。这些均属后续实验建议，检验时以现有测厚结果为基准。

七、参考文献：薄膜光学与不确定性方法

前章提出光学标定与重复测量的方向，本节说明界面反射和区间检验的方法来源，区分物理推导与观测核对。空气、外延层和衬底构成三层介质，两种偏振的界面导纳及往返场在模型建立部分由边界条件和逐束叠加推出。表3与表5区分了观测给定量和待估参数；附件没有折射率曲线，通用光学关系中的材料参数保留为待估量。

连续留段把相邻谱点保留在同一区段，再检查未参与拟合的光谱。问题二和问题三分别使用 320 个和 640 个测试点核对反射率预测区间，表13汇总其经验覆盖，即落入区间的测试点所占比例。以校准残差的经验分位构造区间的思路参考文献 [1]；由于谱点连续相关，本文按留出点实计覆盖，检验对象是反射率预测，而非几何厚度。

问题一的 19.54 μm 是周期等效量 $T = 5000/\Delta\sigma$ ，包含色散和界面相位变化的贡献，与后两问由光学模型求得的拟合厚度分列。各问的扰动比较记录厚度随光学条件的变化，文献提供通用方法，本题观测与模型计算支持样片结论。附录按问题列出计算材料、数据口径与完整比较，便于复核公式如何落实到观测与结果。

[1] Vovk V, Gammerman A, Shafer G. Algorithmic Learning in a Random World[M]. New York: Springer, 2005.

八、附录：核心源码与补充数值

附录把正文的厚度答案、光场公式和对照结果放回各自的数据对象中，便于逐项复核。材料按两束场、同片双角估计和完整往返场的顺序展开，保留计算所需的输入约定与依赖，不用一串外部链接代替计算过程。

原始观测来自 4 个附件，每谱均有 7469 个共同波数点；附件的数量与独立晶圆的数量不是同一概念。表15按材料和计算任务列出产物，使相同名称的厚度结果能够对应到不同问题，而不是脱离点集与模型单独引用。

表 15 三问计算产物与复核对象

对象	产物名称	对应内容
原始观测	附件一至附件四工作簿	材料、角度、原始波数与反射率
输入核对	数据档案.json	附件标识、材料归属与单位
合成条件	合成设计.json	已知厚度情景及生成参数

对象	产物名称	对应内容
光学情景	路线侦察.json	往返场初始情景及共同约束
问题一	求解 _ 问题 1.py	两束场、合成验证与周期诊断
问题二	求解 _ 问题 2.py	共同厚度、连续留段与条件范围
问题三	求解 _ 问题 3.py	同条件候选整合与两级精修
问题一	模型验证.json	两束展开、单位与真实谱周期
问题一	合成验证.json	已知厚度、失配情景与峰距对照
问题一	独立角度切分验证.json	整角度留出预测及逐例误差
问题二	厚度结果.json	同片厚度、逐折厚度与条件范围
问题二	参数剖面.csv	厚度与折射率的近等损失组合
问题二	验证.json	连续留段、留角度与基线对照
问题二	基准交接.json	向完整往返比较提供同片基准
问题三	厚度结果.json	全量硅厚度与碳化硅基准
问题三	同口径对照.json	逐块误差与有限级数核对
问题三	条件判定.json	逐附件支持与碳化硅处理依据
问题二、三	留段预测.csv	原始行号、观测与留出预测
问题二、三	区间覆盖.json	反射率覆盖的分母与构造
问题二、三	灵敏度.csv	参数及窗口扰动后的重新估计

产物按各问所属目录区分，同名记录不相互替代。表中光谱预测误差检验的是曲线解释能力，合成厚度误差检验的是已知光学条件下的反演；两类检验分别保留，避免把曲线贴合程度当成真实晶圆的测厚精度。以下先核对一次返回场，再把共同厚度约束带入实测双角光谱。

问题一的两束场由界面直接反射和从衬底界面返回的部分组成，两者叠加产生条纹。早期尝试曾考虑直接用实测反射率评价绝对厚度误差，但附件没有厚度真值及折射率曲线，因此改用已知光学参数的合成光谱核对反演关系，并把真实光谱的周期诊断单独保留。

真实谱的周期等效量为 $19.54\mu\text{m}$ ，而匹配合成情景的平均厚度相对误差为 0.0008%；它们分别回答条纹给出了什么量、给定光学条件后能否反推出厚度。表16把场的代数核对与厚度比较分开，完整的计算内容见程序2。

表 16 两束场核对与合成误差分项

核对对象	数值	数值含义
场强两种算法	1.6653×10^{-16}	反射率比例的最大绝对差
共同匹配情景	0.0008%	平均厚度绝对相对误差
新增匹配情景	0.0011%	平均厚度绝对相对误差
模型失配情景	0.3028%	平均厚度绝对相对误差
常折射率峰距	2.5875%	同组情景的朴素对照
双向整角度留出	0.0018%	留出厚度平均相对误差
合成锚例扰动	6.66–10.01 μm	同一合成观测的重估范围

代数一致说明两种场强写法描述的是同一物理对象；失配情景则改变生成光谱的光学条件，检验问题从“计算是否一致”变成“假设改变后厚度如何移动”。因此程序保留两种强度写法、同型峰距基线和整角度留出，而非只展示最小的一项误差。

完整实现见清单2，此处不重复列出。

两束关系转入碳化硅实测谱后，需要让双角共同决定厚度和折射变化，同时让每个角度解释自己的慢变基线、振幅与固定相位。科学尝试比较了这种共享参数方式与逐角厚度简单平均，发现不同波段的反射率起伏差异明显，且缺少折射率曲线，因而保留共同物理参数和逐角低阶响应。

问题二的 $10.80\mu\text{m}$ 来自每角 480 点的共同原坐标样本，并非主窗口全部原始点的另一名称。表17同时列出点估计、两折及五折厚度，保持连续区段变化与整体拟合的对应关系。

条件范围 0.64–18.11 μm 汇集了数据切分、厚度与折射率补偿关系及合法参数情景，含义是改变这些条件后得到的厚度跨度。它与共同样本的整体拟合并列，说明范围的构造比单次拟合多考虑了哪些变化。

表 17 碳化硅厚度按点集与范围分列

对象	厚度 / μm	构造
同片非触边代表	10.80	原搜索代表，另保留触边候选
两折连续留段	13.08, 10.65	按各折训练区段重新估计
五折连续留段	13.37, 10.49, 6.53	前三折的独立拟合厚度
五折连续留段	13.51, 5.53	后两折的独立拟合厚度
条件范围	0.64–18.11	切分、剖面与合法情景并集

厚度范围与反射率预测覆盖有不同的分母和解释对象。程序7将二者分别计算：厚度范围追踪物理

参数随条件的变化，反射率覆盖核对留出光谱点是否落入预测带，后者不会被用来给前者附加概率含义。

完整实现见清单7，此处不重复列出。

共同样本的厚度与条件范围明确后，完整往返场的比较继续沿用相同留出区段，并另建硅的主窗口全量对象。所谓完整往返，就是在首个返回场之后，让后续每轮返回再乘同一衰减和相位因子，最后将所有返回场相加；截去后续返回便恢复两束关系。

硅的全量估计为 $2.17\ \mu\text{m}$ ，所用主窗口每角含 5392 个原始点。表18将它与计划折分放在不同的行，避免把训练区段产生的厚度差异压缩成一个平均值。

计划折分每角 480 点，两次训练得到 $1.73\ \mu\text{m}$ 与 $16.04\ \mu\text{m}$ ，差别指向区段选择对厚度分支的影响。碳化硅保留 $10.80\ \mu\text{m}$ 基准，其共同样本与光学条件不会随硅全量对象的建立而改变。

表 18 两材料厚度保留全量与折分区别

材料与对象	点数/角	厚度 / μm	模型
硅原始全量	5392	2.17	完整往返
硅计划折分	480	1.73, 16.04	完整往返
碳化硅基准	480	10.80	问题二两束
碳化硅范围	480	0.64–18.11	问题二条件并集

折间厚度的变化对应同一材料更换训练区段，材料之间的差别则对应不同晶圆，二者不能合成一条共同的误差范围。保留这两层区别后，完整往返场与两束场的优劣才有明确的比较对象。

同段比较时，完整往返的标准化反射率误差为 1.0231，两束对照为 0.9812；实际尝试同时发现硅十度谱存在负改善区段，因此程序保留逐块比较，不让总体平均掩盖局部反向变化。表19保留两问各自的基线尺度，程序11给出对应的往返求和、配对评分和材料处理分支。

表 19 连续留段误差按各问基线比较

问题	比较方法	标准化反射率误差
问题二	共享厚度两束模型	1.2074
问题二	二次趋势	1.4404
问题三	完整往返场	1.0231
问题三	两束截断场	0.9812
问题三	二次趋势	1.4434
问题三	训练均值	2.1929

总体误差未下降，说明高阶返回没有带来整体预测改善；相干长度、光束空间重叠和仪器分辨率则回答这些返回能否在测量中形成可分辨干涉。两种问题分开计算和记录，碳化硅的处理仍使用问题二基准，逐附件判断和窗口扰动保留在相应产物中。

完整实现见清单11，此处不重复列出。

各问完整程序保留依赖导入、数据读取、核心函数和执行入口。问题三以清单??为现行入口，调用清单11的基础引擎和清单??的独立公式；冻结输入及数值对应关系见清单10。程序1列出前两问的执行命令；问题三保持现有附件与碳化硅基准不变，按清单12隔离重算，基础引擎不再单独充当现行计算，复算也不自动覆盖本文结果。

Listing 1: 从工作根执行前两问计算

```
python3 求解/问题1/求解_问题1.py
python3 求解/问题2/求解_问题2.py
```

这些计算入口将模型、点集和结果逐一对应，补充材料再据此展开源行索引、参数对照及逐块数值，而不改变正文已经给出的答案。

8.1 问题一源码：两束场与合成验证

一次往返场展开在本附录中保持原来的光路边界：外延层上表面的直接反射场，与进入外延层、到达衬底界面后返回的首束场相加。空气、外延层和衬底的界面关系沿用图4，角度与单位沿用表5；入射角分别为 10° 和 15° ，估计反射强度时不加入后续往返的分母。这样，合成检验与真实条纹诊断都从同一个一次返回模型出发。

直接求光场模平方与展开交叉项之间，最大绝对差为 1.7×10^{-16} ，以反射率比例计；厚度输入 $8\ \mu\text{m}$ 时，内部换为 $0.0008\ \text{cm}$ ，回换后厚度不变。图17中的逐点一致性与表20的基础核验相互对应，把代数展开和量纲换算分别落到可核对的量上。

表 20 一次返回场的代数与量纲核验

核验对象	数值或关系	核对内容
两种强度计算	1.7×10^{-16}	最大绝对差, 比例
零界面返回项	0	退化差, 比例
厚度内部换算	$8 \mu\text{m} \rightarrow 0.0008 \text{ cm}$	指数中的单位相消
厚度回换	$8 \mu\text{m}$	输入与输出一致
同型峰与峰谷系数	5000 / 2500	微米输出的换算系数

代数核验检查的是同一个物理关系的不同写法, 单位核验检查的是传播相位中的长度尺度。它们与材料光学条件的准确程度是两件事; 基础关系闭合之后, 还要改变已知厚度及光学条件, 观察反演是否能回到给定输入。

匹配合成的 24 例情景得到平均厚度绝对相对误差 0.0008%, 表21保留了每例输入, 而不是只留下一个平均值。表中 d_* 为合成时给定的几何厚度, n_0 为外延层折射率基值, a_1 为折射率随归一化波数线性变化的系数, s_R 为加入相关噪声之前的高斯扰动标准差, 按反射率比例计。

表 21 已知厚度的共同匹配情景

情景编号	$d_*/\mu\text{m}$	n_0	a_1	s_R
01	3	2.4	0	0
02	3	2.4	0	0.001
03	3	2.4	0.08	0
04	3	2.4	0.08	0.001
05	3	3.4	0	0
06	3	3.4	0	0.001
07	3	3.4	0.08	0
08	3	3.4	0.08	0.001
09	8	2.4	0	0
10	8	2.4	0	0.001
11	8	2.4	0.08	0
12	8	2.4	0.08	0.001
13	8	3.4	0	0
14	8	3.4	0	0.001
15	8	3.4	0.08	0
16	8	3.4	0.08	0.001
17	20	2.4	0	0
18	20	2.4	0	0.001
19	20	2.4	0.08	0
20	20	2.4	0.08	0.001
21	20	3.4	0	0
22	20	3.4	0	0.001
23	20	3.4	0.08	0
24	20	3.4	0.08	0.001

这些情景把厚度、折射率变化与观测扰动分开组合, 使无噪声输入和含噪声输入都有对应的比较对象。给定厚度参与光谱生成与误差计算, 厚度搜索接收的则是波数、反射率、光学参数和角度, 真值不参与搜索位置的选择。

模型失配使平均厚度绝对相对误差升至 0.3028%; 这里的失配, 是指生成光谱时改变了光学关系, 而反演仍按原先关系估计厚度。表22把这类误差与匹配情景、同谱峰距基线并列, 接续表7中的方法比较。

表 22 匹配与失配情景的厚度误差

情景	例数	平均误差/%	对照条件
共同匹配	24	0.0008	线性色散与已知偏振
新增匹配	16	0.0011	曲率、偏振及噪声变化
模型失配	10	0.3028	生成与反演的光学条件不同
常折射率峰距	24	2.5875	共同匹配光谱的峰距估计

误差分组说明, 代数实现一致之后, 生成光谱与反演模型之间的差别仍会进入厚度估计。新增情景保留了已知光学条件, 失配情景则分别改变色散曲率、偏振、包络、基线、消光及返回次数; 其中多次返回改变的是合成观测, 估计反射强度的表达式始终停在一次返回。

真实条纹的搜索曾被候选区间限制: 早期尝试中, 第二角光谱的最优周期恰好落在 80 cm^{-1} 的上限,

因此扩大周期范围，并把反射率与正余弦项都对同一二次趋势取残差。这项调整把缓慢变化的谱形与条纹振荡分开比较，避免把候选范围的端点当成条纹的周期。

统一处理趋势后的两谱周期分别为 254.85 cm^{-1} 和 256.99 cm^{-1} ，对应各自的外部入射角，而非两次独立测厚。其输入保留原始波数坐标与百分数反射率，周期搜索与合成厚度回收分别输出，因而两类数值的参照对象始终清楚。

两谱有效周期的中位数按 $T = 5000/\Delta\sigma$ 换算为 $19.54 \mu\text{m}$ 。 T 是有限窗口周期统计量，色散和界面相位贡献未被分离，不能直接解释为 dq 或几何厚度。主文式(14)的局部差商关系说明，厚度乘传播因子的近似还要求色散与界面相位导数项可忽略；把局部关系用于整个窗口，则另需相位斜率近似恒定。这些近似未由本次周期统计验证。

完整的计算对应清单2，合成情景的全部参数对应清单3。前者包含原始观测读取、界面场、厚度搜索、合成对照、整角度留出及真实周期计算；后者补足相关噪声、偏振和失配生成条件，使平均误差可以追溯到逐例输入。

清单 2: 一次返回场及合成验证的完整实现

```

1  """问题1: 双束界面场的一次往返反演。
2  主模型严格对应题面一次反射/透射情形:  $E=r_01+t_01*t_{10}*r_{12}*z$ ,
3  不含问题三的多次往返分母。脚本只在显式执行时计算, 使用标准库读取
4  XLSX; 真实附件只用于可复核的光谱诊断, 几何厚度不在缺少折射率时强行发布。
5  所有合成观测均由数据/问题1_冻结合成输入/合成设计.json生成, 真厚度只在
6  评分阶段出现, 且每完成一块结果即原子写盘。
7  """
8  import cmath
9  import csv
10 import hashlib
11 import itertools
12 import json
13 import math
14 import os
15 import posixpath
16 import random
17 import time
18 from pathlib import Path
19 from zipfile import ZipFile
20 import xml.etree.ElementTree as ET
21 ROOT = Path(__file__).resolve().parents[2]
22 RESULTS = ROOT / "求解/问题1/结果"
23 FROZEN = ROOT / "数据/问题1_冻结合成输入"
24 DESIGN = FROZEN / "合成设计.json"
25 ANGLES = (10.0, 15.0)
26 T0 = None
27 SOFT_SECONDS = 660.0
28 THICKNESS_MIN = 0.5
29 THICKNESS_MAX = 40.0
30 XML_NS = {"表": "http://schemas.openxmlformats.org/spreadsheetml/2006/main"}
31 REL_NS = "http://schemas.openxmlformats.org/officeDocument/2006/relationships"
32 class BudgetStop(Exception):
33     pass
34 def elapsed():
35     if T0 is None:
36         return 0.0
37     return time.monotonic() - T0
38 def check_budget():
39     if elapsed() >= SOFT_SECONDS:
40         raise BudgetStop("接近11分钟软截止, 保留已写结果")
41 def atomic_json(path, value):
42     path.parent.mkdir(parents=True, exist_ok=True)
43     tmp = path.with_name("." + path.name + ".tmp")
44     with tmp.open("w", encoding="utf-8") as stream:
45         json.dump(value, stream, ensure_ascii=False, indent=2, allow_nan=False)
46         stream.write("\n")
47         stream.flush()
48         os.fsync(stream.fileno())
49     os.replace(tmp, path)
50 def atomic_csv(path, rows, fields):
51     path.parent.mkdir(parents=True, exist_ok=True)
52     tmp = path.with_name("." + path.name + ".tmp")
53     with tmp.open("w", encoding="utf-8", newline="") as stream:
54         writer = csv.DictWriter(stream, fieldnames=fields, extrasaction="ignore")
55         writer.writeheader()
56         writer.writerows(rows)
57         stream.flush()
58         os.fsync(stream.fileno())
59     os.replace(tmp, path)
60 def save(name, value):
61     payload = dict(value)
62     payload["实际用时秒"] = elapsed()
63     atomic_json(RESULTS / name, payload)

```

```

64 def finite(x):
65     return isinstance(x, (int, float)) and math.isfinite(float(x))
66 def sha256(path):
67     return hashlib.sha256(path.read_bytes()).hexdigest()
68 def read_xlsx(path):
69     """按工作簿关系定位第一张表，只读两列数值。"""
70     with ZipFile(path) as book:
71         workbook = ET.fromstring(book.read("xl/workbook.xml"))
72         rels = ET.fromstring(book.read("xl/_rels/workbook.xml.rels"))
73         targets = {x.get("Id"): x.get("Target") for x in rels}
74         sheet = workbook.find("表:sheets/表:sheet", XML_NS)
75         target = targets[sheet.get("{ " + REL_NS + " }id")]
76         member = target.lstrip("/") if target.startswith("/") else posixpath.normpath("xl/" + target)
77         root = ET.fromstring(book.read(member))
78         rows = []
79         for row in root.findall("表:sheetData/表:row", XML_NS):
80             number = int(row.get("r"))
81             if number == 1:
82                 continue
83             cells = {cell.get("r"): cell for cell in row}
84             vals = []
85             for col in ("A", "B"):
86                 cell = cells[f"{col}{number}"]
87                 if cell.get("t", "n") != "n" or cell.find("表:f", XML_NS) is not None:
88                     raise ValueError(f"{path.name}含非数值或公式")
89                 value = float(cell.findtext("表:v", namespaces=XML_NS))
90                 if not finite(value):
91                     raise ValueError(f"{path.name}含非有限数值")
92                 vals.append(value)
93             rows.append({"原始行号": number, "波数_每厘米": vals[0], "反射率_百分比": vals[1]})
94 if len(rows) != 7469:
95     raise ValueError(f"{path.name}数据行数不是7469")
96 if any(a["波数_每厘米"] >= b["波数_每厘米"] for a, b in zip(rows, rows[1:])):
97     raise ValueError(f"{path.name}波数不是严格递增")
98 return rows
99 def read_real_inputs():
100     archive = json.loads((ROOT / "交接/数据档案.json").read_text(encoding="utf-8"))
101     spectra = []
102     coordinates = None
103     for filename, angle in zip(("附件1.xlsx", "附件2.xlsx"), ANGLES):
104         check_budget()
105         path = ROOT / "数据" / filename
106         entry = next(x for x in archive["文件档案"] if x["文件名"] == filename)
107         digest = sha256(path)
108         if digest != entry["文件哈希"]:
109             raise ValueError(f"{filename}与数据档案哈希不符")
110         rows = read_xlsx(path)
111         current = [r["波数_每厘米"] for r in rows]
112         if coordinates is not None and current != coordinates:
113             raise ValueError("同片两角度原始波数不一致")
114         coordinates = current
115         spectra.append({"源附件": filename, "材料": entry["材料"], "入射角_度": angle,
116             "文件哈希": digest, "观测": rows})
117     summary = {
118         "问题": 1, "运行状态": "真实附件已读取；因缺少光学常数只发表观诊断",
119         "共同原始波数点数": len(coordinates),
120         "波数范围_每厘米": [coordinates[0], coordinates[-1]],
121         "波长范围_微米": [10000.0 / coordinates[-1], 10000.0 / coordinates[0]],
122         "附件诊断": [{"源附件": s["源附件"], "入射角_度": s["入射角_度"],
123             "文件哈希": s["文件哈希"], "点数": len(s["观测"]),
124             "零值点数": sum(r["反射率_百分比"] == 0 for r in s["观测"]),
125             "超过百分之百点数": sum(r["反射率_百分比"] > 100 for r in s["观测"]),
126             "原始首点": s["观测"][0], "原始末点": s["观测"][-1]} for s in spectra],
127         "共同抽样索引": [i * 7468 // 255 for i in range(256)],
128         "输入边界": "题面附件未提供折射率、消光系数、偏振态、真实厚度；不发布未经光学条件支持的几何厚度。",
129     }
130     save("实测输入核验.json", summary)
131     return coordinates, spectra
132 def passive_root(value):
133     root = cmath.sqrt(value)
134     if root.imag < 0 or (abs(root.imag) < 1e-15 and root.real < 0):
135         root = -root
136     return root
137 def index_at(sigma, params):
138     x = (sigma - 2200.0) / 1800.0
139     real = params["折射率基值"] + params.get("色散斜率", 0.0) * x + params.get("色散曲率", 0.0) * x * x
140     kappa = params.get("消光系数", 0.0)
141     layer = complex(real, kappa)
142     return layer, layer + complex(params.get("衬底折射率增量", 0.8), 0.0)
143 def interface_components(sigma, angle, params):
144     layer, substrate = index_at(sigma, params)
145     indices = (1.0 + 0j, layer, substrate)
146     transverse = math.sin(math.radians(angle))

```

```

147 q = tuple(passive_root(n * n - transverse * transverse) for n in indices)
148 result = []
149 for polarization in ("垂直", "平行"):
150     admittance = q if polarization == "垂直" else tuple(n * n / qj for n, qj in zip(indices, q))
151     y0, y1, y2 = admittance
152     r01 = (y0 - y1) / (y0 + y1)
153     r12 = (y1 - y2) / (y1 + y2)
154     r10 = (y1 - y0) / (y1 + y0)
155     t01 = 2.0 * y0 / (y0 + y1)
156     t10 = 2.0 * y1 / (y1 + y0)
157     result.append((r01, t01 * t10 * r12, r10 * r12, q[1]))
158 return result
159 def propagation(sigma, q, thickness_um):
160     return cmath.exp(4.0j * math.pi * sigma * q * thickness_um * 1e-4)
161 def direct_reflectance(sigma, angle, params, thickness_um, returns=1):
162     weights = (params.get("垂直偏振权重", 0.5), 1.0 - params.get("垂直偏振权重", 0.5))
163     total = 0.0
164     for weight, (surface, echo, repeat, q) in zip(weights, interface_components(sigma, angle, params)):
165         field = surface
166         if returns > 0:
167             z = propagation(sigma, q, thickness_um)
168             term = echo * z
169             for _ in range(int(returns)):
170                 field += term
171                 term *= repeat * z
172             total += weight * abs(field) ** 2
173     return total
174 def expanded_coefficients(sigma, angle, params):
175     weights = (params.get("垂直偏振权重", 0.5), 1.0 - params.get("垂直偏振权重", 0.5))
176     surface_power = 0.0
177     echo_power = 0.0
178     cross = 0j
179     comps = interface_components(sigma, angle, params)
180     for weight, (surface, echo, _repeat, _q) in zip(weights, comps):
181         surface_power += weight * abs(surface) ** 2
182         echo_power += weight * abs(echo) ** 2
183         cross += weight * surface.conjugate() * echo
184     return sigma, surface_power, echo_power, cross, comps[0][3]
185 def expanded_reflectance(coefficient, thickness_um):
186     sigma, surface_power, echo_power, cross, q = coefficient
187     z = propagation(sigma, q, thickness_um)
188     return surface_power + echo_power * abs(z) ** 2 + 2.0 * (cross * z).real
189 def make_optics(coordinates, params, angles):
190     return [[expanded_coefficients(s, a, params) for s in coordinates] for a in angles]
191 def loss(optics, observations, thickness_um):
192     total = 0.0
193     count = 0
194     for angle_optics, observed in zip(optics, observations):
195         for coefficient, target in zip(angle_optics, observed):
196             total += (expanded_reflectance(coefficient, thickness_um) - target) ** 2
197             count += 1
198     return total / max(1, count)
199 def golden(objective, left, right):
200     phi = (math.sqrt(5.0) - 1.0) / 2.0
201     x1, x2 = right - phi * (right - left), left + phi * (right - left)
202     f1, f2 = objective(x1), objective(x2)
203     for _ in range(26):
204         check_budget()
205         if f1 > f2:
206             left, x1, f1 = x1, x2, f2
207             x2, f2 = left + phi * (right - left), objective(left + phi * (right - left))
208         else:
209             right, x2, f2 = x2, x1, f1
210             x1, f1 = right - phi * (right - left), objective(right - phi * (right - left))
211     return (x1, f1) if f1 < f2 else (x2, f2)
212 def estimate_thickness(coordinates, observations, params, angles=ANGLES):
213     """只接收坐标、观测、光学情景和角度；不接收真厚度。"""
214     optics = make_optics(coordinates, params, angles)
215     grid = [THICKNESS_MIN + (THICKNESS_MAX - THICKNESS_MIN) * i / 128.0 for i in range(129)]
216     scored = []
217     for d in grid:
218         check_budget()
219         scored.append((loss(optics, observations, d), d))
220     scored.sort()
221     step = (THICKNESS_MAX - THICKNESS_MIN) / 128.0
222     candidates = []
223     for _, center in scored[:4]:
224         left, right = max(THICKNESS_MIN, center - 1.6 * step), min(THICKNESS_MAX, center + 1.6 * step)
225         d, value = golden(lambda x: loss(optics, observations, x), left, right)
226         candidates.append({"厚度_微米": d, "联合残差平方均值_比例平方": value})
227     candidates.sort(key=lambda x: x["联合残差平方均值_比例平方"])
228     return {"厚度_微米": candidates[0]["厚度_微米"], "联合残差平方均值_比例平方": candidates[0]["联合残差平方均

```

```

    值_比例平方"], "近优候选": candidates, "粗网格点数": len(grid)}
229 def smooth(values, width):
230     radius = width // 2
231     return [sum(values[max(0, i - radius):min(len(values), i + radius + 1)]) / (min(len(values), i +
        radius + 1) - max(0, i - radius)) for i in range(len(values))]
232 def median(values):
233     if not values:
234         return None
235     ordered = sorted(values)
236     n = len(ordered)
237     return ordered[n // 2] if n % 2 else (ordered[n // 2 - 1] + ordered[n // 2]) / 2.0
238 def peak_spacing_baseline(coordinates, observations, params):
239     estimates = []
240     details = []
241     for angle, values in zip(ANGLES, observations):
242         y = smooth(values, 3)
243         peaks = [i for i in range(1, len(y) - 1) if y[i] >= y[i - 1] and y[i] > y[i + 1]]
244         gaps = [coordinates[b] - coordinates[a] for a, b in zip(peaks, peaks[1:]) if coordinates[b] >
        coordinates[a]]
245         delta = median(gaps)
246         q = math.sqrt(max(1e-12, params["折射率基值"] ** 2 - math.sin(math.radians(angle)) ** 2))
247         d = 5000.0 / (q * delta) if delta else None
248         if finite(d) and d > 0:
249             estimates.append(d)
250         details.append({"角度_度": angle, "峰数量": len(peaks), "同型峰距中位数_每厘米": delta, "厚度_微米":
        d})
251     return (sum(estimates) / len(estimates) if estimates else None), details
252 def gaussian(rng):
253     return rng.gauss(0.0, 1.0)
254 def build_specs(design):
255     base = design["基础情景"]
256     specs = []
257     index = 0
258     for d, n0, slope, noise in itertools.product(base["厚度_微米"], base["折射率基值"], base["色散斜率"],
        base["噪声标准差_比例"]):
259         specs.append({"编号": f"共同{index + 1:02d}", "类别": "共同匹配", "案例索引": index, "真厚度_微米":
        d,
        "折射率基值": n0, "色散斜率": slope, "色散曲率": 0.0, "噪声标准差_比例": noise,
        "噪声相关系数": base["噪声相关系数"], "垂直偏振权重": base["垂直偏振权重"],
        "衬底折射率增量": base["衬底折射率增量"], "消光系数": 0.0, "生成返回束数": 1,
        "拟合曲率": 0.0, "拟合权重": base["垂直偏振权重"]})
260         index += 1
261     for item in design["新增匹配情景"]:
262         item = dict(item)
263         specs.append({"编号": f"新增匹配{index - 23:02d}", "类别": "新增匹配", "案例索引": index,
        "真厚度_微米": item["厚度_微米"], "噪声相关系数": base["噪声相关系数"],
        "衬底折射率增量": base["衬底折射率增量"], "消光系数": 0.0,
        "生成返回束数": 1, "拟合曲率": item.get("色散曲率", 0.0), "拟合权重": item["垂直偏振权重"]})
264         index += 1
265     for item in design["失配情景"]:
266         item = dict(item)
267         specs.append({"编号": f"失配{index - 39:02d}", "类别": "模型失配", "案例索引": index,
        "真厚度_微米": item["厚度_微米"], "色散曲率": item.get("生成曲率", 0.0),
        "垂直偏振权重": item.get("生成权重", base["垂直偏振权重"]),
        "噪声相关系数": base["噪声相关系数"], "衬底折射率增量": base["衬底折射率增量"],
        "消光系数": item.get("生成消光系数", 0.0), "生成返回束数": item.get("生成返回束数", 1),
        "拟合曲率": 0.0, "拟合权重": item.get("拟合权重", base["垂直偏振权重"]})
268         index += 1
269     return specs
270 def params_from(spec, fit=False, scale=1.0):
271     if fit:
272         curvature = spec.get("拟合曲率", 0.0)
273         weight = spec.get("拟合权重", 0.5)
274         kappa = 0.0
275         substrate = 0.8
276     else:
277         curvature = spec.get("色散曲率", 0.0)
278         weight = spec.get("垂直偏振权重", 0.5)
279         kappa = spec.get("消光系数", 0.0)
280         substrate = spec.get("衬底折射率增量", 0.8)
281     return {"折射率基值": spec["折射率基值"] * scale, "色散斜率": spec.get("色散斜率", 0.0) * scale,
        "色散曲率": curvature * scale, "垂直偏振权重": weight, "消光系数": kappa * scale,
        "衬底折射率增量": substrate * scale}
282 def noise_seeds(spec):
283     seed = 20260909 + 2 * int(spec["案例索引"])
284     return [seed, seed + 1]
285 def generate_case(coordinates, spec, seed_override=None):
286     truth = params_from(spec, fit=False)
287     observations = []
288     seeds = noise_seeds(spec) if seed_override is None else [seed_override, seed_override + 1]
289     for angle_index, angle in enumerate(ANGLES):
290         rng = random.Random(seeds[angle_index])

```



```

305     previous = 0.0
306     values = []
307     for sigma in coordinates:
308         value = direct_reflectance(sigma, angle, truth, spec["真厚度_微米"], spec.get("生成返回束数",
1))
309         x = (sigma - 2200.0) / 1800.0
310         value = value * (1.0 + spec.get("生成增益", 0.0) * x) + spec.get("生成基线", 0.0) * x * x
311         sd = spec.get("噪声标准差_比例", 0.0)
312         innovation = sd * gaussian(rng)
313         rho = spec.get("噪声相关系数", 0.6)
314         error = math.sqrt(max(0.0, 1.0 - rho * rho)) * innovation if not values else rho *
previous + math.sqrt(max(0.0, 1.0 - rho * rho)) * innovation
315         previous = error
316         values.append(value + error)
317         observations.append(values)
318     return observations
319 def rmse(a, b):
320     return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)) / max(1, len(a)))
321 def model_validation(coordinates):
322     params = {"折射率基值": 2.8, "色散斜率": 0.08, "色散曲率": 0.0, "垂直偏振权重": 0.5, "消光系数": 0.0, "衬
底折射率增量": 0.8}
323     differences = []
324     for angle in ANGLES:
325         for sigma in coordinates[:32]:
326             for d in (3.0, 8.0, 20.0):
327                 direct = direct_reflectance(sigma, angle, params, d, 1)
328                 expanded = expanded_reflectance(expanded_coefficients(sigma, angle, params), d)
329                 differences.append(abs(direct - expanded))
330     zero_interface = dict(params)
331     zero_interface["衬底折射率增量"] = 0.0
332     zero_echo = abs(direct_reflectance(2199.9, 10.0, zero_interface, 8.0, 1) - direct_reflectance
(2199.9, 10.0, zero_interface, 8.0, 0))
333     sigma = 2200.0
334     h = 1e-3
335     q_plus = passive_root(index_at(sigma + h, params)[0] ** 2 - math.sin(math.radians(10.0)) ** 2)
336     q_minus = passive_root(index_at(sigma - h, params)[0] ** 2 - math.sin(math.radians(10.0)) ** 2)
337     # cmath.sqrt 即使在透明极限也返回 ``complex(..., 0)``; 这里的
338     # 参数为无吸收情景, 相位导数应取其实际部后再写入 JSON。
339     phase_slope = float(((sigma + h) * q_plus - (sigma - h) * q_minus) / (2.0 * h)).real)
340     result = {"问题": 1, "模型": "E=r01+t01*t10*r12*z; z=exp(4*pi*i*d_cm*sigma*q)",
"场展开一致性最大绝对差_比例": max(differences), "零界面返回项差_比例": zero_echo,
341     "被动根与传播衰减": "已检验: 采样参数下Im(q)>=0且|z|<=1", "单位换算": {"输入厚度_微米": 8.0, "厚度
_厘米": 8e-4, "回换厚度_微米": 8.0},
342     "相位导数中心差分_无量纲": phase_slope, "常折射率退化": {"同型系数_微米": 5000.0, "峰谷系数_微米"
: 2500.0, "系数比": 2.0},
"零界面结论": "r12=0时交叉干涉项为0; 零返回项与一次往返模型一致。"}
343     save("模型验证.json", result)
344 def solve_small_linear(matrix, vector):
345     """高斯消元解小型正规方程; 仅用于三列低阶基线, 避免引入额外依赖。"""
346     n = len(vector)
347     augmented = [list(row) + [vector[i]] for i, row in enumerate(matrix)]
348     for col in range(n):
349         pivot = max(range(col, n), key=lambda row: abs(augmented[row][col]))
350         if abs(augmented[pivot][col]) < 1e-14:
351             raise ValueError("周期投影的低阶基线矩阵奇异")
352         augmented[col], augmented[pivot] = augmented[pivot], augmented[col]
353         scale = augmented[col][col]
354         augmented[col] = [value / scale for value in augmented[col]]
355         for row in range(n):
356             if row == col:
357                 continue
358             factor = augmented[row][col]
359             augmented[row] = [a - factor * b for a, b in zip(augmented[row], augmented[col])]
360     return [augmented[i][-1] for i in range(n)]
361 def residualize(values, basis, normal=None):
362     """按同一低阶基线残差化响应或谐波列, 保证投影口径一致。"""
363     width = len(basis[0])
364     if normal is None:
365         normal = [[sum(row[i] * row[j] for row in basis) for j in range(width)] for i in range(width)]
366     rhs = [sum(row[i] * value for row, value in zip(basis, values)) for i in range(width)]
367     coefficients = solve_small_linear(normal, rhs)
368     return [value - sum(row[i] * coefficients[i] for i in range(width)) for row, value in zip(basis,
values)]
369 def apparent_dq(coordinates, rows):
370     """宽周期真实谱诊断; 返回 d*q、两谱周期和逐谱搜索审计信息。"""
371     periods = []
372     diagnostics = []
373     initial_min, initial_max = 1.5, 400.0
374     grid_points = 8001
375     for table_index, table in enumerate(rows, start=1):
376         values = [(x, y) for x, y in zip(coordinates, table) if 2800.0 <= x <= 3600.0]
377         n = len(values)
378         if n < 10:

```

```

381         continue
382     xs = [x for x, _ in values]
383     ys = [y for _, y in values]
384     center = (xs[0] + xs[-1]) / 2.0
385     scale = max((xs[-1] - xs[0]) / 2.0, 1.0)
386     normalized = [(x - center) / scale for x in xs]
387     basis = [[1.0, x, x * x] for x in normalized]
388     normal = [[sum(row[i] * row[j] for row in basis) for j in range(3)] for i in range(3)]
389     y_residual = residualize(ys, basis, normal)
390     period_min, period_max = initial_min, initial_max
391     best = None
392     boundary_hit = True
393     for _ in range(5):
394         step = (period_max - period_min) / (grid_points - 1)
395         best = None
396         for j in range(grid_points):
397             if j % 256 == 0:
398                 check_budget()
399                 period = period_min + step * j
400                 omega = 2.0 * math.pi / period
401                 cos_column = [math.cos(omega * x) for x in xs]
402                 sin_column = [math.sin(omega * x) for x in xs]
403                 cos_residual = residualize(cos_column, basis, normal)
404                 sin_residual = residualize(sin_column, basis, normal)
405                 g00 = sum(value * value for value in cos_residual)
406                 g01 = sum(a * b for a, b in zip(cos_residual, sin_residual))
407                 g11 = sum(value * value for value in sin_residual)
408                 b0 = sum(a * b for a, b in zip(y_residual, cos_residual))
409                 b1 = sum(a * b for a, b in zip(y_residual, sin_residual))
410                 determinant = g00 * g11 - g01 * g01
411                 if determinant <= 1e-20:
412                     continue
413                 coefficient_cos = (b0 * g11 - b1 * g01) / determinant
414                 coefficient_sin = (b1 * g00 - b0 * g01) / determinant
415                 score = b0 * coefficient_cos + b1 * coefficient_sin
416                 if best is None or score > best[0]:
417                     best = (score, period)
418             if best is None:
419                 raise ValueError(f"附件{table_index}周期投影无有效候选")
420             boundary_hit = best[1] <= period_min + step or best[1] >= period_max - step
421             if not boundary_hit:
422                 break
423             if best[1] >= period_max - step:
424                 period_max *= 2.0
425             else:
426                 period_min = max(period_min / 2.0, 0.1)
427         if boundary_hit:
428             raise ValueError(f"附件{table_index}周期候选仍命中搜索边界, 禁止发布表现dq")
429         periods.append(best[1])
430         diagnostics.append({"附件序号": table_index, "窗口_每厘米": [2800.0, 3600.0],
431                             "周期_每厘米": best[1], "搜索范围_每厘米": [period_min, period_max],
432                             "周期网格点数": grid_points, "边界命中": False,
433                             "基线和谐波口径": "二次基线; 响应和正余弦列均先对同一基线做最小二乘残差化"})
434     period = median(periods)
435     return (5000.0 / period if period else None), periods, diagnostics
436 def synthetic_validation(coordinates, design):
437     specs = build_specs(design)
438     records = []
439     group_errors = {"共同匹配": [], "新增匹配": [], "模型失配": []}
440     baseline_errors = []
441     for position, spec in enumerate(specs):
442         check_budget()
443         observations = generate_case(coordinates, spec)
444         fit = params_from(spec, fit=True)
445         result = estimate_thickness(coordinates, observations, fit)
446         error = abs(result["厚度_微米"] - spec["真厚度_微米"]) / spec["真厚度_微米"] * 100.0
447         group_errors[spec["类别"]].append(error)
448         row = {"编号": spec["编号"], "类别": spec["类别"], "案例索引": spec["案例索引"], "真厚度_微米": spec["真厚度_微米"],
449               "厚度_微米": result["厚度_微米"], "绝对相对误差_百分比": error, "残差平方均值_比例平方": result["联合残差平方均值_比例平方"],
450               "近优候选": result["近优候选"], "噪声种子_按角度": noise_seeds(spec)}
451         records.append(row)
452         if spec["类别"] == "共同匹配":
453             baseline, detail = peak_spacing_baseline(coordinates, observations, fit)
454             if finite(baseline):
455                 baseline_errors.append(abs(baseline - spec["真厚度_微米"]) / spec["真厚度_微米"] * 100.0)
456                 row["常折射率峰距基线_厚度_微米"] = baseline
457         if position % 4 == 3:
458             atomic_csv(RESULTS / "合成逐例.csv", records, ["编号", "类别", "案例索引", "真厚度_微米", "厚度_微米", "绝对相对误差_百分比", "残差平方均值_比例平方", "近优候选", "噪声种子_按角度", "常折射率峰距基线_厚度_微米"])
459     save("合成验证.json", {"问题": 1, "运行状态": "逐例计算中", "计划案例数": len(specs), "已完成案例

```

```

数": len(records), "主指标": {"数值": sum(group_errors["共同匹配"]) / max(1, len(group_errors["共同匹配"])), "单位": "%", "含义": "全部24例共同匹配情景等权平均厚度绝对相对误差, 不剔除案例"}, "分组汇总": {k: {"案例数": len(v), "平均绝对相对误差_百分比": sum(v) / max(1, len(v))} for k, v in group_errors.items()}}, "常折射率峰距基线": {"平均绝对相对误差_百分比": sum(baseline_errors) / max(1, len(baseline_errors))}})
460 atomic_csv(RESULTS / "合成逐例.csv", records, ["编号", "类别", "案例索引", "真厚度_微米", "厚度_微米", "绝对相对误差_百分比", "残差平方均值_比例平方", "近优候选", "噪声种子_按角度", "常折射率峰距基线_厚度_微米"])
461 result = {"问题": 1, "运行状态": "完成", "计划案例数": len(specs), "已完成案例数": len(records),
462 "主指标": {"数值": sum(group_errors["共同匹配"]) / len(group_errors["共同匹配"]), "单位": "%", "含义": "全部24例共同匹配情景等权平均厚度绝对相对误差, 不剔除案例"},
463 "分组汇总": {k: {"案例数": len(v), "平均绝对相对误差_百分比": sum(v) / len(v)} for k, v in group_errors.items()}},
464 "常折射率峰距基线": {"平均绝对相对误差_百分比": sum(baseline_errors) / len(baseline_errors), "失败数": 24 - len(baseline_errors)},
465 "经验覆盖率": {"对象": "合成厚度点估计", "名义覆盖率": None, "测试经验覆盖率": None, "说明": "本问无真实厚度真值; 条件压力范围不是概率区间。"},
466 "冻结输入": "数据/问题1_冻结合成输入/合成设计.json"}
467 save("合成验证.json", result)
468 return specs, records
469 def angle_split_validation(coordinates, specs):
470     rows = []
471     errors, test_rmse, train_rmse = [], [], []
472     for spec in specs[:24]:
473         check_budget()
474         observations = generate_case(coordinates, spec)
475         fit = params_from(spec, fit=True)
476         for train_index, test_index in ((0, 1), (1, 0)):
477             train_angle = ANGLES[train_index], ANGLES[test_index]
478             estimated = estimate_thickness(coordinates, [observations[train_index]], fit, (train_angle,))["厚度_微米"]
479             test_optics = make_optics(coordinates, fit, (test_angle,))[0]
480             train_optics = make_optics(coordinates, fit, (train_angle,))[0]
481             test_prediction = [expanded_reflectance(c, estimated) for c in test_optics]
482             train_prediction = [expanded_reflectance(c, estimated) for c in train_optics]
483             err = abs(estimated - spec["真厚度_微米"]) / spec["真厚度_微米"] * 100.0
484             test_rmse.append(rmse(test_prediction, observations[test_index]))
485             train_rmse.append(rmse(train_prediction, observations[train_index]))
486             errors.append(err)
487             rows.append({"编号": spec["编号"], "案例索引": spec["案例索引"], "训练角度_度": train_angle, "留出角度_度": test_angle,
488 "真厚度_微米": spec["真厚度_微米"], "训练得到厚度_微米": estimated, "留出厚度预测绝对相对误差_百分比": err,
489 "训练反射率RMSE_比例": train_rmse[-1], "留出反射率RMSE_比例": test_rmse[-1], "噪声种子_按角度": noise_seeds(spec)})
490         if len(rows) % 8 == 0:
491             save("独立角度切分验证.json", {"问题": 1, "方法": "双向整角度留出", "运行状态": "计算中", "已完成方向数": len(rows), "计划方向数": 48, "逐例结果": rows})
492     result = {"问题": 1, "方法": "双向独立角度留出", "运行状态": "完成", "已完成方向数": len(rows), "计划方向数": 48,
493 "留出指标": {"留出厚度预测平均绝对相对误差_百分比": sum(errors) / len(errors), "留出反射率RMSE_比例": sum(test_rmse) / len(test_rmse), "训练反射率RMSE_比例": sum(train_rmse) / len(train_rmse)},
494 "切分原则": "10度训练留出15度、15度训练留出10度; 不随机拆同一角度波数行", "逐例结果": rows,
495 "经验覆盖率": None}
496 save("独立角度切分验证.json", result)
497 def sensitivity(coordinates, design):
498     anchor = {"类别": "锚例", "案例索引": 50, "真厚度_微米": design["灵敏度锚例"]["厚度_微米"], **design["灵敏度锚例"], "衬底折射率增量": 0.8, "消光系数": 0.0, "生成返回束数": 1, "拟合曲率": 0.0, "拟合权重": 0.4}
499     observations = generate_case(coordinates, anchor, design["灵敏度锚例"]["随机种子"])
500     reference = estimate_thickness(coordinates, observations, params_from(anchor, fit=True))["厚度_微米"]
501     outputs = [{"扰动": "参照", "厚度_微米": reference}]
502     for factor in (0.8, 1.2):
503         outputs.append({"扰动": f"折射率整条曲线乘{factor}", "厚度_微米": estimate_thickness(coordinates, observations, params_from(anchor, fit=True, scale=factor))["厚度_微米"], "缩放": factor})
504     for factor in (0.8, 1.2):
505         altered = params_from(anchor, fit=True)
506         altered["色散斜率"] *= factor
507         outputs.append({"扰动": f"色散斜率乘{factor}", "厚度_微米": estimate_thickness(coordinates, observations, altered)["厚度_微米"]})
508     for weight in (0.4, 0.6):
509         altered = params_from(anchor, fit=True)
510         altered["垂直偏振权重"] = weight
511         outputs.append({"扰动": f"偏振权重{weight}", "厚度_微米": estimate_thickness(coordinates, observations, altered)["厚度_微米"]})
512     for width in (1, 3, 5):
513         smoothed = [smooth(v, width) for v in observations]
514         outputs.append({"扰动": f"观测平滑{width}点", "厚度_微米": estimate_thickness(coordinates, smoothed, params_from(anchor, fit=True))["厚度_微米"]})
515     altered_angles = (9.5, 14.5)
516     outputs.append({"扰动": "两角度各减0.5度", "厚度_微米": estimate_thickness(coordinates, observations, params_from(anchor, fit=True, altered_angles)["厚度_微米"])})
517     values = [x["厚度_微米"] for x in outputs]
518     result = {"问题": 1, "方法": "固定合成锚例的全量重估", "锚例": {"参照估计厚度_微米": reference, "真厚度仅用

```

```

于生成": True),
519     "重估结果": outputs, "条件压力范围_微米": [min(values), max(values)], "最大绝对漂移_微米": max
(abs(x - reference) for x in values),
520     "范围含义": "光学/偏振/平滑/角度扰动的条件敏感性范围, 不是统计置信区间", "经验覆盖率": None}
521 save("灵敏度.json", result)
522 def main():
523     global T0
524     T0 = time.monotonic()
525     status = {"问题": 1, "运行状态": "进行中", "开始时间": time.strftime("%Y-%m-%dT%H:%M:%S%z"), "已完成块"
: []}
526     atomic_json(RESULTS / "执行状态.json", status)
527     try:
528         coordinates, spectra = read_real_inputs()
529         model_validation(coordinates)
530         design = json.loads(DESIGN.read_text(encoding="utf-8"))
531         specs, records = synthetic_validation([coordinates[i * 7468 // 255] for i in range(256)],
design)
532         status["已完成块"].extend(["实测输入核验", "模型验证", "合成验证"])
533         atomic_json(RESULTS / "执行状态.json", status)
534         angle_split_validation([coordinates[i * 7468 // 255] for i in range(256)], specs)
535         sensitivity([coordinates[i * 7468 // 255] for i in range(256)], design)
536         dq, periods, diagnostics = apparent_dq(coordinates, [[r["反射率_百分比"] for r in s["观测"]] for
s in spectra])
537         save("模型验证.json", {**json.loads((RESULTS / "模型验证.json").read_text(encoding="utf-8")), "真
实附件诊断": {"表观光学厚度乘积_dq_微米": dq, "周期_每厘米": periods, "窗口_每厘米": [2800.0, 3600.0], "搜
索范围_每厘米": [x["搜索范围_每厘米"] for x in diagnostics], "边界命中": any(x["边界命中"] for x in
diagnostics), "周期投影诊断": diagnostics}})
538         status.update({"运行状态": "正常完成", "已完成块": ["实测输入核验", "模型验证", "合成验证", "独立角度切
分验证", "灵敏度", "真实附件表观dq诊断"]})
539     except BudgetStop as exc:
540         status.update({"运行状态": "软截止后保留部分结果", "停止原因": str(exc)})
541     except Exception as exc:
542         status.update({"运行状态": "异常停止", "停止原因": repr(exc)})
543         raise
544     finally:
545         status["实际用时秒"] = elapsed()
546         atomic_json(RESULTS / "执行状态.json", status)
547         summary = {"运行状态": status["运行状态"], "实际用时秒": elapsed(), "结果目录": str(RESULTS)}
548         if (RESULTS / "合成验证.json").exists():
549             summary["合成主指标"] = json.loads((RESULTS / "合成验证.json").read_text(encoding="utf-8")).get("
主指标")
550         print(json.dumps(summary, ensure_ascii=False))
551 if __name__ == "__main__":
552     main()

```

清单 3: 合成情景的完整参数

```

1 {
2     "版本": "问题1冻结合成输入-v1",
3     "用途": "问题1双束界面场反演的可复算合成验证; 不作为真实晶圆厚度答案",
4     "坐标来源": {"文件": "数据/附件1.xlsx与数据/附件2.xlsx", "内容约束": "四附件共有7469个严格递增波数点, 按floor(k
*7468/255)取256点", "使用点数_每角": 256},
5     "角度_度": [10.0, 15.0],
6     "基础情景": {"厚度_微米": [3.0, 8.0, 20.0], "折射率基值": [2.4, 3.4], "色散斜率": [0.0, 0.08], "噪声标准差_
比例": [0.0, 0.001], "噪声相关系数": 0.6, "垂直偏振权重": 0.5, "衬底折射率增量": 0.8, "消光系数": 0.0, "返
回路数": 1, "随机种子": 20260909},
7     "新增匹配情景": [
8         {"厚度_微米": 5.0, "折射率基值": 2.8, "色散斜率": -0.08, "色散曲率": -0.06, "噪声标准差_比例": 0.0, "垂直偏
振权重": 0.3},
9         {"厚度_微米": 5.0, "折射率基值": 2.8, "色散斜率": 0.08, "色散曲率": 0.06, "噪声标准差_比例": 0.0015, "垂直
偏振权重": 0.7},
10        {"厚度_微米": 5.0, "折射率基值": 3.1, "色散斜率": -0.08, "色散曲率": 0.06, "噪声标准差_比例": 0.0015, "垂
直偏振权重": 0.3},
11        {"厚度_微米": 5.0, "折射率基值": 3.1, "色散斜率": 0.08, "色散曲率": -0.06, "噪声标准差_比例": 0.0, "垂直偏
振权重": 0.7},
12        {"厚度_微米": 12.0, "折射率基值": 2.8, "色散斜率": -0.08, "色散曲率": 0.06, "噪声标准差_比例": 0.0015, "垂
直偏振权重": 0.7},
13        {"厚度_微米": 12.0, "折射率基值": 2.8, "色散斜率": 0.08, "色散曲率": -0.06, "噪声标准差_比例": 0.0, "垂直偏
振权重": 0.3},
14        {"厚度_微米": 12.0, "折射率基值": 3.1, "色散斜率": -0.08, "色散曲率": -0.06, "噪声标准差_比例": 0.0, "垂直
偏振权重": 0.7},
15        {"厚度_微米": 12.0, "折射率基值": 3.1, "色散斜率": 0.08, "色散曲率": 0.06, "噪声标准差_比例": 0.0015, "垂
直偏振权重": 0.3},
16        {"厚度_微米": 5.0, "折射率基值": 2.8, "色散斜率": 0.0, "色散曲率": 0.0, "噪声标准差_比例": 0.0015, "垂直偏
振权重": 0.5},
17        {"厚度_微米": 5.0, "折射率基值": 3.1, "色散斜率": 0.0, "色散曲率": 0.0, "噪声标准差_比例": 0.0, "垂直偏振权
重": 0.5},
18        {"厚度_微米": 12.0, "折射率基值": 2.8, "色散斜率": 0.0, "色散曲率": 0.0, "噪声标准差_比例": 0.0, "垂直偏振
权重": 0.5},
19        {"厚度_微米": 12.0, "折射率基值": 3.1, "色散斜率": 0.0, "色散曲率": 0.0, "噪声标准差_比例": 0.0015, "垂直偏
振权重": 0.5},
20        {"厚度_微米": 8.0, "折射率基值": 2.8, "色散斜率": -0.08, "色散曲率": 0.06, "噪声标准差_比例": 0.0015, "垂

```

```

    直偏振权重": 0.3},
21 {"厚度_微米": 8.0, "折射率基值": 3.1, "色散斜率": 0.08, "色散曲率": -0.06, "噪声标准差_比例": 0.0015, "垂
    直偏振权重": 0.7},
22 {"厚度_微米": 20.0, "折射率基值": 2.8, "色散斜率": -0.08, "色散曲率": -0.06, "噪声标准差_比例": 0.0, "垂直
    偏振权重": 0.3},
23 {"厚度_微米": 20.0, "折射率基值": 3.1, "色散斜率": 0.08, "色散曲率": 0.06, "噪声标准差_比例": 0.0015, "垂
    直偏振权重": 0.7}
24 ],
25 "失配情景": [
26 {"厚度_微米": 5.0, "折射率基值": 2.8, "色散斜率": 0.08, "生成曲率": 0.12, "噪声标准差_比例": 0.001, "说明
    ": "遗漏色散曲率"},
27 {"厚度_微米": 5.0, "折射率基值": 3.1, "色散斜率": -0.08, "生成权重": 0.2, "噪声标准差_比例": 0.001, "说明
    ": "偏振权重误设"},
28 {"厚度_微米": 8.0, "折射率基值": 2.8, "色散斜率": 0.08, "生成增益": 0.05, "噪声标准差_比例": 0.001, "说明
    ": "遗漏线性包络"},
29 {"厚度_微米": 8.0, "折射率基值": 3.1, "色散斜率": -0.08, "生成基线": 0.01, "噪声标准差_比例": 0.001, "说明
    ": "遗漏二次基线"},
30 {"厚度_微米": 12.0, "折射率基值": 2.8, "色散斜率": 0.08, "生成消光系数": 0.002, "噪声标准差_比例": 0.001,
    "说明": "遗漏消光系数"},
31 {"厚度_微米": 12.0, "折射率基值": 3.1, "色散斜率": -0.08, "生成返回束数": 8, "噪声标准差_比例": 0.001, "说
    明": "遗漏高阶往返"},
32 {"厚度_微米": 20.0, "折射率基值": 2.8, "色散斜率": 0.08, "生成曲率": -0.12, "生成权重": 0.2, "噪声标准差_比
    例": 0.001, "说明": "曲率与偏振共同失配"},
33 {"厚度_微米": 20.0, "折射率基值": 3.1, "色散斜率": -0.08, "生成增益": -0.05, "生成基线": 0.01, "噪声标准差
    _比例": 0.001, "说明": "包络与基线共同失配"},
34 {"厚度_微米": 8.0, "折射率基值": 2.8, "色散斜率": 0.0, "生成消光系数": 0.002, "生成返回束数": 8, "噪声标准差
    _比例": 0.001, "说明": "吸收与高阶共同失配"},
35 {"厚度_微米": 8.0, "折射率基值": 3.1, "色散斜率": 0.0, "生成权重": 0.2, "生成曲率": 0.12, "噪声标准差_比例
    ": 0.001, "说明": "偏振与曲率共同失配"}
36 ],
37 "灵敏度锚例": {"厚度_微米": 8.0, "折射率基值": 2.8, "色散斜率": 0.08, "色散曲率": 0.0, "噪声标准差_比例":
    0.001, "垂直偏振权重": 0.4, "随机种子": 20261909},
38 "生成规则": {"归一化波数": "x=(sigma-2200)/1800", "层折射率": "n=n0+slope*x+curvature*x*x+i*kappa", "衬底
    折射率": "n_layer+0.8", "案例索引": "按基础24例、新增16例、失配10例连续编号", "角度种子": "seed
    =20260909+2*案例索引+角度索引", "噪声": "AR(1), e0=sqrt(1-rho^2)u0, et=rho*e(t-1)+sqrt(1-rho^2)ut"}
39 }

```

基础核验可先于全部情景计算执行。清单4从原始附件抽取核验坐标，独立比较场平方与展开式，再检查零界面返回项、长度回换以及有吸收时的传播衰减；执行位置为附件、求解与交接目录的共同上级。

清单 4: 基础核验执行命令

```

python3 - <<'PY'
import importlib.util
import math
import time
from pathlib import Path
source = Path("求解/问题1/求解_问题1.py")
module_spec = importlib.util.spec_from_file_location("q1_model", source)
model = importlib.util.module_from_spec(module_spec)
module_spec.loader.exec_module(model)
model.T0 = time.monotonic()
parameters = {
    "折射率基值": 2.8,
    "色散斜率": 0.08,
    "色散曲率": 0.0,
    "垂直偏振权重": 0.5,
    "消光系数": 0.0,
    "衬底折射率增量": 0.8,
}
rows = model.read_xlsx(Path("数据/附件1.xlsx"))
coordinates = [row["波数_每厘米"] for row in rows]
differences = []
for angle in model.ANGLES:
    for sigma in coordinates[:32]:
        for thickness in (3.0, 8.0, 20.0):
            model.check_budget()
            direct = model.direct_reflectance(
                sigma, angle, parameters, thickness, returns=1
            )
            expanded = model.expanded_reflectance(
                model.expanded_coefficients(sigma, angle, parameters),
                thickness,
            )
            differences.append(abs(direct - expanded))
maximum_difference = max(differences)
assert maximum_difference < 1e-12
thickness_um = 8.0
thickness_cm = thickness_um * 1e-4
assert math.isclose(thickness_cm, 0.0008, abs_tol=1e-15)
assert math.isclose(thickness_cm * 1e4, thickness_um, abs_tol=1e-12)
zero_interface = dict(parameters, 衬底折射率增量=0.0)

```



```
with_echo = model.direct_reflectance(
    2199.9, 10.0, zero_interface, thickness_um, returns=1
)
without_echo = model.direct_reflectance(
    2199.9, 10.0, zero_interface, thickness_um, returns=0
)
assert abs(with_echo - without_echo) < 1e-12
absorbing = dict(parameters, 消光系数=0.002)
for angle in model.ANGLES:
    for sigma in coordinates[:,32]:
        model.check_budget()
        for surface, echo, repeat, longitudinal in model.interface_components(
            sigma, angle, absorbing
        ):
            assert longitudinal.imag >= 0.0
            assert abs(model.propagation(sigma, longitudinal, thickness_um)) <= 1.0
print(f"场展开最大绝对差（比例）：{maximum_difference:.3e}")
print(f"厚度换算：{thickness_um:g} 微米 = {thickness_cm:g} 厘米")
print("零界面返回、单位回换和被动传播核验完成")
PY
```

清单 5: 全部情景与真实周期的执行命令

```
test -r "数据/附件1.xlsx" &&
test -r "数据/附件2.xlsx" &&
test -r "交接/数据档案.json" &&
test -r "数据/问题1_冻结合成输入/合成设计.json" &&
python3 "求解/问题1/求解_问题1.py"
```

清单5在同一目录执行全部情景与真实周期计算，并保留逐例估计、分组误差、角度留出和光学参数扰动的结果。至此，一次返回的物理边界、已知厚度回收和真实周期等效量能够分别核对；问题二据此转向同片双角信息的共同约束，而不把合成误差带入真实晶圆的测厚精度。

8.2 问题二源码：变投影与重采样

同片双角的复核从共同波数坐标开始：先确定哪些连续区段参与拟合、校准和测试，再核对厚度及预测误差。这里的重采样指保留原坐标的选点与连续留段重估，训练、校准、测试各自承担不同任务。

共同窗口取 1200–3800 cm⁻¹，两角同步保留 480 个原始坐标点。图6标出的边界与表8的选段口径共同确定了比较对象；选点改变计算规模，波数值和反射率原值保持不变。

连续区段而非相邻行的随机混合构成测试单位。每块含 40 个坐标点，训练点与训练、非训练区段交界的距离至少为 20 cm⁻¹；表23列出两角同步使用的块号，保护带内的训练点被排除。

表 23 两角同步留段的完整块号

折号	训练块	校准块	测试块
一	1, 2, 4, 5, 7, 8, 10, 11	3, 9	6, 12
二	2, 3, 5, 6, 8, 9, 11, 12	4, 10	1, 7

边界确定后，坐标中心、半跨度以及反射率的标准化尺度均由训练点确定，校准点只决定预测区间半宽，测试点只评价预测。早期曾比较用原谱相关程度与连续留段表现判断可靠性：原谱相关系数达到 0.9987，但两角来自同一晶圆，因此改为同波数共同分组，以留出光谱的表现检验模型。这一取舍把共享厚度的物理要求与独立检验的统计要求分开，也确定了后续参数估计的数据边界。

双角色散消干扰让两种角度共同决定厚度和低维折射变化，让每个角度自己解释慢变基线、振幅和固定相位。变投影就是给定厚度与折射特性后，先从观测中直接求出基线、振幅，再搜索较少的物理参数；本题两角条纹位置接近而反射率水平不同，恰好需要把这两类参数分开。

厚度记为 d ，单位为微米；参考折射率 n_0 和色散系数 c 均为无量纲量。各角常相位记为 ψ_θ ，以弧度计， θ 表示外部入射角。表24固定了这些量的取值范围和搜索尺度：全量主搜索完成 864 组粗网格组合，普通扰动比较保持相同网格，避免把计算精细程度的变化混入物理假设比较。

表 24 共享参数与逐角投影的搜索设置

对象	取值或设置	生效位置
厚度 d	[0.5, 40] μm	两角共享
参考折射率 n_0	[1.2, 6]	两角共享
色散系数 c	[-1, 1]	两角共享
色散参考波数	2000 cm^{-1}	折射率参数化
传播约束	$n(\sigma) > \sin 15^\circ$	窗口两端检查
基线与幅值阶数	二次、一次	逐角线性投影
常相位 ψ_θ	$[0, \pi)$ ，初查 12 点	逐角重估

对象	取值或设置	生效位置
相位局部细化	初步长 $\pi/12$, 减半 8 次	逐角相位搜索
训练尺度	二次趋势残差四分位距	下限 10^{-4}
初始折射率	2, 3, 4	与色散起点组合
初始色散系数	-0.5, 0, 0.5	与厚度网格组合
主搜索厚度网格	96 点	全量与主留段
网格加密对照	192 点	单独重估厚度
主搜索局部起点	上限 12 个	有界局部细化
局部细化停止设置	70 次迭代, 容差 10^{-9}	线搜索上限 20
五折搜索	48 点, 至多 4 个起点	连续区段重估
主搜索时限	每情景 42 s	普通扰动相同
五折搜索时限	每折 30 s	独立搜索预算
总计算时限	1080 s	逐阶段保存结果
校准经验分位点	0.75	反射率预测半宽
近等损失阈值	最小训练损失的 1.10 倍	厚度与折射率剖面
跨切分判据	极差与中位数之比 0.20	厚度稳定性

有限阶基线与幅值吸收角度差异, 常相位则在整段内保持不变; 逐点自由相位会把本应由厚度解释的条纹一起吸收, 因此未采用这种放宽。经验色散描述的是选定波段内的折射变化, 而不是预先已知的材料常数。完整实现保留各角的线性系数、相位、矩阵秩和条件数计算, 使共享参数之外的拟合自由度也可逐项复核。

全量非触边代表厚度为 $10.80\ \mu\text{m}$, 跨连续切分、光学剖面 and 合法扰动的条件范围为 $0.64\text{--}18.11\ \mu\text{m}$ 。图7把点与范围放在同一厚度轴上, 表9给出正文答案; 表26进一步保留全量及主留段的共享参数, 同时在表25保留更低损失的触边候选。

固定点集、模型及参数边界的补查保留全部局部终点与停止原因, 清单6可复算表25中的搜索差异。

表 25 固定模型下的网格与起点补查

搜索设置	厚度/ μm	参考折射率	损失	触边状态
原 96 点网格	10.8035	1.4930	0.2747	无
加密 192 点	13.5337	1.2000	0.2744	折射率下界
加密 384 点	9.2842	1.7174	0.2750	无

补查固定每角 480 个原始坐标点, 并在原代表解和边界解附近改变厚度起点; 局部优化以相对目标变化 10^{-9} 、投影梯度容差 10^{-5} 或最多 70 次迭代结束。所有已得候选共同比较, 不能因更密网格单次所得更差便丢弃此前较优点; 局部停止也不证明厚度全域唯一。

局部补查的 28 个终点中, 17 个满足数值停止准则, 另 11 个达到迭代上限, 后者也完整保留。达到上限的终点与满足停止准则的终点分开登记, 补查并未证明各分支均已收敛。边界候选在已得候选中损失最低。原非触边点按前述规则承担比较坐标; 训练损失衡量曲线拟合程度, 真实厚度的准确性仍需实物标定检验。

清单 6: 固定模型的网格与起点补查

```

import importlib.util
import json
import time
from pathlib import Path

5 import numpy as np

ROOT = Path(__file__).resolve().parents[2]
START = time.monotonic()
10 BUDGET = 240.0
OUTPUT = ROOT / '求解/问题2/结果/搜索补查.json'
spec = importlib.util.spec_from_file_location('original_solver', ROOT / '求解/问题2/求解_问题2.py')
solver = importlib.util.module_from_spec(spec)
spec.loader.exec_module(solver)
15 sigma, values, source_rows, datasets, audit = solver.read_inputs()
fold = solver.full_fold(sigma, values)
original = json.loads((ROOT / '求解/问题2/结果/厚度结果.json').read_text())
validation = json.loads((ROOT / '求解/问题2/结果/验证.json').read_text())
result = {'固定条件': {'每角点数': len(sigma), '窗口_每厘米': [1200, 3800],
20         '参数边界': solver.BOUNDS, '基线阶数': 2, '幅值阶数': 1},
         '停止条件': {'总预算_秒': BUDGET, '单次搜索预算_秒': 60, '单起点最大迭代': 70,
                     '函数相对容差': 1e-9, '默认梯度容差': 1e-5, '最大线搜索步数': 20},
         '候选': [], '加密搜索': [], '局部终点': []}
```

```

25 def summarize(params, loss, origin):
    margins = [(value - lower) / (upper - lower) for value, (lower, upper) in zip(
        params, solver.BOUNDS)]
    boundary = [name for name, margin in zip(('厚度', '参考折射率', '色散系数'), margins
    )
                if min(margin, 1 - margin) <= 1e-4]
    return {'来源': origin, '厚度_微米': float(params[0]), '参考折射率': float(params[1]),
30         '色散系数': float(params[2]), '损失': float(loss), '触边参数': boundary,
            '最小相对边界距离': float(min(min(value, 1 - value) for value in margins))}

def save():
    result['实际用时_秒'] = time.monotonic() - START
35     solver.write_json(OUTPUT, result)

for index in (0, 8):
    entry = validation['窗口与模型灵敏度'][index]
    params = (entry['厚度_um'], entry['参考折射率'], entry['色散系数'])
40     candidate = solver.fit_candidate(params, fold)
    result['候选'].append(summarize(params, candidate['损失'], entry['情景名称']))
    result['候选'][-1]['原记录损失'] = entry['训练标准化损失']
save()
native_minimize = solver.minimize

45 def record_minimize(objective, initial, **kwargs):
    outcome = native_minimize(objective, initial, **kwargs)
    entry = summarize(outcome.x, outcome.fun, '连续优化终点')
    entry.update({'初值': list(initial), '迭代数': int(outcome.nit),
50         '成功终止': bool(outcome.success), '停止说明': str(outcome.message)})
    result['局部终点'].append(entry)
    return outcome

solver.minimize = record_minimize
55 for grid in (192, 384):
    if time.monotonic() - START > BUDGET - 70:
        break
    candidate, info = solver.search_fold(fold, START, grid_size=grid, max_starts=16,
        budget_seconds=60)
    entry = summarize(candidate['参数'], candidate['损失'], f'厚度网格{grid}点')
60     result['候选'].append(entry)
    result['加密搜索'].append({'网格点数': grid, '详情': info, '最优终点': entry})
    save()

for entry in result['候选'][:2]:
65     for shift in (-0.01, 0.01):
        if time.monotonic() - START > BUDGET - 20:
            break
        initial = np.array([entry['厚度_微米'] * (1 + shift), entry['参考折射率'],
            entry['色散系数']])
        def objective(params):
70             if time.monotonic() - START >= BUDGET - 5:
                return 1e10
            candidate = solver.fit_candidate(params, fold)
            return 1e10 if candidate is None else candidate['损失']
        record_minimize(objective, initial, method='L-BFGS-B', bounds=solver.BOUNDS,
75             options={'maxiter': 70, 'ftol': 1e-9, 'maxls': 20})
        save()

best = min(result['候选'] + result['局部终点'], key=lambda entry: entry['损失'])
result['补查最低损失候选'] = best
80 result['局部停止汇总'] = {'终点数': len(result['局部终点']),
                            '满足数值停止准则': sum(entry['成功终止'] for entry in result['局
                                部终点']),
                            '达到迭代上限': sum('ITERATIONS REACHED LIMIT' in entry['停止说明
                                '] for entry in result['局部终点'])}
result['原网格到加密厚度变化_百分比'] = 100 * (result['候选'][1]['厚度_微米'] / result['候
    选'][0]['厚度_微米'] - 1)
result['原代表相对最低损失差_百分比'] = 100 * (result['候选'][0]['损失'] / best['损失'] -
    1)
85 result['代表点规则'] = '沿用原96点网格搜索的最低损失可行解作为跨问题比较坐标；要求各参数相对边界距
    离大于0.0001。补查更低损失候选全部保留，不称代表点为全域最优，也不将触边候选删去。'

```

```

result['解释'] = '固定观测、模型和物理边界的移动属于数值搜索不确定性；改变窗口、光学参数或切分属于
另列情景变化。局部算法终止不证明全域唯一性。'
save()
ledger = ROOT / '交接/实验记录.json'
records = json.loads(ledger.read_text())
90 records.append({'问题': '问题2', '类别': '科学尝试', '尝试': '固定每角480点、原模型及边界，补
查192与384点网格并扰动原解和边界解的厚度起点。',
'现象': f"原代表厚度{result['候选'][0]['厚度_微米']}微米，加密候选{result['
候选'][1]['厚度_微米']}微米；补查最低损失{best['损失']}, 厚度{best['厚度_微米']}微米，触边
参数{best['触边参数']}"},
'决定': '保留原非触边点作为比较代表，显式展示边界候选，将数值搜索变化与物理情景变
化分开。',
'依据': '求解/问题2/结果/搜索补查.json'})
solver.write_json(ledger, records)
95 print(json.dumps({'最低损失候选': best, '候选数': len(result['候选']) + len(result['局部
终点']), '用时_秒': result['实际用时_秒']}, ensure_ascii=False))

```

表 26 全量与主留段的共享参数

拟合对象	$d/\mu\text{m}$	n_0	c
全量共同样本	10.8035	1.4930	-0.2089
主留段第一折	13.0790	1.2000	-0.1897
主留段第二折	10.6475	1.6614	-0.1253

参考折射率与厚度随切分共同移动，说明共享同一块晶圆的约束仍容许不同参数组合。全量厚度重新使用全部共同样本估计，留段厚度则用于检查换一段光谱会产生什么变化；二者分工明确，折间平均不参与全量答案的定义。

五折重估的厚度相对极差为 0.7604，表27逐折保留厚度与测试误差。相对极差指最大厚度与最小厚度之差除以中位数，为无量纲量；它直接刻画连续区段改变时厚度分支的移动程度。

表 27 连续五折重估的厚度与谱形误差

折号	测试块	训练块	厚度/ μm	标准化误差
一	1, 2	3, 4, 5, 6, 7, 8, 9, 10	13.3683	1.9693
二	3, 4	1, 2, 5, 6, 7, 8, 9, 10	10.4926	0.6847
三	5, 6	1, 2, 3, 4, 7, 8, 9, 10	6.5335	0.3407
四	7, 8	1, 2, 3, 4, 5, 6, 9, 10	13.5109	0.4882
五	9, 10	1, 2, 3, 4, 5, 6, 7, 8	5.5322	1.2048

这些切分均以末端块作为校准区，其余训练块还须排除边界保护带。谱形误差较小的切分并没有把厚度聚拢到同一分支，因此厚度对象同时保留全量点、逐折点和条件范围；反射率预测的基线与覆盖检验再回答曲线解释能力的问题。

正式主方法的连续留段标准化均方根误差为 1.2074，二次趋势基线为 1.4404，两者均为无量纲量。图8展示局部残差，表28把各测试块的主方法与两种基线并排比较；标准化误差是该块反射率均方根误差除以对应角度的训练尺度。

表 28 各测试块的主方法与基线误差

折号	角度	块号	主方法	二次趋势	训练均值
一	10°	6	0.2624	0.7603	1.3219
一	10°	12	1.0909	1.1170	2.2385
一	15°	6	0.5014	0.5942	0.9831
一	15°	12	1.1833	1.1641	2.5130
二	10°	1	3.1195	3.1543	7.8071
二	10°	7	0.0810	0.6649	1.1553
二	15°	1	3.3155	3.4041	8.1029
二	15°	7	0.1055	0.6641	1.1406

局部比较揭示平均改善的来源：高波数平缓区的条纹结构能补充二次趋势，而部分末端区段仍出现反向差异。正式平均改善为 16.17%，描述的是同样测试块上的谱形预测变化；真实厚度的准确程度需要另有厚度参照。

预测区间检验进一步把拟合曲线与区间宽度分开。每折每角先按校准绝对残差的 75% 经验分位确定对称半宽，再保持半宽预测测试块；正式测试覆盖率为 49.06%。图9中的覆盖差异可由表29的逐块计数复核，误差与半宽均换算为反射率百分点。

表 29 反射率测试误差与区间覆盖计数

折号	角度	块号	均方根误差	区间半宽	覆盖数/点数
一	10°	6	0.1776	0.4869	40/40
一	10°	12	0.7384	0.4869	0/40
一	15°	6	0.3600	0.4941	32/40
一	15°	12	0.8497	0.4941	0/40
二	10°	1	1.7278	0.2346	4/40
二	10°	7	0.0449	0.2346	40/40
二	15°	1	1.7860	0.2692	1/40
二	15°	7	0.0568	0.2692	40/40
合计	—	—	—	—	157/320

同一校准半宽在不同测试区段的覆盖表现分化，反映的是残差随波数位置改变，不能通过把全部相邻点当成独立重复来消除。厚度条件范围来自切分、剖面与物理情景的并集；反射率预测区间来自校准残差，两种范围始终对应不同对象。

留角度对照保留来源相位后，首个测试区段的双向标准化误差分别为 3.4568 和 2.9647。表30与图15保留这种区段差异：共享厚度、参考折射率和色散取自主留段的双角联合拟合，来源角度的常相位固定，目标角度只重新估计有限阶基线与幅值。

表 30 固定来源相位后的双向留角度误差

折号	测试块	10° → 15°	15° → 10°
一	6	0.5013	0.2573
一	12	1.1858	1.0867
二	1	3.4568	2.9647
二	7	0.1678	0.2494

方向比较检查的是联合参数与来源相位向另一角度的迁移，区别于完全舍弃目标角度后重新确定厚度。至此，点估计、跨切分变化、谱形基线与覆盖计数各有对应对象；问题三沿用相同坐标、切分和评分单位比较多次往返的作用。

清单7节录连续留段、逐角投影与分块评分；完整可执行程序随附于 / 2/ _ 2.py。

清单 7: 双角投影与留段评分核心实现

```
def make_fold(sigma, values, spec, angles=ANGLES):
    train_blocks, calibration_blocks, test_blocks = (set(spec[k]) for k in ("训练块",
    "校准块", "测试块"))
    if train_blocks & calibration_blocks or train_blocks & test_blocks or
    calibration_blocks & test_blocks:
4         raise ValueError("折分块重叠")
    if train_blocks | calibration_blocks | test_blocks != set(range(1, 13)):
        raise ValueError("折分块未覆盖1至12")
    edges = np.array([(sigma[k - 1] + sigma[k]) / 2 for k in range(40, 480, 40)])
    guard = [edge for block, edge in enumerate(edges, 1)]
9         if (block in train_blocks) != (block + 1 in train_blocks)]
    train_indices = np.array([i for i in range(len(sigma))
                             if block_of(i) in train_blocks and all(abs(sigma[i] -
    edge) >= 20 for edge in guard)], dtype=int)
    if len(train_indices) < 30:
        raise ValueError("保护带后训练点过少")
14    center = float((sigma[train_indices].min() + sigma[train_indices].max()) / 2)
    half_span = float((sigma[train_indices].max() - sigma[train_indices].min()) / 2)
    x = (sigma[train_indices] - center) / half_span
    angle_data = []
    for ai, angle in enumerate(angles):
19        y = values[ai, train_indices]
        base = np.column_stack((np.ones(len(x)), x, x * x))
        coefficient = np.linalg.lstsq(base, y, rcond=None)[0]
        residual = y - base @ coefficient
        scale = max(percentile(residual, 0.75) - percentile(residual, 0.25), 1e-4)
24        angle_data.append({"角度_度": float(angle), "sigma": sigma[train_indices], "x"
        : x,
                             "y": y, "scale": float(scale), "基线系数": coefficient})
    return {"spec": spec, "sigma": sigma, "values": values, "angles": angle_data,
            "train_indices": train_indices, "center": center, "half_span": half_span}
```



```

    '
        "guard_edges": guard, "折分_训练点数": int(len(train_indices))}
29
def full_fold(sigma, values, angles=ANGLES):
    spec = {"折号": 0, "训练块": list(range(1, 13)), "校准块": [], "测试块": []}
    return make_fold(sigma, values, spec, angles)

34 def valid_parameters(params):
    d, n_ref, dispersion = params
    if not all(lo <= value <= hi for value, (lo, hi) in zip(params, BOUNDS)):
        return False
    n_low = n_ref + dispersion * ((2000 / 1200) ** 2 - 1)
39 n_high = n_ref + dispersion * ((2000 / 3800) ** 2 - 1)
    return min(n_low, n_high) > math.sin(math.radians(15))

def phase_rate(sigma, angle, n_ref, dispersion):
    n = n_ref + dispersion * ((2000.0 / sigma) ** 2 - 1.0)
44 q = np.sqrt(np.maximum(n * n - math.sin(math.radians(angle)) ** 2, 0.0))
    return 4.0 * math.pi / 10000.0 * sigma * q

def fit_angle(params, data, base_degree=2, amp_degree=1, fixed_phase=None,
angle_override=None):
    angle = data["角度_度"] if angle_override is None else float(angle_override)
49 rate = phase_rate(data["sigma"], angle, params[1], params[2]) * params[0]
    phase_candidates = [float(fixed_phase)] if fixed_phase is not None else [j * math.
pi / 12 for j in range(12)]

    def at_phase(phase):
        phase = float(phase) % math.pi
54 columns = [data["x"] ** k for k in range(base_degree + 1)]
        columns.extend([(data["x"] ** k) * np.cos(rate + phase) for k in range(
amp_degree + 1)])
        matrix = np.column_stack(columns)
        coefficient, _, rank, _ = np.linalg.lstsq(matrix, data["y"], rcond=None)
        residual = data["y"] - matrix @ coefficient
59 return {"相位_rad": phase, "系数": coefficient, "残差": residual,
        "损失": float(np.mean(residual * residual) / data["scale"] ** 2),
        "秩": int(rank), "条件数": float(np.linalg.cond(matrix)),
        "rate": rate, "角度_度": angle, "基线阶数": base_degree, "幅值阶数":
amp_degree}

64 best = min((at_phase(phase) for phase in phase_candidates), key=lambda item:
item["损失"])
    if fixed_phase is None:
        step = math.pi / 12
        for _ in range(8):
            candidates = [best, at_phase(best["相位_rad"] - step), at_phase(best["相位
_rad"] + step)]
69 best = min(candidates, key=lambda item: item["损失"])
        step /= 2
    return best

def fit_candidate(params, fold, base_degree=2, amp_degree=1, angles=ANGLES):
74 if not valid_parameters(params):
    return None
    models = [fit_angle(params, data, base_degree, amp_degree)
for data in fold["angles"]]
    return {"参数": tuple(float(x) for x in params), "模型": models,
        "损失": float(np.mean([model["损失"] for model in models]))}
79 def predict_model(model, sigma, center, half_span):
    x = (np.asarray(sigma, dtype=float) - center) / half_span
    rate = model["rate"] if len(model["rate"]) == len(x) else None
    if rate is None:
84 raise ValueError("预测坐标必须与模型rate同步生成")
    matrix = np.column_stack([np.ones(len(x)), x, x * x,
        np.cos(rate + model["相位_rad"]),
        x * np.cos(rate + model["相位_rad"])])
    if len(model["系数"]) != 5:
89 powers = [x ** k for k in range(len(model["系数"]) - 2)]
        # 仅供灵敏度结果使用; 主模型始终为二次基线、一次幅值。
        matrix = np.column_stack(powers[:3] + [x ** k * np.cos(rate + model["相位_rad

```

```

    ])
    for k in range(len(model["系数"]) -
3))
    return matrix @ model["系数"]
94
def predict_params(params, model, sigma, center, half_span, angle):
    rate = phase_rate(np.asarray(sigma, dtype=float), angle, params[1], params[2]) *
    params[0]
    x = (np.asarray(sigma, dtype=float) - center) / half_span
    coefficient = model["系数"]
99
    base_degree = int(model.get("基线阶数", 2))
    amp_degree = int(model.get("幅值阶数", len(coefficient) - base_degree - 1))
    columns = [x ** k for k in range(base_degree + 1)]
    columns.extend([(x ** k) * np.cos(rate + model["相位_rad"]) for k in range(
    amp_degree + 1)])
    return np.column_stack(columns) @ coefficient
104
def score_blocks(fold, candidate):
    blocks = []
    if not fold["spec"]["测试块"]:
        return blocks
109
    cal_indices = np.array([i for i in range(len(fold["sigma"]))
        if block_of(i) in set(fold["spec"]["校准块"])], dtype=int
    )
    for ai, model in enumerate(candidate["模型"]):
        angle = ANGLES[ai]
        if len(cal_indices):
114
            cal_pred = predict_params(candidate["参数"], model, fold["sigma"][
            cal_indices],
                fold["center"], fold["half_span"], angle)
            cal_abs = np.sort(np.abs(fold["values"][ai, cal_indices] - cal_pred))
            rank = min(len(cal_abs) - 1,
                max(0, math.ceil(CALIBRATION_QUANTILE * len(cal_abs)) - 1))
119
            half_width = float(cal_abs[rank])
        else:
            half_width = float("nan")
        for block in fold["spec"]["测试块"]:
            indices = np.array([i for i in range(len(fold["sigma"])) if block_of(i)
== block], dtype=int)
124
            pred = predict_params(candidate["参数"], model, fold["sigma"][indices],
                fold["center"], fold["half_span"], angle)
            observed = fold["values"][ai, indices]
            residual = observed - pred
            coverage = None if not math.isfinite(half_width) else float(np.mean(np.abs
(residual) <= half_width))
129
            blocks.append({"折号": fold["spec"]["折号"], "角度_度": angle, "测试块":
            block,
                "点数": int(len(indices)), "样本序号": (indices + 1).tolist
            (),
                "波数_cm^-1": fold["sigma"][indices].tolist(),
                "观测反射率_比例": observed.tolist(), "预测反射率_比例": pred.
            tolist(),
                "残差_比例": residual.tolist(), "均方根误差_比例": float(np.
            sqrt(np.mean(residual ** 2))),
134
                "训练尺度_比例": float(fold["angles"][ai]["scale"]),
                "标准化均方根误差": float(np.sqrt(np.mean(residual ** 2)) /
            fold["angles"][ai]["scale"]),
                "校准点数": int(len(cal_indices)), "区间半宽_比例":
            half_width,
                "校准经验分位点": CALIBRATION_QUANTILE,
                "经验覆盖率": coverage,
139
                "区间下界_比例": (pred - half_width).tolist() if math.
            isfinite(half_width) else [],
                "区间上界_比例": (pred + half_width).tolist() if math.
            isfinite(half_width) else []})
    return blocks

```

逐角系数的复核采用清单8：固定本次保存的全精度共享参数，恢复各角常相位、基线系数、幅值系数与训练尺度，并逐点核对测试预测。它同时核对保护带、块级评分、覆盖计数、条件范围和五折相对极差，保留原计算结果不变；执行位置为原始附件、求解和交接目录的共同上级。

清单 8: 完整参数恢复与逐块结果核对命令

```

1 python3 - <<'PY'
2 import importlib.util
3 import json
4 import signal
5 import time
6 from pathlib import Path
7
8 import numpy as np
9
10 signal.alarm(60)
11 started = time.monotonic()
12 root = Path.cwd()
13 result_dir = root / "求解/问题2/结果"
14 module_spec = importlib.util.spec_from_file_location(
15     "appendix_problem_two", root / "求解/问题2/求解_问题2.py"
16 )
17 solver = importlib.util.module_from_spec(module_spec)
18 module_spec.loader.exec_module(solver)
19
20 def read_result(name):
21     return json.loads((result_dir / name).read_text(encoding="utf-8"))
22
23 def parameter_record(name, fold, candidate):
24     angle_records = []
25     for angle_data, model in zip(fold["angles"], candidate["模型"]):
26         angle_records.append({
27             "角度_度": model["角度_度"],
28             "常相位_弧度": model["相位_rad"],
29             "基线系数_常数至二次": model["系数"][3:].tolist(),
30             "幅值系数_常数至一次": model["系数"][3:].tolist(),
31             "训练尺度_比例": angle_data["scale"],
32             "矩阵秩": model["秩"],
33             "矩阵条件数": model["条件数"],
34         })
35     return {
36         "拟合对象": name,
37         "共享参数顺序": ["厚度_微米", "参考折射率", "色散系数"],
38         "共享参数": list(candidate["参数"]),
39         "坐标中心_每厘米": fold["center"],
40         "坐标半跨度_每厘米": fold["half_span"],
41         "训练点数_每角": len(fold["train_indices"]),
42         "逐角参数": angle_records,
43     }
44
45 thickness = read_result("厚度结果.json")
46 validation = read_result("验证.json")
47 coverage = read_result("区间覆盖.json")
48 benchmark = read_result("基准交接.json")
49 coordinates, values, source_rows, datasets, audit = solver.read_inputs()
50 assert source_rows == benchmark["抽样源行"]
51 assert len(coordinates) == benchmark["抽样点数_每角度"]
52 np.testing.assert_array_equal(
53     coordinates, [entry["波数_cm^-1"] for entry in audit["抽样索引"]]
54 )
55
56 all_parameters = []
57 rebuilt_blocks = []
58 rebuilt_baselines = []
59 for fold_spec in benchmark["折分"]:
60     fold = solver.make_fold(coordinates, values, fold_spec)
61     for edge in fold["guard_edges"]:
62         distances = np.abs(coordinates[fold["train_indices"]] - edge)
63         assert np.min(distances) >= benchmark["训练保护带_cm^-1"]
64     saved = next(
65         item for item in validation["双向留角度冻结诊断"]
66         if item["折号"] == fold_spec["折号"]
67     )
68     shared = (
69         saved["冻结厚度_um"], saved["冻结参考折射率"],
70         saved["冻结色散系数"],
71     )
72     candidate = solver.fit_candidate(shared, fold)
73     all_parameters.append(parameter_record(fold_spec["折号"], fold, candidate))
74     rebuilt_blocks.extend(solver.score_blocks(fold, candidate))
75     rebuilt_baselines.extend(solver.baseline_blocks(fold))
76
77 for rebuilt in rebuilt_blocks:
78     saved = next(
79         item for item in validation["分块"]
80         if all(item[key] == rebuilt[key]
81             for key in ("折号", "角度_度", "测试块"))

```

```

82 )
83 for key in ("波数_cm^-1", "预测反射率_比例", "标准化均方根误差",
84            "训练尺度_比例", "区间半宽_比例", "经验覆盖率"):
85     np.testing.assert_allclose(rebuilt[key], saved[key], rtol=1e-8, atol=1e-10)
86
87 main_score = np.mean([item["标准化均方根误差"] for item in rebuilt_blocks])
88 baseline_score = np.mean([
89     item["标准化均方根误差"] for item in rebuilt_baselines
90     if item["基线"] == "训练二次趋势"
91 ])
92 improvement = 100 * (baseline_score - main_score) / baseline_score
93 for actual, key in (
94     (main_score, "正式主方法分数_连续留段标准化均方根误差"),
95     (baseline_score, "正式二次趋势基线分数_连续留段标准化均方根误差"),
96     (improvement, "正式主方法相对正式二次趋势改善_百分比"),
97 ):
98     np.testing.assert_allclose(actual, validation[key], rtol=1e-8)
99
100 covered = sum(
101     np.count_nonzero(np.abs(np.asarray(item["残差_比例"]))
102                     <= item["区间半宽_比例"])
103     for item in rebuilt_blocks
104 )
105 test_count = sum(item["点数"] for item in rebuilt_blocks)
106 assert test_count == coverage["测试点数"]
107 np.testing.assert_allclose(covered / test_count, coverage["测试经验覆盖率"])
108 five_thicknesses = np.asarray(thickness["五折条件厚度_微米"])
109 relative_range = np.ptp(five_thicknesses) / np.median(five_thicknesses)
110 np.testing.assert_allclose(
111     relative_range, validation["五折跨切分稳定性"]["相对极差"]
112 )
113 range_members = [
114     value for group in thickness["条件区间组成"].values() for value in group
115 ]
116 np.testing.assert_allclose(
117     [min(range_members), max(range_members)], thickness["条件区间_微米"]
118 )
119
120 full_fold = solver.full_fold(coordinates, values)
121 full_shared = (
122     thickness["同片最佳厚度_微米"],
123     thickness["同片最佳厚度的参考折射率"],
124     thickness["同片最佳厚度的经验色散系数"],
125 )
126 full_candidate = solver.fit_candidate(full_shared, full_fold)
127 all_parameters.append(parameter_record("全量", full_fold, full_candidate))
128 summary = {
129     "完整参数": all_parameters,
130     "逐块评分": [
131         {key: item[key] for key in (
132             "折号", "角度_度", "测试块", "点数", "标准化均方根误差",
133             "均方根误差_比例", "区间半宽_比例", "经验覆盖率",
134         )} for item in rebuilt_blocks
135     ],
136     "主方法误差": float(main_score),
137     "二次趋势误差": float(baseline_score),
138     "改善率_百分比": float(improvement),
139     "覆盖点数": int(covered),
140     "测试点数": int(test_count),
141     "五折相对极差": float(relative_range),
142     "条件范围_微米": thickness["条件区间_微米"],
143     "用时_秒": time.monotonic() - started,
144 }
145 print(json.dumps(summary, ensure_ascii=False, indent=2))
146 print("复核摘要: 共同坐标、完整参数、逐块预测与正式汇总一致。")
147 signal.alarm(0)
148 PY

```

重新计算全部情景采用清单9，原始附件与输入档案保持原位，输出包括各阶段结果和计算摘要。该计算与固定参数复核承担不同任务：前者重新搜索，后者检查本文所列数值能否由保存的参数与观测重建。

清单 9: 全部情景的重新计算命令

```
python3 求解/问题2/求解_问题2.py
```

完整实现以原始附件重建全部中间量；逐块数值则保留本次正式计算的结果。后续补充材料据此索引原始行号和全部比较，使同一厚度及其预测表现始终能够追溯到同一组观测。

问题二完整源码统一见清单7，重新计算命令见清单9。

8.3 问题三源码：往返场及配对对照

两束截断与完整往返的机制比较要固定数据、尺度、光学参数及响应，只切换返回阶数；若分别重估参数，所得差值还包含估参变化。两束截断保留首轮返回光，完整往返则把后续每轮带着衰减和相位的返回光依次相加；逐次往返厚度对照据此组织两种场，再分别核对硅的全量厚度和碳化硅的沿用结果。

图10说明固定界面关系时返回场如何叠加，图11则展示分别拟合后的预测与残差。前者对应场的构造，后者对应整套估计方案；共同测试区段只固定了比较对象，不能单独隔离返回阶数的作用。

清单11中的配对求解让两模型共享粗候选、起点和评估上限，均重估全部光学参数及各角响应。分别估参的厚度差应与固定同一光学参数的纯场差分开，不能由其中一项代替另一项。

点集核对改变了最初的全量计算方案：每角 480 点的计划折分曾同时承担全量估计，但逐行核对发现主窗口实际包含每角 5392 个原始点。因此将计划折分保留为区段稳定性检验，并另用窗口内全部原始坐标和值重新估计厚度，避免以抽取点集冒称全量。

原始全量拟合给出的硅厚度为 2.17 μm ，输入保留每角 5392 个观测点。图13将这一全量点估计与折间结果分开标识；硅的厚度计算独立于碳化硅是否需要修正，必要物理条件的判断不取消硅的数值回答。

折间厚度分别为 1.73 和 16.04 μm ，各自对应既定连续区段的估计对象。它们描述区段选择带来的分支移动，既不平均成全量答案，也不把两端点当作全量厚度的概率区间。全量与折间分别保留之后，材料处理才有明确的比较基准。

材料处理还须区分反射率预测与厚度修正：完整往返反射率区间的测试覆盖率为 73.91%，低于名义 90%。图12保留各材料、角度及区段的改善方向，表11则并列谱形误差与厚度；覆盖率的计量对象始终是反射率观测点，而修正厚度还需要相干性、空间重叠与仪器分辨率的观测依据。

碳化硅沿用第二问的 10.80 μm 及 0.64–18.11 μm 光学情景范围，因为附件缺少相干、空间重叠和仪器分辨率资料，谱形改善尚未构成更换厚度的物理依据。表31把全量、折间和沿用关系分列，完整往返候选与实际采用厚度各有对应位置。

表 31 两材料全量厚度与折间结果分列

计算对象	厚度 / μm	数据或沿用关系
硅全量	2.17	每角 5392 个原始点
硅连续区段	1.73、16.04	每角 480 点的计划切分
碳化硅同窗对照	2.7870 $\rightarrow$ 2.8301	两束 $\rightarrow$ 完整场，采用值不变
碳化硅采用值	10.80	沿用第二问同片基准
碳化硅情景范围	0.64–18.11	沿用第二问光学情景
共同波数窗口	—	1200–3800 cm^{-1}

沿用关系使碳化硅的处理前后保持同一个对象，而不是把换模型后的候选直接冠以修正厚度。现有观测对应保留基准的分支，两束与完整往返的候选仍成对保存；补充仪器资料后，可以沿同一数据对象重新判断高阶返回是否足以改变厚度。

已知光学参数下，匹配情景的最大厚度相对误差绝对值为 0.0521%；表32按原情景列出全部 12 例直接厚度回收。低损耗、强损耗与无高阶负对照使用匹配的光学关系，色散失配组在估计时将色散设为零。真厚度、观测和生成条件保持不变，表中厚度单位均为 μm 。

表 32 已知厚度的回收误差与包络包含

生成情景	真厚度	估计厚度	误差幅度 (%)	包络含真值
低损耗	2.3500	2.3501	0.0048	否
低损耗	7.4500	7.4498	0.0023	否
低损耗	12.6500	12.6500	3.55×10^{-5}	否
强损耗	2.3500	2.3488	0.0521	是
强损耗	7.4500	7.4496	0.0056	否
强损耗	12.6500	12.6496	0.0031	否
无高阶负对照	2.3500	2.3501	0.0029	否
无高阶负对照	7.4500	7.4503	0.0036	否
无高阶负对照	12.6500	12.6502	0.0019	否
色散失配	2.3500	2.2629	3.7082	否
色散失配	7.4500	6.7823	8.9622	否
色散失配	12.6500	11.7205	7.3479	否

点估计回收误差与包络包含是不同问题：即使最优厚度贴近真值，已搜索到的近优分支仍可能集中在真值同一侧。这里以同一训练目标最优值的 10% 为容差构成合成近优包络，包含判定使用未舍入端点；匹配组的 1/9 是模拟计数，不是实测置信水平。实测多情景包络还联合切分与光学扰动，其宽度来源与这组固定光学参数检验不同。

往返场实现见清单11，其来源与结果记录中的重算入口一致。程序依赖 Python、NumPy 及 SciPy；工作簿的解压、数值单元格读取及散列核对由标准库完成。附件、数据档案和第二问基准的相对位置固

定，输入索引见清单10；不读取旧第三问程序的估计值作为新答案。

各角响应先用训练数据确定中心与尺度，再从物理反射率列中扣除低阶背景投影，使背景变化与条纹响应分开拟合。物理响应使用非负增益，并对增益与背景系数施加收缩。背景阶数、增益阶数和收缩强度均由训练内连续留段选择，两种场共用候选、起点与评估上限。校准数据构造反射率区间，测试数据用于评分；内层比较未完成时沿用预先固定的响应设置，不从部分候选中择优。朴素模型胜出只改变预测模型标签，硅厚度仍由完整往返场估计。

一次重算得到的是独立保存的候选产物，不自动替换本文采用的声明。清单10把估计、数值核对与汇总的现有步骤分别对应到产物；汇总程序按案例拆出逐块评分和扰动记录，并保留碳化硅的前问基准。随后形成的解读核验更新声明措辞，厚度仍对应原有核心指标。新候选需重新完成这些核对，不能沿用旧运行的比较结论。

清单 10: 第三问的输入与结果对应关系

```
输入（相对工作根）：
数据/附件1.xlsx 至 数据/附件4.xlsx
交接/数据档案.json
求解/问题2/结果/基准交接.json

唯一估计入口：求解/问题3/升格3/求解.py
源码SHA256:
653c3f11046b42d388403f1342de2bae5d0e47733a8d1986290cba237b7ae45f
对应：求解/问题3/结果/公平对照.json 的实现SHA256

原估计产物：求解/问题3/升格3/结果/
隔离重算产物：本次运行目录/结果/
公平对照.json；厚度结果.json；区间覆盖.json
结果声明_问题3.json（候选声明）；执行状态.json
训练内选择_<材料>_折<折号>.json
基础估计_<材料>_折<折号>.json
预测_<材料>_折<折号>.csv
配对重采样_<材料>.json；已知厚度验证.json
扩展计算的完成范围以实际记录为准。

既有核对：求解/问题3/裁决_升格.py
调用：求解/问题3/核验_升格标准库.py
产物：日志/升格裁决_问3核验/同口径比较.json
      日志/升格裁决_问3核验/独立公式与评分核验.json
既有汇总：求解/问题3/整理_升格裁决.py
产物：求解/问题3/结果/候选同口径比较.json
      求解/问题3/结果/裁决核验.json
      求解/问题3/结果/同口径对照.json
      求解/问题3/结果/灵敏度.json
      求解/问题3/结果/声明自检指标.json
      求解/问题3/结果/条件判定.json
      求解/问题3/结果/优胜来源.json
      交接/结果声明_问题3.json
本轮解读核对：求解/问题3/核验解读_回炉轮2.py
核对对象：求解/问题3/结果/回炉轮2候选/升格3复算/
核对产物：求解/问题3/结果/解读核验_回炉轮2.json
当前声明：交接/结果声明_问题3.json
```

上述核对、汇总程序会写入各自的结果、交接和日志目录，因此不在隔离重算命令中执行。清单12只调用当前估计器，将其所有数值输出重定向到新建目录；既有汇总产物作为只读对应索引，不与本次候选混用。

清单 11: 往返场与同段对照的完整实现

```
1  """问题3变体3：保留往返衰减场，重写响应、光程搜索及训练内验证。
2
3  直接运行本文件；所有结果写入本文件旁的结果目录，不改主线声明或评审。
4  默认18分钟主动停止扩展计算；初始六案例先保存，随后逐案例、逐扰动保存。
5  本文件在设计交付阶段仅作静态检查，不执行反演。
6  """
7
8  import csv
9  import hashlib
10 import itertools
11 import json
12 import math
13 import os
14 import posixpath
15 import time
16 import xml.etree.ElementTree as element_tree
17 from pathlib import Path
18 from zipfile import ZipFile
19
20 import numpy as np
21 from scipy.optimize import minimize
```

```

22
23
24 ROOT = Path(__file__).resolve().parents[3]
25 RESULT = Path(__file__).resolve().parent / "结果"
26 NAMESPACE = {"sheet": "http://schemas.openxmlformats.org/spreadsheetml/2006/main"}
27 MATERIALS = {"硅": (3, 4), "碳化硅": (1, 2)}
28 MODELS = ("两束", "完整往返")
29 ANGLES = (10.0, 15.0)
30 BOUNDS = np.array([[0.5, 40.0], [1.2, 6.0], [-1.0, 1.0], [-1.5, 2.5], [0.0, 3.0]])
31 OPTICAL_LOW = np.array([0.6, 1.2, -1.0, -1.5, 0.0])
32 OPTICAL_SPAN = np.array([239.4, 4.8, 2.0, 4.0, 3.0])
33 CONFIGS = ((0, 0, 0.1), (1, 0, 0.1), (2, 0, 0.1),
34            (1, 1, 0.1), (2, 1, 0.1), (2, 1, 1.0))
35 DEFAULT_CONFIG = (1, 0, 0.1)
36 GAIN_CAP = 4.0
37 SOFT_SECONDS = 1080.0
38 SEED = 20260910
39 VERSION = "往返场_正交收缩响应_光程多起点_嵌套原块_v3"
40
41
42 def native(value):
43     if isinstance(value, np.ndarray):
44         return native(value.tolist())
45     if isinstance(value, np.generic):
46         return value.item()
47     if isinstance(value, dict):
48         return {str(key): native(item) for key, item in value.items()}
49     if isinstance(value, (list, tuple)):
50         return [native(item) for item in value]
51     return value
52
53
54 def atomic_json(path, payload):
55     path.parent.mkdir(parents=True, exist_ok=True)
56     temporary = path.with_name(path.name + f".{os.getpid()}.tmp")
57     with temporary.open("w", encoding="utf-8") as stream:
58         json.dump(native(payload), stream, ensure_ascii=False, indent=2, allow_nan=False)
59         stream.write("\n")
60         stream.flush()
61         os.fsync(stream.fileno())
62     os.replace(temporary, path)
63
64
65 def fingerprint(path):
66     return hashlib.sha256(path.read_bytes()).hexdigest()
67
68
69 def read_sheet(path):
70     with ZipFile(path) as archive:
71         workbook = element_tree.fromstring(archive.read("xl/workbook.xml"))
72         relations = element_tree.fromstring(archive.read("xl/_rels/workbook.xml.rels"))
73         targets = {item.get("Id"): item.get("Target") for item in relations}
74         sheet = next(item for item in workbook.findall("sheet:sheets/sheet:sheet", NAMESPACE)
75                     if item.get("name") == "Sheet1")
76         relation = sheet.get("{http://schemas.openxmlformats.org/officeDocument/2006/relationships}id")
77         target = targets[relation]
78         member = target.lstrip("/") if target.startswith("/") else posixpath.normpath(
79             posixpath.join("xl", target))
80         document = element_tree.fromstring(archive.read(member))
81         records = []
82         for row in document.findall("sheet:sheetData/sheet:row", NAMESPACE):
83             source_row = int(row.get("r"))
84             if source_row == 1:
85                 continue
86             cells = {cell.get("r"): cell for cell in row.findall("sheet:c", NAMESPACE)}
87             values = []
88             for column in ("A", "B"):
89                 cell = cells.get(f"{column}{source_row}")
90                 if (cell is None or cell.get("t", "n") != "n"
91                     or cell.find("sheet:f", NAMESPACE) is not None):
92                     raise ValueError(f"{path.name}第{source_row}行并非原始数值")
93                 values.append(float(cell.findtext("sheet:v", namespaces=NAMESPACE)))
94             if not all(math.isfinite(value) for value in values) or values[0] <= 0:
95                 raise ValueError(f"{path.name}第{source_row}行有非法数值")
96             records.append((source_row, values[0], values[1] / 100.0))
97         records.sort(key=lambda item: item[1])
98         table = np.asarray(records)
99         if len(table) != 7469 or len(np.unique(table[:, 1])) != 7469:
100             raise ValueError("原附件行数或波数唯一性不符数据档案")
101         return table
102
103
104 def load_inputs():

```

```

105 archive = json.loads((ROOT / "交接/数据档案.json").read_text(encoding="utf-8"))
106 baseline_path = ROOT / "求解/问题2/结果/基准交接.json"
107 baseline = json.loads(baseline_path.read_text(encoding="utf-8"))
108 if baseline["共同窗口_cm^-1"] != [1200, 3800] or baseline["抽样点数_每角度"] != 480:
109     raise ValueError("上游窗口或抽样口径变化, 需显式重新定义比较")
110 tables = {}
111 hashes = {}
112 for number in range(1, 5):
113     path = ROOT / "数据" / f"附件{number}.xlsx"
114     hashes[path.name] = fingerprint(path)
115     tables[number] = read_sheet(path)
116     metadata = next(item for item in archive["文件档案"] if item["文件名"] == path.name)
117     if hashes[path.name] != metadata["文件哈希"]:
118         raise ValueError(f"{path.name} 哈希不符数据档案")
119 for record in baseline["输入哈希"]:
120     if hashes[record["文件名"]] != record["SHA256"]:
121         raise ValueError("问题2基准与原附件不匹配")
122 sigma = tables[1][:, 1]
123 if any(not np.array_equal(sigma, tables[number][:, 1]) for number in range(2, 5)):
124     raise ValueError("四谱不再逐行对齐")
125 full_indices = np.flatnonzero((sigma >= 1200.0) & (sigma <= 3800.0))
126 selected = full_indices[np.arange(480) * (len(full_indices) - 1) // 479]
127 rows = tables[1][selected, 0].astype(int).tolist()
128 if baseline["抽样源行"] != [rows, rows]:
129     raise ValueError("480点源行不符上游; 禁止静默换样本")
130 expected = [
131     {"折号": 1, "训练块": [1, 2, 4, 5, 7, 8, 10, 11], "校准块": [3, 9], "测试块": [6, 12]},
132     {"折号": 2, "训练块": [2, 3, 5, 6, 8, 9, 11, 12], "校准块": [4, 10], "测试块": [1, 7]},
133 ]
134 if baseline["折分"] != expected or baseline["训练保护带_cm^-1"] != 20:
135     raise ValueError("原外层块或保护带改变")
136 sampled_sigma = sigma[selected]
137 edges = (sampled_sigma[39:440:40] + sampled_sigma[40:480:40]) / 2.0
138 return {"tables": tables, "full_indices": full_indices, "selected": selected,
139         "edges": edges, "folds": expected, "baseline": baseline, "hashes": hashes,
140         "baseline_hash": fingerprint(baseline_path), "rows": rows}
141
142
143 def build_case(material, tables, indices, edges, specification=None, train=None):
144     coordinates = tables[1][indices, 1].copy()
145     values = np.asarray([tables[number][indices, 2] for number in MATERIALS[material]])
146     blocks = np.searchsorted(edges, coordinates, side="right") + 1
147     if specification is None:
148         specification = {"折号": 0, "训练块": list(range(1, 13)), "校准块": [], "测试块": []}
149     if train is None:
150         train_blocks = specification["训练块"]
151         guards = [edge for block, edge in enumerate(edges, 1)
152                  if (block in train_blocks) != (block + 1 in train_blocks)]
153         keep = np.isin(blocks, train_blocks)
154         for edge in guards:
155             keep &= np.abs(coordinates - edge) >= 20.0
156         train = np.flatnonzero(keep)
157     case = {"material": material, "spec": specification, "sigma": coordinates,
158            "values": values, "blocks": blocks, "train": np.asarray(train, dtype=int),
159            "rows": tables[1][indices, 0].astype(int), "angles": ANGLES,
160            "weight": 0.5, "angle_weights": np.array([0.5, 0.5]), "cache": {}}
161     return prepare_case(case)
162
163
164 def prepare_case(case):
165     case = dict(case, cache={})
166     case["abscissa"] = (case["sigma"] - 2500.0) / 1300.0
167     train = case["train"]
168     if len(train) < 16:
169         raise ValueError("训练成员少于16点")
170     basis = np.vander(case["abscissa"], 3, increasing=True)
171     case["means"] = case["values"][:, train].mean(axis=1)
172     coefficients = np.linalg.solve(basis[train].T @ basis[train] + 1e-12 * np.eye(3),
173                                   basis[train].T @ case["values"][:, train].T)
174     residual = case["values"][:, train] - (basis[train] @ coefficients).T
175     case["scales"] = np.maximum(np.quantile(residual, 0.75, axis=1)
176                                - np.quantile(residual, 0.25, axis=1), 1e-4)
177     case["naive"] = {"训练均值": np.repeat(case["means"][:, None], len(case["sigma"]), axis=1),
178                    "二次趋势": (basis @ coefficients).T}
179     return case
180
181
182 def select_training(case, train):
183     return prepare_case(dict(case, train=np.asarray(train, dtype=int)))
184
185
186 def feasible(parameters, case):
187     if np.any(parameters < BOUNDS[:, 0] - 1e-9) or np.any(parameters > BOUNDS[:, 1] + 1e-9):

```

```

188         return False
189     film = parameters[1] + parameters[2] * ((2000.0 / case["sigma"][[0, -1]]) ** 2 - 1.0)
190     threshold = max(math.sin(math.radians(angle)) for angle in case["angles"]) + 1e-6
191     return bool(np.min(film) > threshold and np.min(film + parameters[3]) > threshold)
192
193
194 def fields(parameters, case, complete, return_terms=False):
195     if not feasible(parameters, case):
196         return None
197     thickness, reference, dispersion, contrast, loss = parameters
198     sigma = case["sigma"]
199     film = reference + dispersion * ((2000.0 / sigma) ** 2 - 1.0)
200     substrate = film + contrast
201     spectra = []
202     term_rows = []
203     for angle in case["angles"]:
204         sine_squared = math.sin(math.radians(angle)) ** 2
205         normal_air = math.cos(math.radians(angle))
206         normal_film = np.sqrt(film ** 2 - sine_squared)
207         normal_substrate = np.sqrt(substrate ** 2 - sine_squared)
208         propagation = np.exp(-loss * film / normal_film
209                               + 4j * math.pi * thickness * sigma * normal_film / 10000.0)
210         intensities = []
211         for air, layer, support in ((normal_air, normal_film, normal_substrate),
212                                     (1 / normal_air, film ** 2 / normal_film,
213                                      substrate ** 2 / normal_substrate)):
214             reflection = (air - layer) / (air + layer)
215             interface = (layer - support) / (layer + support)
216             first = 4 * air * layer / (air + layer) ** 2 * interface * propagation
217             ratio = -reflection * interface * propagation
218             if np.max(np.abs(ratio)) >= 1.0:
219                 return None
220             field = reflection + (first / (1 - ratio) if complete else first)
221             intensities.append(np.abs(field) ** 2)
222             if return_terms:
223                 term_rows.append((reflection, first, ratio, field))
224             spectra.append(case["weight"] * intensities[0] + (1 - case["weight"]) * intensities[1])
225     return (np.asarray(spectra), term_rows) if return_terms else np.asarray(spectra)
226
227
228 def box_quadratic(gram, right, gain_cap):
229     best = None
230     for state in itertools.product((0, 1, 2), repeat=len(right)):
231         gains = np.array([gain_cap if flag == 2 else 0.0 for flag in state])
232         free = np.flatnonzero(np.array(state) == 1)
233         fixed = np.flatnonzero(np.array(state) != 1)
234         if len(free):
235             gains[free] = np.linalg.solve(gram[np.ix_(free, free)],
236                                           right[free] - gram[np.ix_(free, fixed)] @ gains[fixed])
237         if np.min(gains) < -1e-9 or np.max(gains) > gain_cap + 1e-9:
238             continue
239         gains = np.clip(gains, 0.0, gain_cap)
240         objective = float(gains @ gram @ gains - 2 * right @ gains)
241         if best is None or objective < best[0]:
242             best = (objective, gains)
243     return best[1]
244
245
246 def response_fit(case, physical, config):
247     degree, gain_degree, penalty = config
248     train = case["train"]
249     cache_key = (degree, gain_degree, penalty)
250     if cache_key not in case["cache"]:
251         background = np.vander(case["abscissa"], degree + 1, increasing=True)
252         inverse = np.linalg.pinv(background[train], rcond=1e-12)
253         trend_penalty = penalty / 10.0 * np.arange(degree + 1, dtype=float) ** 2
254         background_gram = background[train].T @ background[train] / len(train)
255         background_gram += np.diag(trend_penalty)
256         weights = (np.ones((len(background), 1)) if gain_degree == 0 else
257                   np.column_stack(((1 - case["abscissa"]) / 2, (1 + case["abscissa"]) / 2)))
258         case["cache"][cache_key] = (background, inverse, background_gram, weights, trend_penalty)
259     background, inverse, background_gram, weights, trend_penalty = case["cache"][cache_key]
260     predictions = []
261     rows = []
262     objectives = []
263     for angle_index, spectrum in enumerate(physical):
264         scale = case["scales"][angle_index]
265         mean = case["means"][angle_index]
266         target = (case["values"][angle_index, train] - mean) / scale
267         raw = spectrum[:, None] * weights
268         projection = inverse @ raw[train]
269         centered = raw - background @ projection
270         normalization = np.maximum(np.sqrt(np.mean(centered[train] ** 2, axis=0)), 1e-8)

```

```

271     response = centered / normalization
272     coefficients = np.linalg.solve(background_gram, background[train].T @ target / len(train))
273     residual_target = target - background[train] @ coefficients
274     gram = response[train].T @ response[train] / len(train) + penalty * np.eye(gain_degree + 1)
275     right = response[train].T @ residual_target / len(train)
276     gain_cap = case.get("gain_cap", GAIN_CAP)
277     gains = box_quadratic(gram, right, gain_cap)
278     predicted = mean + scale * (background @ coefficients + response @ gains)
279     residual = (case["values"][angle_index, train] - predicted[train]) / scale
280     objective = float(np.mean(residual ** 2) + penalty * (gains @ gains)
281                     + np.sum(trend_penalty * coefficients ** 2))
282     predictions.append(predicted)
283     objectives.append(objective)
284     rows.append({"背景系数": coefficients, "标准化非负增益": gains, "物理投影系数": projection,
285                "物理列尺度": normalization, "训练响应均值_比例": mean,
286                "训练响应尺度_比例": scale, "训练标准化均方误差": np.mean(residual ** 2),
287                "惩罚后损失": objective, "增益上界": gain_cap})
288     return np.asarray(predictions), rows, float(np.dot(case["angle_weights"], objectives))
289
290
291 def evaluate(parameters, case, model, config):
292     parameters = np.asarray(parameters, dtype=float)
293     physical = fields(parameters, case, model == "完整往返")
294     if physical is None:
295         return None
296     predicted, response, loss = response_fit(case, physical, config)
297     if not np.isfinite(loss) or not np.all(np.isfinite(predicted)):
298         return None
299     return {"parameters": parameters.copy(), "prediction": predicted, "response": response,
300           "loss": loss, "config": config, "model": model}
301
302
303 def encode(parameters):
304     optical = np.array(parameters, dtype=float)
305     optical[0] *= optical[1]
306     return (optical - OPTICAL_LOW) / OPTICAL_SPAN
307
308
309 def decode(unit):
310     parameters = OPTICAL_LOW + np.asarray(unit) * OPTICAL_SPAN
311     parameters[0] /= parameters[1]
312     return parameters
313
314
315 class StopSearch(Exception):
316     pass
317
318
319 def refine(seed, case, model, config, deadline, maximum=100, fixed=None):
320     fixed = {} if fixed is None else dict(fixed)
321     parameters = np.asarray(seed, dtype=float).copy()
322     for index, value in fixed.items():
323         parameters[index] = value
324     best = evaluate(parameters, case, model, config)
325     if best is None:
326         for reference, dispersion, contrast in itertools.product(
327             (parameters[1], 2.0, 3.5, 5.8), (parameters[2], 0.0), (parameters[3], 0.8)):
328             repaired = parameters.copy()
329             repaired[1:4] = (reference, dispersion, contrast)
330             for index, value in fixed.items():
331                 repaired[index] = value
332             best = evaluate(repaired, case, model, config)
333             if best is not None:
334                 parameters = repaired
335                 break
336     if best is None:
337         return None, {"停止原因": "固定条件下未找到可行初值, 不等于证明无解", "评估次数": 0, "完整": False}
338     initial = encode(parameters)
339     free = [index for index in range(5) if index not in fixed]
340     calls = 0
341     reason = "优化结束"
342
343     def rebuild(values):
344         unit = initial.copy()
345         unit[free] = values
346         candidate = decode(unit)
347         for index, value in fixed.items():
348             candidate[index] = value
349         return candidate
350
351     def objective(values):
352         nonlocal calls, best
353         if time.monotonic() >= deadline:

```



```

354         raise StopSearch("时间预算")
355     if calls >= maximum:
356         raise StopSearch("共同评估上限")
357     calls += 1
358     candidate = evaluate(rebuild(values), case, model, config)
359     if candidate is None:
360         return 1e12
361     if candidate["loss"] < best["loss"]:
362         best = candidate
363     return candidate["loss"]
364
365 def constraints(values):
366     candidate = rebuild(values)
367     endpoints = case["sigma"][[0, -1]]
368     film = candidate[1] + candidate[2] * ((2000.0 / endpoints) ** 2 - 1)
369     threshold = max(math.sin(math.radians(angle)) for angle in case["angles"]) + 2e-6
370     return np.concatenate((candidate[0] - 0.5, 40 - candidate[0]],
371                             film - threshold, film + candidate[3] - threshold))
372
373 try:
374     outcome = minimize(objective, initial[free], method="SLSQP", bounds=[(0, 1)] * len(free),
375                         constraints={"type": "ineq", "fun": constraints},
376                         options={"maxiter": 30, "ftol": 1e-7, "eps": 1e-6})
377     reason = "收敛" if outcome.success else f"优化器停止:{outcome.status}"
378 except StopSearch as error:
379     reason = str(error)
380 return best, {"停止原因": reason, "评估次数": calls, "共同评估上限": maximum,
381              "完整": reason != "时间预算", "优化器收敛": reason == "收敛",
382              "全局最优保证": False, "固定参数索引": list(fixed)}
383
384
385 def spectral_seeds(case, deadline):
386     train = case["train"]
387     if len(train) > 480:
388         train = train[np.arange(480) * (len(train) - 1) // 479]
389     sigma = case["sigma"][train]
390     baseline = np.vander(case["abscissa"][train], 3, increasing=True)
391     residual = case["values"][:, train].T
392     residual = residual - baseline @ np.linalg.lstsq(baseline, residual, rcond=1e-12)[0]
393     residual /= case["scales"][None, :]
394     seeds = [np.array([5.0, 3.0, 0.0, 0.8, 0.2])]
395     grid = np.linspace(0.5, 40.0, 384)
396     for reference, dispersion in itertools.product((1.6, 2.6, 3.6, 4.8, 5.8), (-0.3, 0.0, 0.3)):
397         if time.monotonic() >= deadline:
398             break
399         seed = np.array([5.0, reference, dispersion, 0.8, 0.2])
400         if not feasible(seed, case):
401             continue
402         scores = np.zeros(len(grid))
403         film = reference + dispersion * ((2000 / sigma) ** 2 - 1)
404         for angle_index, angle in enumerate(case["angles"]):
405             normal = np.sqrt(film ** 2 - math.sin(math.radians(angle)) ** 2)
406             phase = 4 * math.pi * grid[:, None] * sigma[None, :] * normal[None, :] / 10000
407             cosine, sine = np.cos(phase), np.sin(phase)
408             cosine -= cosine.mean(axis=1, keepdims=True)
409             sine -= sine.mean(axis=1, keepdims=True)
410             scores += case["angle_weights"][angle_index] * (
411                 (cosine @ residual[:, angle_index]) ** 2 / np.maximum(np.sum(cosine ** 2, axis=1), 1e
412 ↪12)
413                 + (sine @ residual[:, angle_index]) ** 2 / np.maximum(np.sum(sine ** 2, axis=1), 1e
414 ↪12))
415         chosen = []
416         for index in np.argsort(scores)[::-1]:
417             if all(abs(grid[index] - old) >= 0.5 for old in chosen):
418                 chosen.append(float(grid[index]))
419                 for contrast in (-0.6, 0.8):
420                     candidate = np.array([grid[index], reference, dispersion, contrast, 0.2])
421                     if feasible(candidate, case):
422                         seeds.append(candidate)
423                 if len(chosen) == 2:
424                     break
425         return seeds
426
427 def fit_pair(case, config, seeds, deadline, maximum=100, starts=2, fixed=None):
428     pools = {model: [] for model in MODELS}
429     coarse_complete = True
430     for seed in seeds:
431         if time.monotonic() >= deadline and pools[MODELS[0]]:
432             coarse_complete = False
433             break
434         for model in MODELS:
435             candidate = evaluate(seed, case, model, config)

```

```

435         if candidate is not None:
436             pools[model].append(candidate)
437     if any(not pool for pool in pools.values()):
438         raise ValueError("共同物理候选均不可行")
439     common = []
440     for rank in range(starts):
441         for model in MODELS:
442             ordered = sorted(pools[model], key=lambda item: item["loss"])
443             seed = ordered[min(rank, len(ordered) - 1)]["parameters"]
444             if not any(np.allclose(seed, previous, rtol=0, atol=1e-10) for previous in common):
445                 common.append(seed)
446     best = {model: min(pool, key=lambda item: item["loss"]) for model, pool in pools.items()}
447     traces = {model: [] for model in MODELS}
448     for seed in common:
449         for model in MODELS:
450             candidate, trace = refine(seed, case, model, config, deadline, maximum=maximum, fixed=fixed)
451             traces[model].append(trace)
452             if candidate is not None:
453                 pools[model].append(candidate)
454                 if candidate["loss"] < best[model]["loss"]:
455                     best[model] = candidate
456     complete = coarse_complete and all(row["完整"] for group in traces.values() for row in group)
457     branches = {}
458     for model in MODELS:
459         branches[model] = {f"训练目标容差{tolerance:g}%": [
460             float(min(item["parameters"][0] for item in pools[model]
461                 if item["loss"] <= best[model]["loss"] * (1 + tolerance / 100) + 1e-12)),
462             float(max(item["parameters"][0] for item in pools[model]
463                 if item["loss"] <= best[model]["loss"] * (1 + tolerance / 100) + 1e-12))]
464         for tolerance in (0, 5, 10)}
465     return best, {"公平性成立": complete, "共同初值": common, "共同粗候选数": len(seeds),
466                 "粗搜索完整": coarse_complete, "优化记录": traces, "训练近优分支范围_um": branches}
467
468
469 def rmse(case, prediction, indices):
470     return np.sqrt(np.mean((prediction[:, indices] - case["values"][:, indices]) ** 2, axis=1)) / case
471     ↪["scales"]
472
473 def inner_partitions(case):
474     allowed = case["train"]
475     available = sorted(set(case["blocks"][allowed].tolist()))
476     chosen = sorted(set((available[0], available[-1], available[len(available) // 3],
477         available[2 * len(available) // 3])))
478     partitions = []
479     for block in chosen:
480         validation = allowed[case["blocks"][allowed] == block]
481         lower, upper = case["sigma"][validation[0, -1]]
482         training = allowed[(case["sigma"][allowed] < lower - 20)
483             | (case["sigma"][allowed] > upper + 20)]
484         if len(training) >= 16 and len(validation) >= 8:
485             partitions.append((select_training(case, training), validation, block))
486     return partitions
487
488
489 def choose_config(case, deadline, progress):
490     partitions = inner_partitions(case)
491     records = [] for config in CONFIGS]
492     details = []
493     complete = True
494     for inner, validation, block in partitions:
495         if time.monotonic() >= deadline:
496             complete = False
497             break
498         seeds = spectral_seeds(inner, deadline)
499         naive = {label: rmse(inner, predicted, validation) for label, predicted in inner["naive"].
500     ↪items()}
501         detail = {"原始块号": block, "训练源行": inner["rows"][inner["train"]],
502             "验证源行": inner["rows"][validation], "朴素误差_逐角": naive, "候选": []}
503         for index, config in enumerate(CONFIGS):
504             if time.monotonic() >= deadline:
505                 complete = False
506                 break
507             best, trace = fit_pair(inner, config, seeds, deadline, maximum=70, starts=1)
508             scores = {model: rmse(inner, best[model]["prediction"], validation) for model in MODELS}
509             reference = np.minimum(naive["训练均值"], naive["二次趋势"])
510             ratios = np.concatenate([scores[model] / np.maximum(reference, 1e-8) for model in MODELS])
511             records[index].append({"分块误差": scores, "朴素误差": naive,
512                 "风险比": ratios, "公平完整": trace["公平性成立"]})
513             detail["候选"].append({"响应设置": config, "误差_逐角": scores,
514                 "公平完整": trace["公平性成立"]})
515         complete &= trace["公平性成立"]
516     details.append(detail)

```

```

516     progress({"内层分块": details, "状态": "训练内比较中"})
517 complete &= all(len(rows) == len(partitions) for rows in records) and len(partitions) >= 3
518 summary = []
519 for config, rows in zip(CONFIGS, records):
520     if rows:
521         ratios = np.concatenate([row["风险比"] for row in rows])
522         summary.append({"响应设置": config, "完成内折数": len(rows),
523             "平均风险比": float(ratios.mean()), "最大风险比": float(ratios.max()),
524             "选择目标": float(0.5 * ratios.mean() + 0.5 * ratios.max())})
525     selected = min(summary, key=lambda row: (row["选择目标"], sum(row["响应设置"][:2]))) if complete
526     config = tuple(selected["响应设置"]) if selected else DEFAULT_CONFIG
527     model_scores = {}
528     if complete:
529         chosen = records[CONFIGS.index(config)]
530         for model in (*MODELS, "训练均值", "二次趋势"):
531             values = np.concatenate([row["分块误差" if model in MODELS else "朴素误差"][model] for row
532                 in chosen])
533             model_scores[model] = float(0.5 * np.mean(values) + 0.5 * np.max(values))
534     selected_model = min(model_scores, key=model_scores.get) if model_scores else "完整往返"
535     report = {"状态": "完整" if complete else "计算截断, 使用预先固定响应, 不从部分候选中择优",
536         "选型完整": bool(complete), "内层分块": details, "候选汇总": summary,
537         "所选响应设置": config, "内层预测选择分数": model_scores,
538         "所选预测模型": selected_model, "条件厚度模型": "完整往返",
539         "选择依据": "同原始块的端点与内部留段; 两物理模型等权, 平均与最差风险比各占一半",
540         "校准用途": "只构造区间, 不选择参数、响应或模型",
541         "解释": "若朴素模型胜出仅改变预测标签, 不以朴素误差冒充往返场精度"}
542     progress(report)
543     return config, selected_model, report
544
545 def parameter_record(candidate):
546     parameters = candidate["parameters"]
547     unidentifiable = all(np.max(row["标准化非负增益"]) <= 1e-8 for row in candidate["response"])
548     return {"厚度_um": parameters[0], "参考折射率": parameters[1], "色散系数": parameters[2],
549         "衬底折射率对比": parameters[3], "有效往返损耗": parameters[4],
550         "参考光学厚度_um": parameters[0] * parameters[1], "参数向量": parameters,
551         "响应设置_背景阶_增益阶_惩罚": candidate["config"], "训练惩罚后目标": candidate["loss"],
552         "各角响应": candidate["response"], "厚度未识别": unidentifiable,
553         "点值含义": "零增益平台的算法代表值, 有效物理答案是全搜索范围" if unidentifiable else "给定光学与响应族
554         的条件解, 不保证唯一",
555         "触边参数索引": np.flatnonzero(np.any(np.isclose(parameters[:, None], BOUNDS,
556             rtol=0, atol=1e-4), axis=1)),
557         "补充条件": "需独立折射率色散、仪器响应或标定厚度消解多解; 经验色散不是已测光学常数"}
558
559 def empirical_half_width(residuals, alpha=0.1):
560     residuals = np.sort(np.abs(np.asarray(residuals)))
561     rank = math.ceil((len(residuals) + 1) * (1 - alpha))
562     return None if rank > len(residuals) else float(residuals[rank - 1])
563
564
565 def score_case(case, best, selected_model):
566     predictions = {model: best[model]["prediction"] for model in MODELS}
567     predictions.update(case["naive"])
568     calibration = np.flatnonzero(np.isin(case["blocks"], case["spec"]["校准块"]))
569     rows = []
570     intervals = []
571     for model, prediction in predictions.items():
572         for angle_index, number in enumerate(MATERIALS[case["material"]]):
573             residuals = prediction[angle_index, calibration] - case["values"][angle_index, calibration]
574             half_width = empirical_half_width(residuals)
575             block_residuals = [float(np.max(np.abs(prediction[angle_index, case["blocks"] == block]
576                 - case["values"][angle_index, case["blocks"] == block]))
577                 for block in case["spec"]["校准块"]]
578             block_width = empirical_half_width(block_residuals)
579             for block in case["spec"]["测试块"]:
580                 indices = np.flatnonzero(case["blocks"] == block)
581                 residual = prediction[angle_index, indices] - case["values"][angle_index, indices]
582                 raw_rmse = float(np.sqrt(np.mean(residual ** 2)))
583                 rows.append({"材料": case["material"], "折号": case["spec"]["折号"], "附件": number,
584                     "入射角_度": case["angles"][angle_index], "测试块": block, "模型": model,
585                     "测试点数": len(indices), "标准化均方根误差": raw_rmse / case["scales"][
586                         angle_index],
587                     "均方根误差_反射率比例": raw_rmse,
588                     "所选预测模型": selected_model, "源行": case["rows"][indices]})
589             distance = np.min(np.abs(case["sigma"][indices, None]
590                 - case["sigma"][case["train"]][None, :]), axis=1)
591             intervals.append({"材料": case["material"], "折号": case["spec"]["折号"], "附件": number,
592                 "测试块": block, "模型": model, "名义覆盖率": 0.9,
593                 "点残差半宽_比例": half_width,

```

```

593         "经验覆盖率": float(np.mean(np.abs(residual) <= half_width)) if
    ↪half_width is not None else 1.0,
594         "平均区间宽度_比例": None if half_width is None else 2 * half_width,
595         "距最近训练波数均值_cm^-1": float(distance.mean()),
596         "块同时覆盖半宽_比例": block_width,
597         "块同时覆盖半宽类型": "正无穷" if block_width is None else "有限",
598         "块同时经验覆盖率": 1.0 if block_width is None else float(np.all(np.abs
    ↪(residual) <= block_width)),
599         "解释": "点残差有限样本秩校正; 连续谱不交换, 不宣称共形保证。仅2校准块时90%块共
    ↪形半宽必为无穷"})
600     return rows, intervals
601
602
603 def field_checks(case, candidate, deadline):
604     physical, terms = fields(candidate["parameters"], case, True, return_terms=True)
605     finite_errors = {}
606     tail_bounds = {}
607     maximum_ratio = max(float(np.max(np.abs(row[2]))) for row in terms)
608     needed = 1 if maximum_ratio == 0 else min(4096, max(64, math.ceil(math.log(1e-15) / math.log(
    ↪maximum_ratio))))
609     for count in sorted(set((1, 2, 4, 8, 16, 32, 64, needed))):
610         if time.monotonic() >= deadline:
611             break
612         errors = []
613         tails = []
614         for reflection, first, ratio, complete in terms:
615             term = first.copy()
616             finite = reflection.astype(complex).copy()
617             for order in range(count):
618                 finite += term
619                 term *= ratio
620             errors.append(float(np.max(np.abs(finite - complete))))
621             tails.append(float(np.max(np.abs(first) * np.abs(ratio) ** count / (1 - np.abs(ratio)))))
622             finite_errors[str(count)] = max(errors)
623             tail_bounds[str(count)] = max(tails)
624     sample = np.unique(np.linspace(0, len(case["sigma"]) - 1, 33, dtype=int))
625     scalar_errors = []
626     thickness, reference, dispersion, contrast, loss = candidate["parameters"]
627     for angle_index, angle in enumerate(case["angles"]):
628         sine_squared = math.sin(math.radians(angle)) ** 2
629         air_normal = math.cos(math.radians(angle))
630         for index in sample:
631             sigma = float(case["sigma"][index])
632             film = reference + dispersion * ((2000 / sigma) ** 2 - 1)
633             substrate = film + contrast
634             film_normal = math.sqrt(film ** 2 - sine_squared)
635             substrate_normal = math.sqrt(substrate ** 2 - sine_squared)
636             phase = 4 * math.pi * thickness * sigma * film_normal / 10000
637             propagation = math.exp(-loss * film / film_normal) * complex(math.cos(phase), math.sin(phase)
    ↪)
638             reflected = []
639             for air, layer, support in ((air_normal, film_normal, substrate_normal),
640                                         (1 / air_normal, film ** 2 / film_normal, substrate ** 2 /
    ↪substrate_normal)):
641                 front = (air - layer) / (air + layer)
642                 back = (layer - support) / (layer + support)
643                 first = 2 * air / (air + layer) * 2 * layer / (air + layer) * back * propagation
644                 field = front + first / (1 + front * back * propagation)
645                 reflected.append(abs(field) ** 2)
646                 value = case["weight"] * reflected[0] + (1 - case["weight"]) * reflected[1]
647                 scalar_errors.append(abs(value - physical[angle_index, index]))
648     two_physical = fields(candidate["parameters"], case, False)
649     gain_degree = candidate["config"][1]
650     weights = (np.ones((len(case["sigma"]), 1)) if gain_degree == 0 else
651               np.column_stack(((1 - case["abscissa"]) / 2, (1 + case["abscissa"]) / 2)))
652     observation = []
653     for angle_index, response in enumerate(candidate["response"]):
654         change = case["scales"][angle_index] * (
655             physical[angle_index] - two_physical[angle_index][:, None]
656             * weights / response["物理列尺度"]) @ response["标准化非负增益"]
657         amplitude = float(np.sqrt(np.mean(change[case["train"]] ** 2)))
658         noise = float(np.sqrt(np.mean((candidate["prediction"][angle_index, case["train"]]
659                                     - case["values"][angle_index, case["train"]]) ** 2)))
660         observation.append(("附件": MATERIALS[case["material"]][angle_index],
661                             "同参数同响应高阶改变量均方根_比例": amplitude,
662                             "训练残差均方根_比例": noise,
663                             "高阶改变量相对残差": amplitude / max(noise, 1e-12),
664                             "必要条件": "非零返回、后续反射且往返模小于1; 不代表相干条件已知",
665                             "未提供条件": ["时间相干长度", "空间重叠", "仪器分辨率"],
666                             "替代解释": "平滑响应失配、色散误设、折射率厚度互补; 需结合逐块验证"})
667     return {"有限项复场最大差": finite_errors, "几何余项上界": tail_bounds,
668            "最大往返乘子模": maximum_ratio, "传播与收敛约束满足": maximum_ratio < 1,
669            "标量独立实现最大差_反射率比例": max(scalar_errors),

```

```

670 "闭式核通过": finite_errors.get(str(needed), 1.0) <= 1e-12 and max(scalar_errors) <= 1e-12,
671 "固定参数两束与完整场均方根差_比例": np.sqrt(np.mean((physical - two_physical) ** 2, axis=1)),
672 "逐附件高阶可观测性诊断": observation}

```

清单11 (续): 扰动与验证

```

675 def sensitivity_specs(candidate):
676     parameters = candidate["parameters"]
677     specs = [("有效损耗未扰动控制", 4, parameters[4], None),
678             ("有效损耗减20%", 4, parameters[4] * 0.8, None),
679             ("有效损耗加20%", 4, parameters[4] * 1.2, None),
680             ("有效损耗绝对增加0.1", 4, 0.1, "仅零损耗"),
681             ("有效损耗绝对增加0.2", 4, 0.2, "仅零损耗")]
682     for index, name, factors in ((3, "衬底对比", (0.8, 1.2)), (1, "参考折射率", (0.97, 1.03)),
683                                (2, "色散系数", (0.8, 1.2))):
684         for factor in factors:
685             specs.append((f'{name}{'减' if factor < 1 else '加'}{abs(factor - 1) * 100:g}%',"
686                          index, parameters[index] * factor, None))
687     specs.extend((label, None, value, kind) for label, value, kind in (
688         ("偏振权重全s", 1.0, "偏振"), ("偏振权重全p", 0.0, "偏振"),
689         ("入射角同时减0.5度", -0.5, "角度"), ("入射角同时加0.5度", 0.5, "角度"),
690         ("窗口下界加100", 1300.0, "下界"), ("窗口上界减100", 3700.0, "上界")))
691     if abs(parameters[2]) <= 1e-8:
692         specs.extend(("零色散绝对减0.05", 2, -0.05, None), ("零色散绝对加0.05", 2, 0.05, None))
693     if abs(parameters[3]) <= 1e-8:
694         specs.extend(("零衬底对比绝对减0.1", 3, -0.1, None), ("零衬底对比绝对加0.1", 3, 0.1, None))
695     specs.extend(("响应收缩减半", None, 0.5, "收缩"), ("响应收缩加倍", None, 2.0, "收缩"),
696                 ("标准化增益上界减半", None, GAIN_CAP / 2, "增益上界"),
697                 ("标准化增益上界加倍", None, GAIN_CAP * 2, "增益上界")))
698     return specs
699
700
701 def sensitivity_case(case, candidate, spec, deadline):
702     label, index, requested, kind = spec
703     row = {"材料": case["material"], "折号": case["spec"]["折号"], "情景": label,
704           "基准厚度_um": candidate["parameters"][0], "请求固定参数值": requested}
705     if kind == "仅零损耗" and abs(candidate["parameters"][4]) > 1e-8:
706         return dict(row, 状态="不适用", 原因="当前基准损耗非零; 不是遗漏重估")
707     if time.monotonic() >= deadline:
708         return dict(row, 状态="未完成", 原因="18分钟软截止; 不把未计算情景写成稳定")
709     changed = dict(case)
710     config = candidate["config"]
711     if kind == "偏振":
712         changed["weight"] = requested
713     elif kind == "角度":
714         changed["angles"] = tuple(angle + requested for angle in ANGLES)
715     elif kind in ("下界", "上界"):
716         keep = case["sigma"][case["train"]] >= requested if kind == "下界" else case["sigma"][case["
717         ←train"]] <= requested
718         changed["train"] = case["train"][keep]
719     elif kind == "收缩":
720         config = (*config[:2], config[2] * requested)
721     elif kind == "增益上界":
722         changed["gain_cap"] = requested
723     changed = prepare_case(changed)
724     fixed = {} if index is None else {index: float(np.clip(requested, *BOUNDS[index]))}
725     control_fixed = {} if index is None else {index: float(candidate["parameters"][index])}
726     control_deadline = time.monotonic() + max(0.0, deadline - time.monotonic()) / 2
727     control, control_trace = refine(candidate["parameters"], case, "完整往返", candidate["config"],
728                                     control_deadline, maximum=110, fixed=control_fixed)
729     best, trace = refine(candidate["parameters"], changed, "完整往返", config,
730                         deadline, maximum=110, fixed=fixed)
731     if best is None:
732         return dict(row, 状态="约束不可行", 停止状态=trace)
733     return dict(row, 状态="已重估", 重估完整参数=parameter_record(best), 停止状态=trace,
734               实际固定参数值=fixed.get(index), 扰动后厚度_um=float(best["parameters"][0]),
735               厚度变化_um=float(best["parameters"][0] - candidate["parameters"][0]),
736               相对变化_百分数=float(100 * (best["parameters"][0] - candidate["parameters"][0] - 1)),
737               未扰动控制参数=None if control is None else parameter_record(control),
738               未扰动控制停止状态=control_trace,
739               扣除控制厚度变化_um=None if control is None else float(best["parameters"][0] - control["
740     ←parameters"][0]),
741               配对可比=bool(control is not None and control_trace["完整"] and trace["完整"]),
742               训练源行=changed["rows"][changed["train"]])
743
744 def cross_angle(case, best, deadline):
745     rows = []
746     for source in range(2):
747         if time.monotonic() >= deadline:
748             break

```



```

748     changed = dict(case, angle_weights=np.eye(2)[source], cache={})
749     seeds = spectral_seeds(changed, deadline)
750     fitted, trace = fit_pair(changed, DEFAULT_CONFIG, seeds, deadline,
751                             maximum=90, starts=1)
752     target = 1 - source
753     test = np.flatnonzero(np.isin(case["blocks"], case["spec"]["测试块"]))
754     rows.append({"训练物理参数的附件": MATERIALS[case["material"]][source],
755                 "目标附件": MATERIALS[case["material"]][target],
756                 "目标响应拟合成员": "仅目标角外训练; 共享物理量冻结",
757                 "响应设置": "预先固定默认值, 不借用双角联合选型结果",
758                 "标准化误差": {model: rmse(case, fitted[model]["prediction"], test)[target] for
    ←model in MODELS},
759                 "条件厚度_um": {model: fitted[model]["parameters"][0] for model in MODELS},
760                 "搜索状态": trace})
761     return rows
762
763 def summarize_validation(rows):
764     groups = {}
765     for row in rows:
766         key = (row["材料"], row["折号"], row["附件"], row["测试块"])
767         groups.setdefault(key, {})[row["模型"]] = row["标准化均方根误差"]
768     comparisons = []
769     for key, scores in groups.items():
770         row = {"材料": key[0], "折号": key[1], "附件": key[2], "测试块": key[3], "各模型误差": scores}
771         row["完整相对两束下降_%"] = 100 * (1 - scores["完整往返"] / max(scores["两束"], 1e-12))
772         for label in ("训练均值", "二次趋势"):
773             row[f"完整相对{label}下降_%"] = 100 * (1 - scores["完整往返"] / max(scores[label], 1e-12))
774         comparisons.append(row)
775     material_summary = {}
776     for material in MATERIALS:
777         relevant = [row for row in comparisons if row["材料"] == material]
778         fold_summary = []
779         for fold in (1, 2):
780             members = [row for row in relevant if row["折号"] == fold]
781             if members:
782                 means = {model: float(np.mean([row["各模型误差"][model] for row in members]))
783                         for model in (*MODELS, "训练均值", "二次趋势")}
784                 fold_summary.append({"折号": fold, "完成比较单元数": len(members), "平均标准化均方根误差":
    ←means,
785                                     "完整相对基线下降_%": {model: 100 * (1 - means["完整往返"] / max(
    ←means[model], 1e-12))
786                                                         for model in ("两束", "训练均值", "二次趋势")}
    ←}))
787     material_summary[material] = {"逐角逐折逐块": relevant, "逐折汇总": fold_summary,
788                                   "预期比较单元数": 8, "完成比较单元数": len(relevant),
789                                   "收益阈值敏感性": [{"收益阈值_%": threshold,
790                                                         "超过阈值单元数": sum(row["完整相对两束下降_%"] > threshold for row in
    ←relevant),
791                                                         "全部单元超过": len(relevant) == 8 and all(row["完整相对两束下降_%"] >
    ←threshold for row in relevant)}
792                                   for threshold in (0.0, 0.5, 1.0, 2.0)],
793                                   "全部块严格胜两朴素基线": len(relevant) == 8 and all(row[f"完整相对{label}下降_%"] > 0
794                                             for row in relevant for label in ("训练均值", "二次趋
    ←势")),
795                                   "高阶收益全部块同向为正": len(relevant) == 8 and all(row["完整相对两束下降_%"] > 0 for row in
    ←relevant),
796                                   "反向块": [row for row in relevant if row["完整相对两束下降_%"] <= 0],
797                                   "通过含义": "只是本次内部逐块数值诊断; 不代写G2裁定, 不宣称新独立验证"})
798     return material_summary
799
800 def bootstrap_pair(case, best, deadline, progress, repetitions=40):
801     random_seed = SEED + (0 if case["material"] == "硅" else 1)
802     generator = np.random.default_rng(random_seed)
803     train = case["train"]
804     blocks = sorted(set(case["blocks"])[train].tolist())
805     rows = []
806     for repetition in range(repetitions):
807         if time.monotonic() >= deadline:
808             break
809         selected = generator.choice(blocks, size=len(blocks), replace=True)
810         members = np.concatenate([train[case["blocks"]][train] == block for block in selected])
811         changed = select_training(case, members)
812         fitted, trace = fit_pair(changed, best["完整往返"]["config"],
813                                 [best[model]["parameters"] for model in MODELS], deadline,
814                                 maximum=60, starts=1)
815         row = {"编号": repetition + 1, "抽取原块": selected,
816               "厚度_um": {model: fitted[model]["parameters"][0] for model in MODELS},
817               "配对厚度差_um": fitted["完整往返"]["parameters"][0] - fitted["两束"]["parameters"][0],
818               "完整": trace["公平性成立"]}
819         rows.append(row)
820     progress(rows)
821

```

```

823     valid = [row for row in rows if row["完整"]]
824     result = {"材料": case["material"], "计划次数": repetitions, "完成次数": len(valid), "随机种子":
    ↪random_seed,
825             "逐次记录": rows, "构造": "同次12个原波数块成对重采样并重估; 固定已选响应, 局部条件稳定性, 不作厚度置
    ↪信区间",
826             "厚度经验覆盖率": None, "厚度经验覆盖率不可计算原因": "真实厚度未随附件提供"}
827     if valid:
828         differences = [row["配对厚度差_um"] for row in valid]
829         result["配对厚度差经验分位范围_um"] = np.quantile(differences, [0.025, 0.975])
830         result["完整厚度经验分位范围_um"] = np.quantile([row["厚度_um"] for row in valid],
    ↪[0.025, 0.975])
831         relative = abs(best["完整往返"]["parameters"][0] / best["两束"]["parameters"][0] - 1) * 100
832         result["厚度影响情景判断"] = [{"阈值%": threshold, "绝对相对变化%": relative,
833                                         "超过情景阈值": relative > threshold} for threshold in (0.5, 1,
    ↪2)]
834     return result
835
836
837 def known_thickness_validation(template, deadline, t0):
838     destination = ROOT / "数据/问题3_冻结合成输入/升格3条件验证"
839     destination.mkdir(parents=True, exist_ok=True)
840     generator = np.random.default_rng(SEED + 30)
841     manifest = {"用途": "独立已知光学条件的数值验证, 不进入实测核心指标", "随机种子": SEED + 30, "文件": []}
842     rows = []
843     types = (("低损耗", 0.05, True), ("强损耗", 1.5, True),
844             ("无高阶负对照", 0.1, False), ("色散失配", 0.05, True))
845     for kind, loss, complete in types:
846         for thickness, reference in ((2.35, 2.2), (7.45, 3.6), (12.65, 5.0)):
847             if time.monotonic() >= deadline:
848                 break
849             truth = np.array([thickness, reference, 0.18, 0.8, loss])
850             physical = fields(truth, template, complete)
851             observed = physical + generator.normal(0.0, 0.002, size=physical.shape)
852             identifier = f"情景{len(rows) + 1:02d}"
853             path = destination / f"{identifier}.csv"
854             temporary = path.with_name(path.name + ".tmp")
855             with temporary.open("w", encoding="utf-8-sig", newline="") as stream:
856                 writer = csv.writer(stream)
857                 writer.writerow(["原附件源行", "波数_cm^-1", "反射率10度_比例", "反射率15度_比例"])
858                 writer.writerows(zip(template["rows"], template["sigma"], observed[0], observed[1]))
859                 stream.flush()
860                 os.fsync(stream.fileno())
861             os.replace(temporary, path)
862             manifest["文件"].append({"文件名": path.name, "用途": kind,
863                                     "生成方式": "固定真参数往返场或两束场, 加独立高斯反射率比例噪声; 原波数坐标不变",
864                                     "参数": {"真厚度_um": thickness, "参考折射率": reference, "色散系数": 0.18,
865                                               "衬底对比": 0.8, "损耗": loss, "噪声标准差_比例": 0.002, "完整往返": complete},
866                                     "SHA256": fingerprint(path)})
867             atomic_json(destination / "输入清单.json", manifest)
868             changed = prepare_case(dict(template, values=observed.copy()))
869             fixed = {1: reference, 2: 0.0 if kind == "色散失配" else 0.18, 3: 0.8, 4: loss}
870             seeds = [np.array([value, fixed[1], fixed[2], fixed[3], fixed[4]])
871                     for value in np.linspace(0.5, 40.0, 384)]
872             best, trace = fit_pair(changed, (0, 0, 0.01), seeds, deadline,
873                                   maximum=90, starts=2, fixed=fixed)
874             selected = "完整往返" if complete else "两束"
875             interval = trace["训练近优分支范围_um"][selected]["训练目标容差10%"]
876             scores, coverage = score_case(changed, best, selected)
877             rows.append({"情景": kind, "冻结输入": str(path.relative_to(ROOT)), "输入SHA256": fingerprint
    ↪(path),
878                        "真厚度_um": thickness, "估计厚度_um": best[selected]["parameters"][0],
879                        "相对误差%": 100 * (best[selected]["parameters"][0] / thickness - 1),
880                        "条件包络_um": interval, "真厚度落入条件包络": interval[0] <= thickness <=
    ↪interval[1],
881                        "配对厚度差_um": best["完整往返"]["parameters"][0] - best["两束"]["parameters"]
    ↪[0],
882                        "逐块评分": scores, "反射率区间覆盖": coverage, "搜索状态": trace,
883                        "口径": "已知光学参数下检验厚度搜索; 失配组故意将色散设零, 不能代替未知光学常数的实测精度"
    ↪})
884             atomic_json(RESULT / "已知厚度验证.json", {"案例": rows, "实际用时秒": time.monotonic() - t0})
885             matched = [row for row in rows if row["情景"] != "色散失配" and row["搜索状态"]["公平性成立"]]
886             null = [abs(row["配对厚度差_um"]) for row in rows
887                    if row["情景"] == "无高阶负对照" and row["搜索状态"]["公平性成立"]]
888             result = {"案例": rows, "完成匹配情景数": len(matched), "计划情景数": 12,
889                     "条件包络经验包含率": float(np.mean([row["真厚度落入条件包络"] for row in matched])) if matched
    ↪else None,
890                     "条件包络名义覆盖率": None,
891                     "包络构造": "同一训练惩罚目标10%内的已计算物理解; 只是条件包络, 不冒充95%置信区间",
892                     "无高阶样本数": len(null), "无高阶绝对厚度差95%经验分位_um": float(np.quantile(null, 0.95)) if
    ↪null else None,
893                     "阈值用途": "少量独立负对照仅校查误判风险, 不足以承诺实测5%误判率或自动修正碳化硅",
894                     "厚度覆盖解释": "这里存在模拟真值; 经验包含率不转用作实测厚度覆盖率",
895                     "冻结输入清单": str((destination / "输入清单.json").relative_to(ROOT)),

```

```

896         "实际用时秒": time.monotonic() - t0}
897     atomic_json(RESULT / "已知厚度验证.json", result)
898     return result

```

清单11 (续): 结果发布与记录

```

901 def write_predictions(case, best):
902     path = RESULT / f"预测_{case['material']}_{折{case['spec']['折号']}}.csv"
903     temporary = path.with_name(path.name + ".tmp")
904     train = set(case["train"].tolist())
905     predictions = {model: best[model]["prediction"] for model in MODELS}
906     predictions.update(case["naive"])
907     with temporary.open("w", encoding="utf-8-sig", newline="") as stream:
908         writer = csv.writer(stream)
909         writer.writerow(["附件", "源行", "波数_cm^-1", "原始块", "用途", "反射率_比例", *predictions])
910         for angle_index, number in enumerate(MATERIALS[case["material"]]):
911             for index, sigma in enumerate(case["sigma"]):
912                 block = int(case["blocks"][index])
913                 kind = ("训练" if index in train else "校准" if block in case["spec"]["校准块"]
914                        else "测试" if block in case["spec"]["测试块"] else "保护带")
915                 writer.writerow([number, case["rows"][index], sigma, block, kind,
916                                case["values"][angle_index, index],
917                                *(prediction[angle_index, index] for prediction in predictions.values
918                                ↪())])
918             stream.flush()
919             os.fsync(stream.fileno())
920             os.replace(temporary, path)
921
922
923 def publish(payload, t0):
924     payload["实际用时秒"] = time.monotonic() - t0
925     cases = payload["案例"]
926     ranges = {}
927     for material in MATERIALS:
928         estimates = [row[model]["厚度_um"] for row in cases if row["材料"] == material for model in
929         ↪MODELS]
930         for row in payload["灵敏度_参数扰动"]:
931             if row["材料"] == material and row.get("状态") == "已重估":
932                 estimates.append(row["扰动后厚度_um"])
933         for row in cases:
934             if row["材料"] != material:
935                 continue
936             for branches in row.get("搜索状态", {}).get("训练近优分支范围_um", {}).values():
937                 estimates.extend(branches["训练目标容差10%"])
938             ranges[material] = ([0.5, 40.0] if any(row[model]["厚度未识别"] for row in cases
939             ↪estimates)))
940             if row["材料"] == material for model in MODELS) else [min(estimates), max(
941             ↪estimates)])
942     silicon = next(row for row in cases if row["材料"] == "硅" and row["折号"] == 0)
943     silicon_folds = {row["折号"]: row for row in cases if row["材料"] == "硅"}
944     baseline = payload["碳化硅正式基准"]
945     core = {"硅全量完整往返厚度_um": silicon["完整往返"]["厚度_um"],
946            "硅折1完整往返条件厚度_um": silicon_folds[1]["完整往返"]["厚度_um"],
947            "硅折2完整往返条件厚度_um": silicon_folds[2]["完整往返"]["厚度_um"],
948            "硅已计算条件包络下限_um": ranges["硅"][0], "硅已计算条件包络上限_um": ranges["硅"][1],
949            "碳化硅正式基准厚度_um": baseline["厚度_um"],
950            "碳化硅正式条件范围下限_um": baseline["条件范围_um"][0],
951            "碳化硅正式条件范围上限_um": baseline["条件范围_um"][1]}
952     payload.update({"核心指标": core, "厚度条件范围_um": ranges,
953                    "硅全量条件厚度_um": core["硅全量完整往返厚度_um"], "硅条件范围_um": ranges["硅"],
954                    "分材料验证": summarize_validation(payload["逐块评分"])}))
955     atomic_json(RESULT / "公平对照.json", payload)
956     atomic_json(RESULT / "厚度结果.json", {"问题": 3, "核心指标": core,
957            "厚度条件范围_um": ranges, "运行状态": payload["状态"],
958            "范围构造": "两模型全量、两折、训练目标10%近优候选及已算扰动的联合包络; 无概率含义",
959            "实际用时秒": payload["实际用时秒"]})
960     atomic_json(RESULT / "结果声明_问题3.json", {"问题": 3, "核心指标": core,
961            "口径说明": {"估计器版本": VERSION, "原始输入哈希": payload["输入哈希"],
962            "输入": "四个真实XLSX; 无合成观测进入任何核心指标",
963            "窗口_cm^-1": [1200, 3800], "全量每角点数": silicon["每角点数"],
964            "折内每角点数": 480, "上游基准哈希": payload["上游基准哈希"],
965            "执行实现SHA256": payload["实现SHA256"],
966            "独立复算": "按本文件与输入源行重算新估计器; 不得沿用旧估计器的对齐结论",
967            "旧外测试": "已用于开发诊断, 仅探索性复查; 未移动、删除或筛选任何测试块",
968            "厚度模型": "完整往返的条件最优, 不随朴素预测胜出而清空或改称朴素模型厚度"},
969            "置信": "缺少已知折射率、厚度与仪器信息; 条件多解, 验证不足照实保留数值及失败幅度",
970            "实际用时秒": payload["实际用时秒"]})
971     atomic_json(RESULT / "区间覆盖.json", {"逐角逐折逐块": payload["预测区间"],
972            "含义": "仅反射率预测经验覆盖; 绝不是厚度覆盖", "实际用时秒": payload["实际用时秒"]})
973
974 def case_record(case, best, trace, selected="完整往返", selection=None):

```

```

973     return {"材料": case["material"], "折号": case["spec"]["折号"], "每角点数": len(case["sigma"]),
974            "切分": case["spec"], "训练源行": case["rows"][case["train"]],
975            "训练样本序号_从零": case["train"], "训练尺度_比例": case["scales"],
976            "**{model: parameter_record(best[model]) for model in MODELS}", "搜索状态": trace,
977            "所选预测模型": selected, "训练内选择": selection}

```

清单11 (续): 执行入口

```

980 def main():
981     t0 = time.monotonic()
982     RESULT.mkdir(parents=True, exist_ok=True)
983     status = {"问题": 3, "状态": "读取真实附件", "实际用时秒": 0.0}
984     atomic_json(RESULT / "执行状态.json", status)
985     payload = None
986     try:
987         inputs = load_inputs()
988         cases = []
989         for material in MATERIALS:
990             cases.append(build_case(material, inputs["tables"], inputs["full_indices"], inputs["edges"]
↪))
991             cases.extend(build_case(material, inputs["tables"], inputs["selected"], inputs["edges"],
↪fold)
992                             for fold in inputs["folds"])
993     payload = {"问题": 3, "变体": 3, "主方法": "往返衰减场反演", "估计器版本": VERSION,
994              "实现SHA256": fingerprint(Path(__file__)),
995              "状态": "基础可行估计已算, 继续训练内选型", "案例": [], "灵敏度_参数扰动": [],
996              "逐块评分": [], "预测区间": [], "输入哈希": inputs["hashes"],
997              "上游基准哈希": inputs["baseline_hash"], "抽样源行": [inputs["rows"], inputs["rows"]],
998              "碳化硅正式基准": {"厚度_um": inputs["baseline"]["全量参数"]["厚度_um"],
999                                "条件范围_um": inputs["baseline"]["条件范围_um"], "是否替换": False
↪},
1000              "指标含义": {"厚度_um": "微米; 完整往返条件模型的估计厚度",
1001                           "标准化均方根误差": "原反射率比例RMSE除以该角训练二次去趋势残差IQR",
1002                           "训练惩罚后目标": "双角标准化MSE加响应收缩惩罚; 不能与留段RMSE混用",
1003                           "厚度条件范围_um": "已计算条件包络, 不是置信区间",
1004                           "相对下降_%": "100×(1-完整误差/对照误差); 负数保留",
1005                           "经验覆盖率": "校准点绝对残差秩区间在原测试点的实际命中比例"},
1006              "已知限制": ["两次旧外测试已参与开发, 不能再称独立检验",
1007                           "无真实厚度, 不能报告实测厚度覆盖率",
1008                           "不把响应增益归零时的任意厚度称为可识别点估计"]}
1009     working = []
1010     for case in cases:
1011         best, trace = fit_pair(case, DEFAULT_CONFIG, [np.array([5.0, 3.0, 0.0, 0.8, 0.2])],
1012                               min(t0 + 90, time.monotonic() + 3), maximum=18, starts=1)
1013         working.append(best)
1014         payload["案例"].append(case_record(case, best, trace))
1015         atomic_json(RESULT / f"基础估计_{case['material']}_折{case['spec']['折号']}.json",
1016                   dict(payload["案例"][-1], 实际用时秒=time.monotonic() - t0))
1017     publish(payload, t0)
1018     for index, case in enumerate(cases):
1019         if time.monotonic() >= t0 + 660:
1020             break
1021         deadline = min(t0 + 660, time.monotonic() + 95)
1022         case_name = f"{case['material']}_折{case['spec']['折号']}"
1023
1024         def save_selection(record, name=case_name):
1025             atomic_json(RESULT / f"训练内选择_{name}.json", dict(record, 实际用时秒=time.monotonic()
↪ t0))
1026
1027         config, selected, selection = choose_config(case, min(deadline, time.monotonic() + 60),
↪save_selection)
1028         seeds = spectral_seeds(case, deadline)
1029         best, trace = fit_pair(case, config, seeds, deadline, maximum=150, starts=3)
1030         trace["响应选择完整"] = selection["选型完整"]
1031         trace["公平性成立"] &= selection["选型完整"]
1032         working[index] = best
1033         record = case_record(case, best, trace, selected, selection)
1034         record["场级数核数"] = field_checks(case, best["完整往返"], t0 + SOFT_SECONDS)
1035         payload["案例"][index] = record
1036         if case["spec"]["折号"]:
1037             scores, intervals = score_case(case, best, selected)
1038             payload["逐块评分"].extend(scores)
1039             payload["预测区间"].extend(intervals)
1040             record["双向冻结留角度"] = cross_angle(case, best, min(t0 + 690, time.monotonic() + 5))
1041         write_predictions(case, best)
1042         publish(payload, t0)
1043         print(f"{case_name} 完整往返厚度={best['完整往返']['parameters'][0]:.8f} um; 预测选择={selected
↪}; 用时={time.monotonic()-t0:.1f}s", flush=True)
1044         for case, best in zip(cases, working):
1045             if not case["spec"]["折号"]:
1046                 continue

```

```

1047     present = any(row["材料"] == case["material"] and row["折号"] == case["spec"]["折号"])
1048         for row in payload["逐块评分"])
1049     if not present:
1050         scores, intervals = score_case(case, best, "完整往返")
1051         payload["逐块评分"].extend(scores)
1052         payload["预测区间"].extend(intervals)
1053         write_predictions(case, best)
1054     publish(payload, t0)
1055     for case, best in zip(cases, working):
1056         if not case["spec"]["折号"]:
1057             continue
1058         for spec in sensitivity_specs(best["完整往返"]):
1059             row = sensitivity_case(case, best["完整往返"], spec,
1060                                   min(t0 + 900, time.monotonic() + 3))
1061             payload["灵敏度_参数扰动"].append(row)
1062             publish(payload, t0)
1063     payload["已知厚度验证"] = known_thickness_validation(cases[1], min(t0 + 980, time.monotonic() +
1064     ↵45), t0)
1065     publish(payload, t0)
1066     payload["配对块重采样"] = []
1067     for case, best in zip(cases, working):
1068         if case["spec"]["折号"] or time.monotonic() >= t0 + 1040:
1069             continue
1070
1071     def save_bootstrap(rows, material=case["material"]):
1072         atomic_json(RESULT / f"配对重采样_{material}.json", {"逐次记录": rows, "实际用时秒": time.
1073     ↵monotonic() - t0))
1074
1075     result = bootstrap_pair(case, best, min(t0 + 1040, time.monotonic() + 60), save_bootstrap)
1076     payload["配对块重采样"].append(result)
1077     atomic_json(RESULT / f"配对重采样_{case['material']}.json", dict(result, 实际用时秒=time.
1078     ↵monotonic() - t0))
1079     publish(payload, t0)
1080     incomplete = (any(not row["搜索状态"].get("响应选择完整", False) for row in payload["案例"])
1081                   or any(row.get("状态") == "未完成" for row in payload["灵敏度_参数扰动"]))
1082     payload["状态"] = ("已形成答案; 部分扩展检验受限" if incomplete or time.monotonic() >= t0 +
1083     ↵SOFT_SECONDS
1084                       else "计算完成; 是否胜出须读取逐块诊断")
1085     publish(payload, t0)
1086     status.update({"状态": "正常结束", "实际用时秒": time.monotonic() - t0,
1087                  "核心指标": payload["核心指标"], "不代写评审结论": True})
1088     atomic_json(RESULT / "执行状态.json", status)
1089     print(json.dumps(native(status), ensure_ascii=False, allow_nan=False), flush=True)
1090 except Exception as error:
1091     status.update({"状态": "异常结束, 保留已计算产物", "原因": f"{type(error).__name__}:{error}",
1092                  "实际用时秒": time.monotonic() - t0})
1093     atomic_json(RESULT / "执行状态.json", status)
1094     if payload is not None and len(payload["案例"]) == 6:
1095         payload["状态"] = status["状态"]
1096         publish(payload, t0)
1097     print(json.dumps(status, ensure_ascii=False), flush=True)
1098     raise
1099
1100 if __name__ == "__main__":
1101     main()

```

隔离复算从工作根执行, 命令见清单12。输入散列由估计器核对, 源码散列与清单10及原结果保存值逐一比对。程序按时间预算保存各阶段结果; 需结合执行状态和每例搜索记录判断计算完成范围, 进程正常结束本身不保证候选数值逐位相同。

清单 12: 独立保存数值与失败日志的复算命令

```

run_dir="$(mktemp -d 日志/问题三复算.XXXXXX)" || exit 1
printf '本次复算目录: %s\n' "$run_dir"
python3 -B - "$run_dir" >"$run_dir/运行日志.txt" 2>&1 <<'PYCODE'
import hashlib
import importlib.util
import json
import signal
import sys
from pathlib import Path

sys.dont_write_bytecode = True
source_path = Path("求解/问题3/升格3/求解.py").resolve()
expected_digest = (
    "653c3f11046b42d388403f1342de2bae5d0"
    "e47733a8d1986290cba237b7ae45f"
)
actual_digest = hashlib.sha256(source_path.read_bytes()).hexdigest()
recorded_digest = json.loads(
    Path("求解/问题3/结果/公平对照.json").read_text(encoding="utf-8")
)

```



```
)["实现SHA256"]
if actual_digest != expected_digest or actual_digest != recorded_digest:
    raise RuntimeError("源码摘要与本文采用的结果不一致")

def stop_at_budget(signum, frame):
    raise TimeoutError("达到二十分钟上限，保留已写出结果")

signal.signal(signal.SIGALRM, stop_at_budget)
signal.alarm(1200)
module_spec = importlib.util.spec_from_file_location(
    "question_three", source_path
)
model = importlib.util.module_from_spec(module_spec)
module_spec.loader.exec_module(model)
model.RESULT = Path(sys.argv[1]).resolve() / "结果"
try:
    model.main()
finally:
    signal.alarm(0)
PYCODE
```

复算结果由点集、场模型和材料处理共同定位，既能核对硅的全量答案，也能追踪碳化硅沿用基准的依据。补充材料据此展开参数来源索引与各路线比较，将数值对应回同一套观测条件。

8.4 补充材料：来源索引与全部比较

每个厚度数值都应能沿着波段回到原始光谱。本节先列源附件与行号，再将路线比较、逐块检验和区间计数分开保存，使同一晶圆的角度差异与不同计算条件各有对应的观测对象。

8.4.1 来源索引：附件行号连接共同波段

源附件按材料和入射角定位，波段按原工作表行号定位。四个附件共享的坐标共有 7469 点，表33将其归回两块晶圆；图2保留原始异常，图14标出共同拟合区段，光学资料的已知与缺项则对应表3。

表 33 四个源附件的材料与工作表行号

源附件	材料	入射角	原始数据行	数据列
附件一	碳化硅	10°	2-7470	波数、反射率
附件二	碳化硅	15°	2-7470	波数、反射率
附件三	硅	10°	2-7470	波数、反射率
附件四	硅	15°	2-7470	波数、反射率

行号包含表头，均指源工作簿的首个工作表；波数列和反射率列分别位于左列与右列。因同材料两角的坐标逐行一致，联合计算直接配对原始行，不先平均反射率，也不通过插值改变观测位置。

主窗口为 1200–3800 cm⁻¹，表34把参与拟合的波段与原始行号并排定位。

附件二的超百反射率段为 801.28–927.11 cm⁻¹。它与主窗口没有交集，因此主窗口的路线差异并非裁掉异常值所带来的改善。

表 34 原始波段及参与计算的点数

对象	波数区间 (cm ⁻¹)	原始行号	每谱点数
四谱全段	399.6747–4000.1220	2–7470	7469
四谱主窗口	1200–3800	1663–7054	5392
连续留段点集	1200.470–3799.562	主窗口内选点	480
附件二超百段	801.2780–927.1104	835–1096	262
四谱共同首点	399.6747	2	1

抽样点集与全量点集承担不同任务：前者使各路线在同一组连续区段上比较，后者用于硅的主窗口整体拟合。共同抽样的 480 个点可由式(29)逐点回到工作表行号，而不是只知道所在波段。

$$\ell_j = 1663 + \left\lfloor \frac{5391j}{479} \right\rfloor, \quad j = 0, \dots, 479. \tag{29}$$

其中， j 为从零起算的共同抽样序号， ℓ_j 为包含表头的源工作表行号， $\lfloor \cdot \rfloor$ 表示向下取整。同一序号在四个附件中对应相同波数，但反射率仍分别取各自原值；原始波段、角度和行号确定后，路线优劣才有共同的比较对象。

8.4.2 路线比较：合成回收与实测预测分列

路线比较先区分是否知道真实厚度。早期曾考虑直接用观测光谱衡量测厚误差，但附件缺少厚度真值和折射率曲线，因而改在原始共同坐标上构造 24 例已知光学条件的合成光谱；这一实际取舍决定了

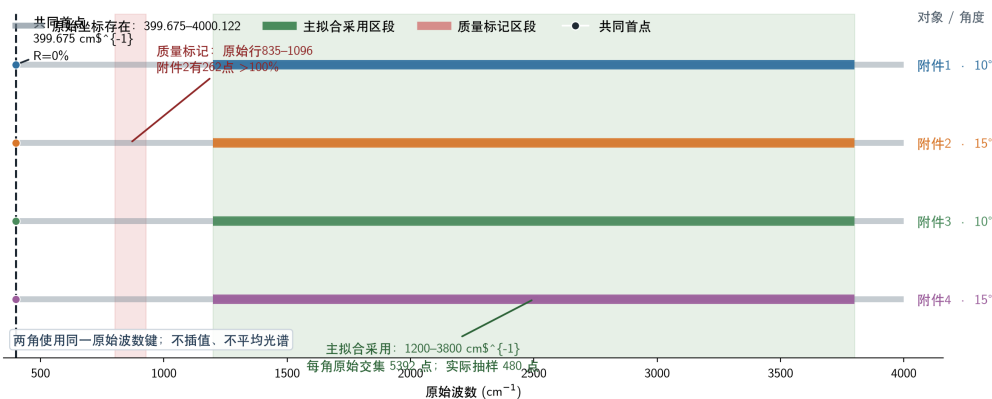


图 14 四谱共同波数坐标及拟合区段

图5和表7中的误差衡量方法回收能力，而实测留段误差衡量谱形预测。
 双束界面场反演在原型共同案例上的平均相对误差为 0.0010%，正式共同匹配组为 0.0008%。表35保留两个阶段的数值而不相互替换；新增匹配与故意改变光学关系的情景也单列，避免把匹配条件下的回收表现说成所有情景下的测厚精度。

表 35 问题一原型与正式合成误差

数据阶段	方法或情景	案例数	平均误差（%）
原型合成	色散峰序反演	24	0.0201
原型合成	双束界面场反演	24	0.0010
原型合成	连续相位拟厚	24	0.0041
正式合成	共同匹配	24	0.0008
正式合成	新增匹配	16	0.0011
正式合成	模型失配	10	0.3028
正式合成	常折射率峰距	24	2.5875

模型失配组改变的是生成观测的光学关系，而非删除困难案例后重新计分；因此它回答的是模型假设改变时的误差增长。原型与正式共同组虽然同属已知光学条件比较，含噪观测和估计步骤均有所不同，应按阶段各自对照。

碳化硅正式留段误差为 1.2074，二次趋势对照为 1.4404，均为无量纲谱形误差。这里的标准化误差，是每块反射率误差除以该折该角训练残差的四分位距，再对测试块等权平均；四分位距表示训练残差中间一半的散布宽度。表36保留三条原型路线与正式对照，表37另列原型厚度，防止把较小的谱形误差直接写成较小的厚度真值误差。

表 36 碳化硅三路线及正式谱形对照

路线或阶段	总体误差	首折误差	次折误差	覆盖（%）
双角色散变投影	1.1717	0.7662	1.5772	49.06
双角峰序匹配	1.2354	0.7319	1.7389	55.31
双角相位回归	1.2202	0.7906	1.6498	53.75
正式主方法	1.2074	—	—	49.06
正式二次趋势	1.4404	—	—	—
正式训练均值	3.1578	—	—	—

折间接近只描述当前切分下的表现。早期的双角色散变投影在两折间相差约 1.90%，随后将连续留段扩为五折时，厚度的相对极差达到 76.04%；因此保留原型候选，同时用更充分的切分比较解释正式厚度范围为何变宽。

表 37 碳化硅原型路线的折间厚度差异

原型路线	首折（μm）	次折（μm）	相对极差（%）
双角色散变投影	5.1710	5.0737	1.90
双角峰序匹配	4.2978	15.0358	111.08
双角相位回归	5.1109	3.9952	24.51

改变数据表达方式后，改善率也必须与本路线的基线成对使用。全谱稳健特征变体相对自身二次趋势改善 26.00%，其留段误差却高于所选路线；表38同时保留方法误差和各自基线，避免以不同分母的改善率直接排序。

表 38 碳化硅变体与各自基线的成对比较

计算条件	方法误差	自身基线误差	改善 (%)
所选双角路线	1.2074	1.4404	16.17
共同峰序变体	2.9711	2.7618	-7.58
全谱稳健特征	1.2500	1.6892	26.00
全点相位耦合	1.9911	0.6281	-216.98

全谱稳健特征和全点相位耦合改变了观测表达或响应约束，属于替代计算条件；负改善则明确表示方法误差高于自己的基线。它们保留为选型证据，不替换同一套正式点集上的比较。

往返衰减场反演在问题三原型中的总体误差为 1.1366，但硅分项相对自身两束对照反而增大 2.24%。这项真实尝试促使材料分项单列：表39保存三种方法的原型指标，正式检验再逐块检查收益方向。

表 39 问题三原型方法的分材料误差

原型方法	总体误差	硅误差	碳化硅误差	覆盖 (%)
往返衰减场反演	1.1366	1.5194	0.7538	65.31
共享相位谐波反演	2.6394	4.0070	1.2718	64.06
周期形状配准	2.2638	3.3372	1.1904	76.09

周期形状配准能形成有限数值，但其训练区段对重复相位的支持较弱；形状预测与界面往返机制因此分开解释。所有原型数值均留在比较中，正式结论则继续由共同测试点上的物理模型对照承担。

完整场的总体标准化误差为 1.0231，高于两束的 0.9812；表40将共同起点、相同预算的两模型与训练均值、二次趋势并列。纯场差须在单一光学参数下计算，重新估参的厚度差则另行比较。

表 40 问题三正式共同测试点的误差

比较方法	标准化误差	样本类别
完整往返	1.0231	正式实测
两束截断	0.9812	正式实测
训练均值	2.1929	正式实测
二次趋势	1.4434	正式实测

训练均值和二次趋势给出不解释干涉条纹的参照；两束截断则保留首轮返回，因此更直接检验高阶返回的增量。总体分数明确后，逐例与逐块记录继续揭示平均数中被掩盖的歧义和方向变化。

8.4.3 逐块比较：保留角度差异与失效案例

平均误差可能掩盖局部方向相反的区段。这里把合成相位歧义、碳化硅的切分变化和硅的逐块误差分别列出，检验对象始终与各自的观测点集对应。

连续相位拟厚在共同合成案例中有 1 例相位歧义，仍计入全部 24 例的平均误差。表41保存原型逐例结果，低于显示精度的误差用上界表示，不将困难案例从均值中移除。

表 41 共同合成案例的逐例相对误差

案例	真厚度 (μm)	峰序 (%)	界面场 (%)	相位 (%)
1	3	< 0.0001	< 0.0001	0.0002
2	3	0.2030	0.0050	0.0114
3	3	0.0124	< 0.0001	0.0006
4	3	0.0883	0.0027	0.0134
5	3	< 0.0001	< 0.0001	0.0001
6	3	0.0532	0.0046	0.0508
7	3	< 0.0001	< 0.0001	0.0003
8	3	0.0983	0.0044	0.0056
9	8	< 0.0001	< 0.0001	< 0.0001
10	8	0.0021	0.0021	0.0098
11	8	0.0007	< 0.0001	< 0.0001
12	8	0.0106	0.0009	0.0012
13	8	< 0.0001	< 0.0001	< 0.0001
14	8	0.0037	0.0012	0.0023
15	8	< 0.0001	< 0.0001	< 0.0001
16	8	0.0078	0.0015	0.0006
17	20	0.0001	< 0.0001	< 0.0001
18	20	0.0004	0.0004	0.0003
19	20	0.0002	< 0.0001	< 0.0001
20	20	0.0003	< 0.0001	0.0002
21	20	0.0004	< 0.0001	< 0.0001

案例	真厚度 (μm)	峰序 (%)	界面场 (%)	相位 (%)
22	20	0.0009	0.0002	0.0012
23	20	0.0001	< 0.0001	< 0.0001
24	20	< 0.0001	0.0005	0.0003

歧义案例的主候选为 $3.0000\ \mu\text{m}$ ，另一个近优候选为 $36.8799\ \mu\text{m}$ 。主候选接近真值与候选是否唯一是不同问题；保留这对候选，才能解释为何低平均误差仍需相位识别检查。

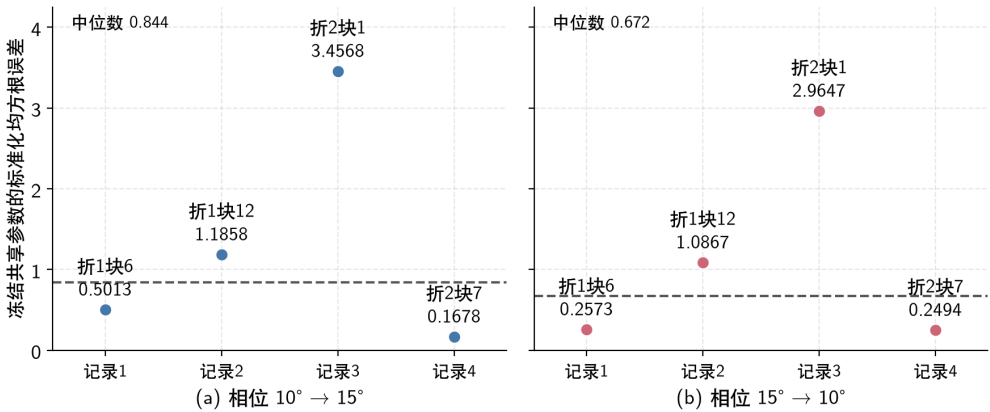
碳化硅五折厚度横跨 $5.5322\text{--}13.5109\ \mu\text{m}$ ，表42列出各折测试块与谱形误差；图15则从冻结另一角的光学参数检验共享厚度。切段和换角作用于不同的信息来源，因此两者分别保留。

表 42 碳化硅五折厚度及留段误差

折号	测试块号	厚度 (μm)	标准化误差
1	1、2	13.3683	1.9693
2	3、4	10.4926	0.6847
3	5、6	6.5335	0.3407
4	7、8	13.5109	0.4882
5	9、10	5.5322	1.2048

五折搜索均取得有效参数，但换一段连续光谱，最合适的厚度分支仍会移动。这种变化解释了共同厚度约束的作用边界：它要求两角共用物理量，却不能凭空补齐未知的折射率曲线。

共享厚度与折射参数：来自双角训练联合拟合，随后固定



来源角度
只冻结该角相位

目标角度
重估有限基线与幅值，预测留出波段

图 15 双角参数固定后的跨角响应核对

完整场的总体误差为 1.0231，高于两束的 0.9812；表43逐块列出同一参数搜索设置下的差值，并与图12区分数值核对和图形展示。差值统一取两束误差减完整场误差，正值表示完整场在该块的预测误差较低。

表 43 全部测试块的完整往返与两束误差

附件	折号	块号	两束误差	完整往返	误差差值
1	1	6	0.1290	0.1287	+0.0003
1	1	12	0.1455	0.1460	-0.0005
1	2	1	2.0936	2.1254	-0.0319
1	2	7	0.1299	0.1251	+0.0048
2	1	6	0.3561	0.3504	+0.0057
2	1	12	0.2853	0.2860	-0.0007
2	2	1	2.4656	2.4803	-0.0147
2	2	7	0.2059	0.2013	+0.0046
3	1	6	0.1401	0.0982	+0.0419
3	1	12	0.2461	0.1861	+0.0600
3	2	1	4.1345	4.5085	-0.3740
3	2	7	0.3344	0.3937	-0.0592
4	1	6	0.3136	0.2514	+0.0621
4	1	12	0.1677	0.1115	+0.0563
4	2	1	4.4083	4.7480	-0.3397

表 43 全部测试块的完整往返与两束误差（续）

附件	折号	块号	两束误差	完整往返	误差差值
4	2	7	0.1433	0.2289	-0.0856

负差值同时出现在硅与碳化硅，材料、角度和区段都会改变比较方向。上述差值衡量反射率预测，对应表11的两材料厚度处理，但不将局部拟合改善直接换算为厚度精度提升。

相位歧义、参考折射率边界和重复相位支持不足，各自对应不同的失败原因。表44连同硅原型的反向收益保存实际记录；其中曾把参考折射率减小 3% 与触及物理边界后的取值混为一谈，后来将严格扰动和边界情景分开，避免用另一种条件冒充原先的扰动。

表 44 实际尝试中的歧义与失效记录

尝试对象	实际现象	保留内容	解释方式
合成相位路线	近优厚度不唯一	保留全部案例	候选成对列出
实测折射率扰动	请求值越过边界	严格与边界分开	边界不并入范围
原型周期形状	重复相位支持弱	保留有限数值	形状与机理分开
原型硅往返场	分材料收益反向	保留硅误差	逐块比较方向

这些记录区分了“算出数值”与“识别唯一厚度”两件事，也保留了复杂路线局部变差的证据。路线和失效对象明确后，区间检验进一步核对预测带实际容纳了多少观测。

8.4.4 区间计数：反射率覆盖与厚度范围分开

反射率区间检查的是未参与拟合和校准的观测值是否落在预测带内；厚度范围则记录切分与光学条件改变后的取值。两类量分别保存分子、分母和参与比较的对象，以便回查表13和表14。

碳化硅正式反射率测试共有 320 点，其中 157 点落入预测带，经验覆盖率为 49.06%；图9给出局部覆盖形态，表45补全分角度计数。预测带的半宽取校准块绝对残差的固定经验分位，覆盖率则由测试块另行计算。

问题二实际采用 75% 校准分位；问题三采用 90% 校准分位。前者没有设定名义覆盖保证，因此两问覆盖率既要结合各自的预测带构造解释，也要保持反射率与厚度这两类对象的区分。

完整往返的反射率测试共有 640 点，其中 473 点被预测带覆盖，经验覆盖率为 73.91%。表45按材料与角度拆开这个总数，避免把某一材料较宽的预测带当成所有观测都得到同等解释。

表 45 反射率测试覆盖的分材料分角计数

问题	材料	角度	覆盖点	测试点	覆盖 (%)
问题三	硅	10°	119	160	74.38
问题三	硅	15°	123	160	76.88
问题三	碳化硅	10°	132	160	82.50
问题三	碳化硅	15°	99	160	61.88
问题二	碳化硅	10°	84	160	52.50
问题二	碳化硅	15°	73	160	45.63
问题二	合计	—	157	320	49.06
问题三	合计	—	473	640	73.91

这些分母统计的是连续测试块内的反射率观测，而非独立晶圆数。厚度随条件改变的范围不参与覆盖计数；它们在表46中按形成方式另列，其中碳化硅的范围为 0.6367–18.1064 μm ，来自切分、光学剖面 and 合法参数情景的并集。

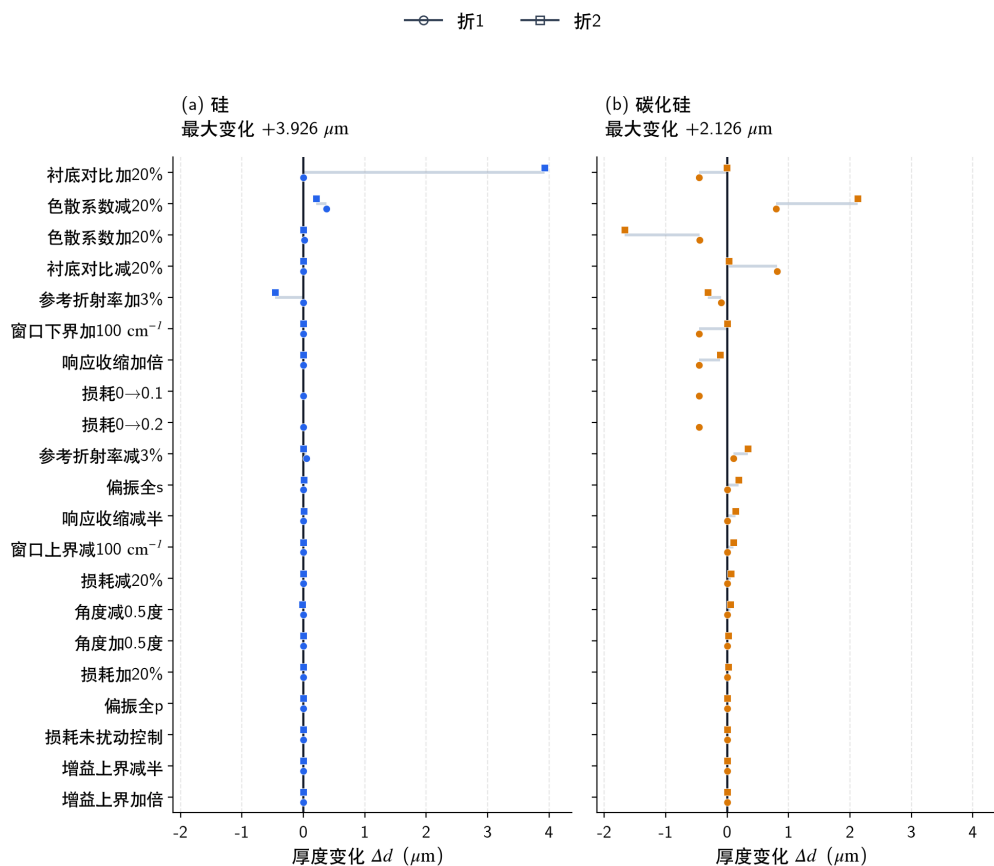
表 46 厚度范围与形成对象分别对应

对象	下限 (μm)	上限 (μm)	范围形成方式
问题一合成扰动	6.6622	10.0142	同一锚例重估
问题二碳化硅	0.6367	18.1064	切分与情景并集
问题三硅两折	1.7322	16.0424	连续留段折间范围
问题三硅包络	1.7246	18.1553	非概率情景联合包络
问题三碳化硅	0.6367	18.1064	沿用问题二基准

折间范围来自重估条件的变化，主窗口全量拟合则使用另一组观测对象：硅每角保留 5392 个原始点，给出的厚度为 2.1651 μm 。同段两束值为 2.3193 μm ，完整往返减两束为 -0.1542 μm 。

硅两折相差 14.3102 μm ，完整包络为 1.7246–18.1553 μm 。全量值、两折跨度与包含近优候选和扰动的情景包络各有不同统计对象。

历史窗口扰动保留原训练块成员并取窗口交集；表47列出当时参与重估的训练点数与厚度，与图16的原参数扰动对象对应。点数是各折训练源行的数量，不再把整个切窗样本数写成实际训练点数；该历史表中的范围不代表本次条件包络端点。



变化 = 重估厚度 - 各折基准；80项已重估，4项绝对加损耗因基准非零而不适用。

图 16 两材料厚度随参数扰动发生偏移

表 47 历史窗口扰动的点数与重估厚度

材料	折号	窗口 (cm^{-1})	训练点数	厚度 (μm)
硅	1	1300–3800	273	1.7345
硅	1	1200–3700	292	1.7321
硅	2	1300–3800	292	16.0426
硅	2	1200–3700	273	16.0424
碳化硅	1	1300–3800	273	2.7611
碳化硅	1	1200–3700	292	3.2168
碳化硅	2	1300–3800	292	10.6616
碳化硅	2	1200–3700	273	10.7635

下界内移时按原始波数筛选训练成员，未在端点外插观测。各折训练源行分别保留，窗口变化和区段切分据此具有明确的样本对象。

上界内移采用同一交集规则；重估厚度对照未扰动的同折解，而不与另一折或全量拟合混合。

历史参数与窗口对照共列 84 项情景，其中 80 项完成重估，4 项因适用条件不满足而单列。表48保留原材料和折次的厚度变化；这些差值相对于当时的同折完整往返解，区别于本次同起点控制差，碳化硅的往返对照与问题二采用值也分别承担不同任务。

表 48 历史扰动的厚度变化量（微米）

扰动情景	硅首折	硅次折	碳化硅首折	碳化硅次折
有效损耗未扰动控制	+0.0000	+0.0002	+0.0000	+0.0000
有效损耗减 20%	+0.0000	+0.0002	+0.0000	+0.0648
有效损耗加 20%	+0.0000	+0.0002	+0.0000	+0.0164
有效损耗绝对增加 0 →0.1	+0.0003	不适用	-0.4553	不适用
有效损耗绝对增加 0 →0.2	+0.0006	不适用	-0.4552	不适用
衬底对比减 20%	+0.0006	+0.0026	+0.8142	+0.0303
衬底对比加 20%	+0.0000	+3.9259	-0.4552	-0.0037
参考折射率减 3%	+0.0535	+0.0002	+0.0997	+0.3370
参考折射率加 3%	+0.0000	-0.4571	-0.0939	-0.3168
色散系数减 20%	+0.3767	+0.2068	+0.7962	+2.1261
色散系数加 20%	+0.0152	+0.0000	-0.4521	-1.6720
偏振全 s	+0.0000	+0.0063	+0.0000	+0.1841

表 48 历史扰动的厚度变化量（微米，续）

扰动情景	硅首折	硅次折	碳化硅首折	碳化硅次折
偏振全 p	+0.0000	-0.0001	-0.0002	+0.0000
入射角同时减 0.5 度	-0.0001	-0.0159	-0.0002	+0.0509
入射角同时加 0.5 度	+0.0001	+0.0000	+0.0000	+0.0208
窗口下界加 100	+0.0024	+0.0002	-0.4557	+0.0000
窗口上界减 100	+0.0000	+0.0000	+0.0000	+0.1019
响应收缩减半	+0.0001	+0.0064	+0.0002	+0.1342
响应收缩加倍	-0.0001	+0.0002	-0.4555	-0.1147
标准化增益上界减半	+0.0000	+0.0002	+0.0000	+0.0000
标准化增益上界加倍	+0.0000	+0.0002	+0.0000	+0.0000

扰动结果同时保留变大、变小与显示精度下不变的取值；它们记录特定光学条件附近的重估响应，而不是给观测增加独立重复次数。源附件、比较路线、失败原因、覆盖计数与有效点数至此逐一对应，正文的各项结论均可回到相同对象核查。

8.4.5 补充图证：场展开与色散响应

场展开的两种表达在图17中逐点对照，比较对象为合场强度与交叉项展开。

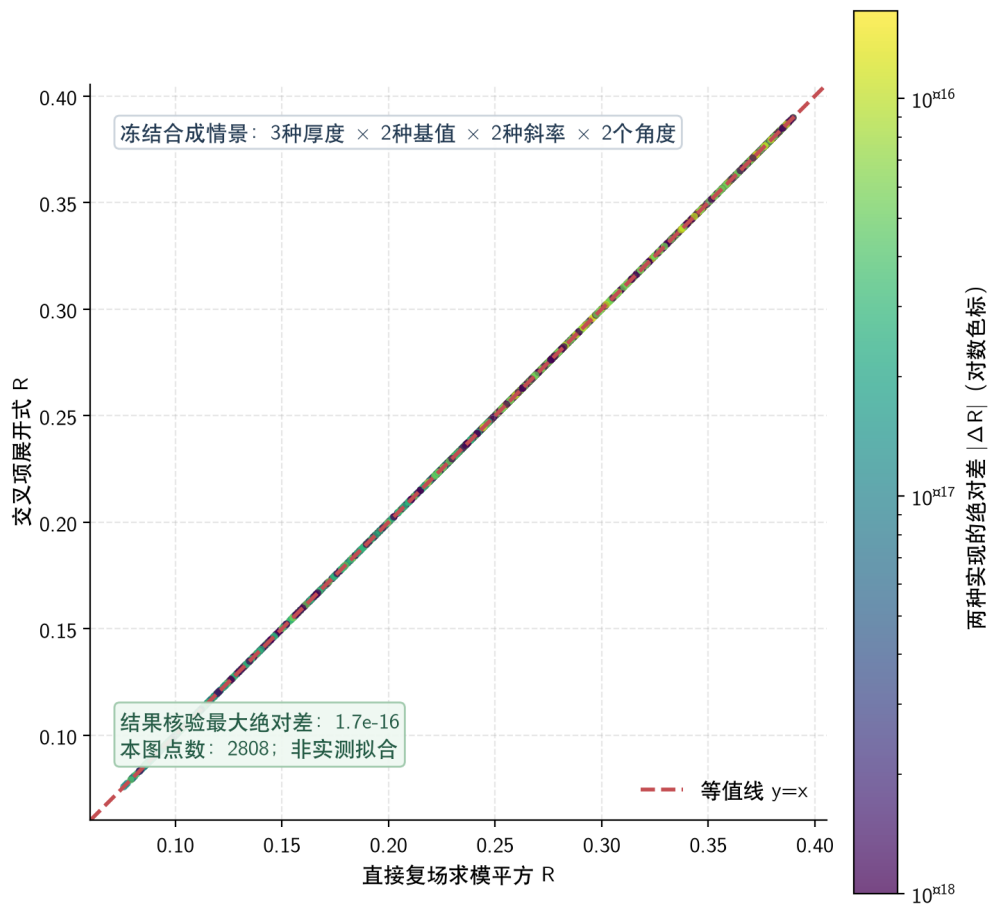


图 17 两种强度展开实现的逐点一致性

图18呈现折射变化对局部峰距的影响，与常折射率近似比较。

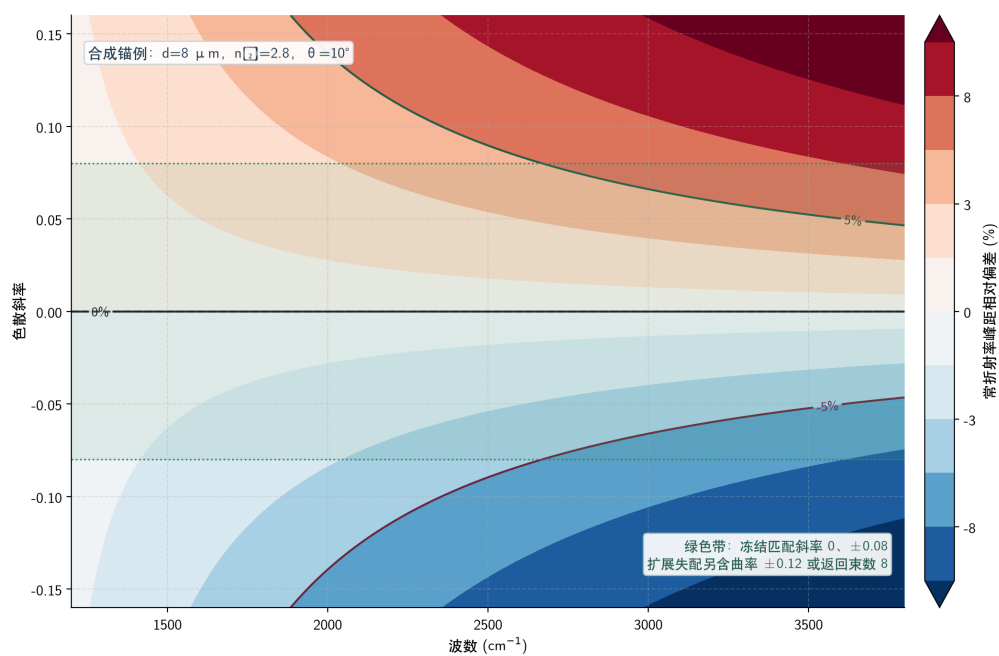


图 18 色散斜率对局部峰距偏差的影响